Kubernetes 23 Topics – 460 Interview Q&A

1. Kubernetes Architecture Basics — 20 Questions

1. Kubernetes Architecture ante enti?
Ans:
Kubernetes Architecture ante Kubernetes Cluster lo different components ela organize ayi, okadanitho okati communicate chestu applications ni manage chestayo aa overall structure ni Kubernetes Architecture antam.
Kubernetes Cluster lo mainly **Control Plane** and **Worker Nodes** untayi.
Control Plane cluster ni manage chestundi. Applications ekkada run avvali, enni Pods run avvali, Pod fail ayithe replacement create cheyyala lanti decisions teesukuntundi.
Worker Nodes actual ga application workloads ni run chestayi. Worker Node lo Pods run avuthayi.
Example:

```
          Kubernetes Cluster
                |
        -------------------
        |           |
  Control Plane     Worker Nodes
        |           |
  Cluster Management    Pods
                |
            Application
```

Suppose nee application ki 3 Pods kavali anuko. Nuvvu Deployment create chestav. Control Plane desired state ni understand chesi, Worker Nodes meeda required Pods run ayye la Kubernetes manage chestundi.
Oka Pod fail ayithe, Kubernetes current state ni observe chesi, required count maintain cheyyadaniki replacement Pod create cheyyachu.
Simple ga:
Kubernetes Architecture = **Control Plane cluster ni manage chestundi + Worker Nodes applications ni run chestayi.**

---

2. Kubernetes lo Control Plane ante enti? Daani main components enti?
Ans:
Control Plane anedi Kubernetes Cluster ki **brain laga** work chestundi. Cluster lo em jaragali, Pods ekkada run avvali, desired state enti, cluster state correct ga unda lanti management decisions Control Plane handle chestundi.
Control Plane lo important components:

- **API Server** — Kubernetes ki main entry point. kubectl and other components API Server tho communicate chestayi.
- **etcd** — Cluster state and configuration information ni store chestundi.
- **Scheduler** — New Pod ni ye Worker Node meeda run cheyyalo decide chestundi.
- **Controller Manager** — Current state ni desired state tho compare chesi required actions teesukuntundi.

Example:

```
kubectl
  |
  ↓
API Server
  |
  ↓
etcd
  |
  ↓
Scheduler
  |
  ↓
Worker Node
  |
  ↓
Pod
```

Suppose Deployment ki 3 replicas kavali. Oka Pod fail ayithe Controller Manager current state 2 replicas ani identify chesi, desired state 3 kabatti replacement Pod create cheyyadaniki process initiate chestundi.
Simple ga:
Control Plane = **Kubernetes Cluster ni control, manage and coordinate chese brain.**

---

3. Kubernetes lo Worker Node ante enti?
Ans:
Worker Node anedi Kubernetes Cluster lo **actual application workloads run ayye machine**. Idi physical server, virtual machine, or cloud instance ayyi undachu.
Worker Node meeda mainly Pods run avuthayi. Pod lopala containers run avuthayi.
Worker Node lo important components:

- **Kubelet**
- **Container Runtime**

- **Kube-proxy** or service networking implementation
- **Pods**

Example:

```
Worker Node
  |
  |--- Kubelet
  |
  |--- Container Runtime
  |
  |--- Kube-proxy
  |
  |--- Pod
      |
      |--- Container
      |
      |--- Application
```

Suppose nee Kubernetes Cluster lo 3 Worker Nodes unnayi.
Worker Node 1 → Pod A
Worker Node 2 → Pod B
Worker Node 3 → Pod C
Control Plane scheduling decisions teesukuntundi, kani application actual ga Worker Node lo run avuthundi.
Simple ga:
Worker Node = **Applications/Pods actual ga run ayye machine.**

---

4. Control Plane and Worker Node madhya difference enti?
Ans:
Control Plane and Worker Node rendu Kubernetes Cluster lo important, kani rendu different responsibilities perform chestayi.
Control Plane cluster ni manage chestundi.
Worker Node applications ni run chestundi.
Example:

```
Control Plane
   |
   |-- API Server
   |-- Scheduler
   |-- Controller Manager
   |-- etcd
   |
   ↓
Worker Nodes
   |
   |-- Pods
   |-- Containers
   |-- Applications
```

Suppose nuvvu oka application deploy chesav.
Control Plane:
"Ee application ki 3 Pods kavali"
"Ee Pod ye Node meeda run avvali?"
"Current state correct ga unda?"
lanti decisions/manage chestundi.
Worker Node:
"Pod ni run chestha"
"Container ni start chestha"
"Application ni execute chestha"
Simple ga:
Control Plane = **Manager/Brain**
Worker Node = **Application run chese worker**

---

5. Kubernetes API Server ante enti?
Ans:
Kubernetes **API Server** anedi Kubernetes Cluster ki main communication point. kubectl, Kubernetes components, applications and automation tools Kubernetes tho interact avvadam mostly API Server dwara jarugutundi.
Nuvvu kubectl command execute chesinappudu direct ga Worker Node tho matladavu. Usually request API Server ki velthundi.
Example:

```
kubectl get pods
```

Flow:

```
User
 |
 ↓
kubectl
 |
 ↓
```

API Server
 |
 ↓
Kubernetes Cluster State
 |
 ↓
Response
Suppose nuvvu:
kubectl create deployment nginx --image=nginx
run chesav anuko.
kubectl request ni API Server ki pampistundi. API Server request ni validate/authenticate chesi Kubernetes objects ni process chestundi.
Simple ga:
API Server = **Kubernetes Cluster ki main entry point / communication gateway.**

---

6. Kubernetes etcd ante enti? Daani purpose enti?
Ans:
etcd anedi Kubernetes use chese **distributed key-value data store**. Kubernetes Cluster ki important state and configuration information ni store cheyyadaniki idi use avuthundi.
For example:

- Cluster configuration
- Kubernetes objects information
- Desired state
- Current cluster state related information

lanti data etcd lo store avuthundi.
Example:
Nuvvu Deployment create chesav:
Deployment
 |
 ↓
API Server
 |
 ↓
etcd
Kubernetes ki aa Deployment gurinchi information persist avvali. Anduke cluster state information etcd lo maintain chestundi.
Suppose etcd unavailable ayithe Kubernetes control plane functionality severely affect avvachu because cluster state information access problem vastundi.
Simple ga:
etcd = **Kubernetes Cluster state/configuration information ni store chese database-like key-value store.**

---

7. Kubernetes Scheduler ante enti?
Ans:
Scheduler anedi **newly created Pod ni ye Worker Node meeda run cheyyalo decide chese Control Plane component**.
Pod create ayinappudu immediate ga random Node meeda run cheyyadu. Scheduler available Nodes ni evaluate chesi suitable Node ni select chestundi.
Scheduler consider cheyyagalige factors:

- Node resources
- Requests
- Constraints
- Affinity/anti-affinity
- Taints and tolerations
- Other scheduling rules

Example:
Pod
|
↓
Scheduler
|
|--- Node 1 ❌ insufficient resources
|--- Node 2 ❌ taint issue
|--- Node 3 ✅ suitable
|
↓
Node 3
Suppose Pod ki 2 CPU kavali. Node 1 lo required resources available levu. Scheduler Node 1 ni choose cheyyadu. Suitable Node ni select chestundi.
Simple ga:
Scheduler = **Pod ki suitable Worker Node ni select chese component.**

---

8. Kubernetes Controller Manager ante enti?

Ans:
Controller Manager lo different Kubernetes controllers run avuthayi. Controllers continuously cluster current state ni observe chesi, **desired state** tho compare chestayi.
Difference unte desired state reach avvadaniki actions teesukuntayi.
Example:
Suppose Deployment:
Desired Pods = 3
Current Pods = 2
Controller observe chesi:
2 ≠ 3
ani identify chestundi.
Required replacement Pod create ayye process ni trigger chestundi.
After some time:
Desired = 3
Current = 3
Appudu state match avuthundi.
Simple ga:
Controller Manager = **Cluster ni continuously observe chesi desired state maintain cheyyadaniki controllers ni run chese component.**

---

9. Kubernetes lo Kubelet ante enti?
Ans:
kubelet anedi prati Worker Node meeda run ayye agent. Daani main responsibility **Node meeda assigned Pods and containers properly run avuthunnaya ani ensure cheyyadam**.
Kubelet API Server tho communicate chestundi and Pod specifications ni follow chestundi.
Example:
Control Plane
  |
  ↓
API Server
  |
  ↓
Kubelet
  |
  ↓
Container Runtime
  |
  ↓
Container
Suppose oka Pod Worker Node 1 ki schedule ayindi.
Kubelet aa Pod specification ni observe chesi container runtime tho required containers create/start cheyyistundi.
Container crash ayithe, configured Pod behavior prakaram kubelet recovery actions ki contribute chestundi.
Simple ga:
Kubelet = **Worker Node lo Pods/containers ni manage chese Kubernetes agent.**

---

10. Container Runtime ante enti? Kubernetes lo enduku kavali?
Ans:
Container Runtime anedi **containers ni actually run chese software**.
Kubernetes itself container ni directly execute cheyyadu. Kubelet container runtime tho communicate chesi containers create, start, stop and manage cheyyistundi.
Modern Kubernetes environments lo **CRI-compatible container runtimes**, such as containerd or CRI-O, commonly use chestaru.
Flow:
Kubelet
 |
 ↓
Container Runtime
 |
 ↓
Container
 |
 ↓
Application
Suppose Pod lo Nginx container undi.
Kubelet:
"Naku ee container run cheyyali"
ani container runtime ki required instructions istundi.
Runtime actual container execution handle chestundi.
Simple ga:
Container Runtime = **Containers ni actual ga run chese software.**

---

11. Kubernetes lo kube-proxy ante enti?

Ans:
kube-proxy traditionally Worker Nodes meeda run ayye component. Kubernetes **Services** kosam network traffic forwarding/load-balancing rules maintain cheyyadaniki help chestundi.
Suppose Service ki 3 backend Pods unnayi.

```
Client
  |
  ↓
Service
  |
  |------ Pod 1
  |------ Pod 2
  |------ Pod 3
```

Service traffic correct backend Pods ki reach avvadam kosam Node-level networking rules maintain cheyyabadutayi.
Modern Kubernetes networking implementations konni environments lo kube-proxy functionality ni different ways lo replace or integrate cheyyachu, but traditional Kubernetes architecture discussions lo kube-proxy important component ga consider chestaru.
Simple ga:
kube-proxy = **Kubernetes Service traffic ni backend Pods ki route cheyyadaniki Node-level network rules maintain cheyyadaniki help chese component.**

---

12. Kubernetes lo Pod create ayye process enti?
Ans:
Nuvvu Pod create chesinappudu immediate ga container run avvadu. Kubernetes lo multiple components coordinate ayi Pod ni Node meeda schedule chesi run chestayi.
Example:
kubectl apply -f pod.yaml
Flow roughly:

```
kubectl
  |
  ↓
API Server
  |
  ↓
Pod information stored
  |
  ↓
Scheduler
  |
  ↓
Suitable Worker Node
  |
  ↓
Kubelet
  |
  ↓
Container Runtime
  |
  ↓
Container
  |
  ↓
Pod Running
```

First API Server request receive chestundi. Scheduler suitable Node select chestundi. Aa Node lo kubelet Pod specification ni observe chesi container runtime tho container run cheyyistundi.
Simple ga:
Pod creation flow = **kubectl → API Server → Scheduler → Worker Node → Kubelet → Container Runtime → Container.**

---

13. Kubernetes lo Desired State and Current State ante enti?
Ans:
Kubernetes architecture lo **Desired State** ante manam Kubernetes ki "ela undali" ani specify chesina state.
Current State ante cluster lo presently actually em jarugutundo.
Kubernetes controllers current state ni desired state ki match cheyyadaniki continuously work chestayi.
Example:
Nuvvu Deployment lo:
replicas: 3
set chesav.
So:
Desired State = 3 Pods
Initially:
Current State = 3 Pods
Oka Pod crash ayindi anuko:
Desired State = 3

Current State = 2
Kubernetes difference ni identify chesi replacement Pod create cheyyadaniki work chestundi.
After recovery:
Desired State = 3
Current State = 3
Simple ga:
Desired State = **Manam kavali ani cheppindi.**
Current State = **Present ga cluster lo unnadi.**

---

14. Kubernetes lo Declarative approach ante enti?
Ans:
Kubernetes mainly **declarative approach** follow chestundi. Ante manam application ni step-by-step ela run cheyyalo cheppadam kanna, **final ga ela undali** ani Kubernetes ki specify chestam.
Kubernetes aa desired state ni reach avvadaniki required actions automatically perform chestundi.
Example:
replicas: 3
Nuvvu:
"3 Pods kavali"
ani specify chestav.
Kubernetes:
Current = 2
Desired = 3
ani identify chesi missing Pod ni create cheyyadaniki work chestundi.
Simple ga:
Declarative approach = **"Ela cheyyalo" kanna "final ga ela undalo" Kubernetes ki cheppadam.**

---

15. kubectl **Kubernetes Architecture lo ekkada untundi?**
Ans:
kubectl anedi Kubernetes Cluster ni manage cheyyadaniki use chese **command-line tool**. Idi Control Plane component kaadu.
User/admin Kubernetes API Server tho interact avvadaniki kubectl use chestaru.
Example:
kubectl get pods
kubectl get nodes
kubectl apply -f deployment.yaml
kubectl describe pod mypod
Flow:
Administrator
|
↓
kubectl
|
↓
API Server
|
↓
Kubernetes Cluster
Suppose nuvvu:
kubectl get nodes
run chesav.
kubectl API Server ni request chestundi. API Server cluster information retrieve chesi response istundi.
Simple ga:
kubectl = **Kubernetes cluster ni command line nunchi interact/manage cheyyadaniki client tool.**

---

16. Kubernetes Cluster lo API Server enduku central component laga consider chestam?
Ans:
API Server Kubernetes components madhya **standard communication interface** laga work chestundi.
Users, kubectl, controllers, scheduler and other Kubernetes clients API Server tho communicate chestayi.
Direct ga every component every other component tho custom communication maintain cheyyalsina requirement reduce chestundi.
Example:
kubectl
|
↓
API Server
/ | \
/ | \
Scheduler Controllers Other Clients
|
↓
etcd

Suppose scheduler ki Pod scheduling information kavali. Scheduler Kubernetes API ni use chesi required cluster objects/information ni access chestundi.
Simple ga:
API Server = **Kubernetes ecosystem lo main communication hub/interface.**

---

17. Kubernetes lo Master Node ane term and Control Plane same aa?
Ans:
Conceptually Kubernetes lo older terminology lo **Master Node** ane term commonly use chesevaru. Modern Kubernetes documentation and interviews lo **Control Plane** ane term preferred.
Control Plane lo Kubernetes management components untayi.
Historically Control Plane components oka dedicated master node lo run ayye architecture common. Modern clusters lo Control Plane components multiple machines lo run chesi **high availability** achieve cheyyachu.
Example:
Old terminology:
Master Node
|
|--- API Server
|--- Scheduler
|--- Controller Manager
|--- etcd
Modern terminology:
Control Plane
|
|--- API Server
|--- Scheduler
|--- Controller Manager
|--- etcd
Simple ga:
Master Node = **older/common terminology**
Control Plane = **modern preferred terminology.**

---

18. Kubernetes Control Plane itself high available ela chestaru?
Ans:
Production Kubernetes Cluster lo Control Plane single machine meeda depend ayithe aa machine failure valla cluster management affect avvachu. Anduke **multiple Control Plane instances** use chesi high availability design chestaru.
For example:
Control Plane 1
Control Plane 2
Control Plane 3
|
↓
Worker Nodes
API Server multiple instances run avvachu. etcd kuda multiple members tho cluster ga run cheyyachu.
Important point enti ante **HA setup ante simply 3 Control Plane servers create cheyyadam matrame kaadu**. Components proper ga configure cheyyali, especially etcd quorum and API-server access.
Example:
3 Control Plane setup lo:
CP1 → API Server
CP2 → API Server
CP3 → API Server
Oka Control Plane machine unavailable ayina remaining Control Plane instances cluster management continue cheyyadaniki allow chestayi, assuming HA correctly configured.
Simple ga:
Control Plane HA = **Multiple Control Plane instances use chesi single point of failure ni reduce cheyyadam.**

---

19. Kubernetes Architecture lo oka Pod fail ayithe complete flow enti?
Ans:
Suppose Deployment lo 3 Pods unnayi.
Pod 1 → Running
Pod 2 → Running
Pod 3 → Running
Pod 2 fail ayindi anuko.
Current state:
Desired = 3
Current = 2
Kubernetes controllers difference ni identify chestayi. Replacement Pod create cheyyadaniki Kubernetes API through request/state update jarugutundi.
New Pod scheduling avasaram unte Scheduler suitable Worker Node ni select chestundi.
Aa Node lo kubelet container runtime tho required container ni run cheyyistundi.
Flow:
Pod failure

↓
Controller observes state
↓
Replacement Pod needed
↓
Scheduler selects Node
↓
Kubelet
↓
Container Runtime
↓
New Pod
Final ga:
Desired = 3
Current = 3
Simple ga:
Kubernetes architecture lo **failure ni observe → difference identify → replacement create → suitable Node lo run** chese automation untundi.

---

20. Real-time scenario: Kubernetes application request nunchi Pod varaku complete architecture flow explain cheyyandi.
Ans:
Suppose oka user browser lo nee application ni access chestunnadu.
User request Kubernetes environment loki enter avuthundi. Depending on the application architecture, request Ingress/LoadBalancer/Service lanti networking components dwara backend Pod ki reach avvachu.
Backend Pod Worker Node meeda run avuthundi.
At the same time, application deployment and management Control Plane handle chestundi.
High-level flow:
User
|
↓
Load Balancer / Ingress
|
↓
Service
|
↓
Backend Pod
|
↓
Container
|
↓
Application
Cluster management side:
Administrator
|
↓
kubectl
|
↓
API Server
|
↓
Control Plane
| | |
| | |
etcd Scheduler Controllers
|
↓
Worker Node
|
Kubelet
|
Container Runtime
|
Pod
Suppose application ki 3 replicas kavali:
Deployment
↓
3 Pods desired
↓

Scheduler
↓
Worker Nodes
↓
Kubelet
↓
Containers
Oka Pod fail ayithe controllers replacement Pod ni maintain cheyyadaniki work chestayi.
User side lo Service stable endpoint provide chestundi, backend Pods change ayina traffic available matching Pods ki route avvadaniki help chestundi.
Simple ga:
Control Plane = Manage & Decide
Worker Node = Run Applications
Pod = Run Workload
Service = Provide Stable Network Access
Idi overall Kubernetes Architecture ni interview lo explain cheyyadaniki **end-to-end flow**.

---

2. Pods — 20 Questions

1. Kubernetes lo Pod ante enti?
Ans:
Pod anedi Kubernetes lo **smallest deployable unit**. Kubernetes directly containers ni deploy/manage cheyyakunda, containers ni **Pod** lopala run chestundi.
Usually oka Pod lo oka main application container untundi. Kaani requirement batti multiple tightly-coupled containers kuda same Pod lo run cheyyachu.
Pod lo unna containers:

- Same network namespace share chestayi.
- Same Pod IP use chestayi.
- Volumes ni share chesukovachu.
- Usually tightly coupled functionality kosam use chestaru.

Example:
Pod
|
|--- Container
|
|--- Nginx Application
Multiple containers example:
Pod
|
|--- Application Container
|
|--- Sidecar Container
Rendu containers same Pod lo unte avi same network context lo communicate cheyyagalavu.
Simple ga:
Pod = **Kubernetes lo containers run ayye smallest deployable unit.**

---

2. Pod and Container madhya difference enti?
Ans:
Container anedi application ni package chesi run chese isolated environment. Pod anedi Kubernetes lo **one or more containers ni group chesi manage cheyyadaniki use chese unit**.
Kubernetes perspective lo:
Kubernetes
|
Pod
|
|--- Container
|--- Container
Suppose Nginx application run cheyyali.
Container:
Nginx Container
Kubernetes lo aa container ni run cheyyadaniki:
Pod
|
└── Nginx Container
use chestam.
Pod networking and storage lanti shared resources provide chestundi.
Simple ga:
Container = **Application run chese unit**
Pod = **Kubernetes containers ni manage/deploy chese unit**

---

3. Oka Pod lo multiple containers run cheyyacha?
Ans:
Yes. Kubernetes Pod lo **one or more containers** run cheyyachu.
Kaani multiple containers ni same Pod lo petti run cheyyadam app design requirement meeda depend avuthundi.
Same Pod lo unna containers:

- Same network namespace share chestayi.
- Same Pod IP use chestayi.
- localhost dwara okadanitho okati communicate cheyyagalavu.
- Same volumes share chesukovachu.

Example:
Pod
|
|--- Application Container
|
|--- Logging Sidecar Container
Application container logs generate chestundi. Sidecar container aa logs ni collect chesi external logging system ki send cheyyachu.
Simple ga:
Yes, **one Pod lo multiple containers run cheyyachu**, especially tightly coupled containers kosam.

---

4. Kubernetes lo one container per Pod better aa? Multiple containers per Pod better aa?
Ans:
General ga **one main application container per Pod** common and simple approach. Multiple containers same Pod lo only tightly coupled ga undi, same lifecycle/network/storage share cheyyalsina requirement unte use chestam.
For example:
Pod 1
|
└── Backend Container
Idi normal application case.
Multiple containers:
Pod
|
|--- Application
|
└── Sidecar
Application and sidecar strongly coupled ga unte useful.
Kaani completely independent applications ni same Pod lo unnecessarily place cheyyakudadhu.
Example:
Frontend and Database ni same Pod lo petti run cheyyadam generally correct design kaadu.
Better:
Frontend Pod
|
Backend Pod
|
Database Pod
Simple ga:
Tightly coupled containers → same Pod
Independent applications → separate Pods

---

5. Pod ki IP address untunda?
Ans:
Yes. Normally Kubernetes lo prati Pod ki **own IP address** assign avuthundi.
Suppose:
Pod 1 → 10.0.0.5
Pod 2 → 10.0.0.6
Pod 3 → 10.0.0.7
Pods network lo communicate cheyyadaniki ee IPs use cheyyachu.
Kaani Pod permanent kaadu. Pod delete/recreate ayithe new Pod ki different IP ravachu.
Example:
Old Pod → 10.0.0.5
↓
Pod deleted
↓
New Pod → 10.0.0.10
Anduke application access kosam direct Pod IP ni normally depend avvakudadhu. Stable access kosam **Service** use chestam.
Simple ga:
Pod ki IP untundi, **kani Pod IP permanent ani guarantee ledu.**

---

6. Pod IP change enduku avuthundi?
Ans:
Pod anedi generally **ephemeral**. Ante Pod permanent machine laga kaadu. Pod delete ayithe replacement Pod create avvachu.

Replacement Pod ki new IP address assign avvachu.
Example:
Initially:
Backend Pod
IP = 10.0.0.5
Pod crash/delete ayindi.
Kubernetes replacement Pod create chesindi:
New Backend Pod
IP = 10.0.0.12
Application logic direct ga 10.0.0.5 ni depend ayithe connection break avvachu.
Anduke:
Client → Service → Pods
pattern use chestam.
Service stable endpoint provide chestundi.
Simple ga:
Pod replace ayithe **new Pod ki new IP ravachu**.

---

7. Pod restart and Pod recreation madhya difference enti?
Ans:
Idi important interview question.
Container restart ante same Pod lo container restart avvadam. Pod itself continue avvachu.
Pod recreation ante old Pod remove ayi, new Pod create avvadam.
Example:
Container crash
↓
Same Pod
↓
Container restart
Kaani:
Pod deleted
↓
New Pod created
↓
New Pod IP
New Pod completely new identity tho create avvachu.
Example:
Old Pod → backend-abc → 10.0.0.5
↓
deleted
↓
New Pod → backend-xyz → 10.0.0.9
So container restart and Pod recreation same thing kaadu.
Simple ga:
Container restart = **same Pod lo container restart**
Pod recreation = **new Pod create avvadam**

---

8. Pod lifecycle states enti?
Ans:
Kubernetes Pod lifecycle lo commonly observe chese Pod phases:

- **Pending**
- **Running**
- **Succeeded**
- **Failed**
- **Unknown**

Pod create chesina immediately Running avvakapovachu.
Example:
Pod Created
↓
Pending
↓
Running
↓
Succeeded / Failed
Pending ante Pod accept ayindi kani required containers inka fully run avvaledu.
Running ante Pod Node ki assigned ayi, containers start ayyi running state lo unnayi or startup process lo unnayi.
Succeeded mostly Job-type workloads successfully complete ayinappudu kanipistundi.
Failed ante containers terminate ayyi failure outcome vachinappudu.
Simple ga:
Pod lifecycle = **Pending → Running → Succeeded/Failed** depending on workload.

---

9. Pod Pending **lo stuck ayithe meaning enti?**

Ans:
Pod Pending lo undadam ante Pod create ayindi kani **fully run avvadaniki required conditions complete kaaledu**.
Common reasons:

- Suitable Node dorakakapovadam
- Insufficient CPU/memory
- Taints/tolerations mismatch
- Affinity rules satisfy kakapovadam
- PVC available kakapovadam
- Image pull mundu scheduling/storage-related issue

Example:
Pod requests:
CPU = 4
Memory = 8Gi
Available Nodes lo required resources levu anuko.
Scheduler suitable Node select cheyyalekapovachu.
Appudu:
Pod
↓
Pending
Troubleshooting:
kubectl describe pod <pod-name>
Events section lo reason chudachu.
Simple ga:
Pending = **Pod create ayindi, kani run avvadaniki required conditions complete kaaledu.**

---

10. Pod Running **ante exact ga enti?**
Ans:
Pod Running ante Pod successfully oka Node ki assigned ayi, Pod lo required containers start ayyi running state lo unnayi or startup process lo unnayi.
Kaani important point: **Pod phase Running undadam application fully healthy ani guarantee kaadu.**
For example:
Pod Phase = Running
Application = Not Ready
Application start avvadaniki inka time kavachu or readiness probe fail avvachu.
Anduke application traffic receive cheyyadaniki **Readiness Probe** important.
Example:
Pod
|
|--- Container → Running
|
└── Readiness → Failed
Pod Running ayina Service traffic aa Pod ki send cheyyakunda undachu, depending on readiness status.
Simple ga:
Running = **Pod execution phase lo undi; application healthy ani automatic guarantee kaadu.**

---

11. Pod lo restartPolicy **ante enti?**
Ans:
restartPolicy anedi Pod lo containers terminate ayinappudu Kubernetes **eppudu restart cheyyali** ane behavior ni define chestundi.
Common values:

- Always
- OnFailure
- Never

Always ante container terminate ayina restart cheyyadaniki Kubernetes try chestundi.
OnFailure ante failure exit code tho terminate ayithe restart chestundi.
Never ante container terminate ayina automatic restart cheyyadu.
Example:

```
spec:
  restartPolicy: OnFailure
```

Suppose Job-type workload lo processing complete ayina taruvatha container exit avuthundi. Every normal completion ni restart cheyyakudadhu. Anduke OnFailure or Never use cases untayi.
Simple ga:
restartPolicy = **Container terminate ayinappudu restart behavior ni control chesedi.**

---

12. Pod lo pause **container ante enti?**
Ans:
Kubernetes Pod implementation lo infrastructure container concept untundi, commonly **pause container** ani pilustaru.
Idi Pod ki shared network namespace lanti infrastructure aspects maintain cheyyadaniki use avuthundi.
Same Pod lo multiple containers unte avi same Pod network namespace share chestayi.
Example:
Pod

|
|--- Pause/Infrastructure
|
|--- App Container
|
└── Sidecar Container

App container and sidecar container same Pod network namespace share chestayi.
So same Pod lo:
App Container
localhost
↕
Sidecar Container
dwara communication possible.
Simple ga:
Pause container = **Pod-level shared infrastructure, especially network namespace, maintain cheyyadaniki Kubernetes/container runtime architecture lo use ayye infrastructure container.**

---

13. Same Pod lo unna two containers ela communicate chestayi?
Ans:
Same Pod lo unna containers **same network namespace** share chestayi. Anduke avi localhost dwara okadanitho okati communicate cheyyagalavu.
Suppose:
Pod
|
|--- Application Container → Port 8080
|
└── Sidecar Container

Sidecar container application ni:
localhost:8080
dwara access cheyyagaladu.
Important point: Same Pod lo containers same IP use chestayi, kabatti **different containers same port ni simultaneously bind cheyyakudadhu**.
Example:
Application → localhost:8080
Sidecar → localhost:8080
ila same port conflict create cheyyachu.
Simple ga:
Same Pod containers = **same network namespace + same Pod IP + localhost communication.**

---

14. Pod lo containers volumes share chesukovacha?
Ans:
Yes. Same Pod lo unna containers **same Volume** mount chesukunte data share chesukovachu.
Example:
Pod
|
|--- App Container
| |
| └── /shared
|
└── Sidecar Container
|
└── /shared

Application container /shared/app.log create chestundi.
Sidecar container same volume mount chesukunte:
/shared/app.log
ni read cheyyagaladu.
Idi logging sidecar pattern lo common concept.
Simple ga:
Same Pod containers **shared Volume use chesi files/data share chesukovachu.**

---

15. Pod YAML structure ela untundi?
Ans:
Pod ni Kubernetes lo YAML manifest dwara define cheyyachu.
Basic structure:

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
```

```
 containers:
   - name: nginx
     image: nginx
```
Important sections:
apiVersion
kind
metadata
spec
apiVersion → Kubernetes API version.
kind → object type, ikkada Pod.
metadata → Pod name, labels lanti information.
spec → Pod ela run avvali ane configuration.
Example flow:
pod.yaml
↓
kubectl apply
↓
API Server
↓
Pod created
Simple ga:
Pod YAML = **Pod ni Kubernetes ki declaratively define chese configuration file.**

---

16. Pod ki labels enduku use chestam?
Ans:
Labels anedi Pod ki attach chese **key-value metadata**. Kubernetes lo Pods ni identify/group/select cheyyadaniki labels very important.
Example:
```
labels:
  app: backend
  env: prod
```
Ippudu multiple Pods ki:
app=backend
label unte Service selector dwara aa Pods ni select cheyyachu.
Example:
Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=frontend
Service:
```
selector:
  app: backend
```
Appudu Service backend Pods ni select chestundi.
Simple ga:
Labels = **Pods/resources ni identify and group cheyyadaniki use chese key-value tags.**

---

17. Pod ni manually create cheyyadam and Deployment dwara create cheyyadam madhya difference enti?
Ans:
Nuvvu direct ga Pod create cheyyachu:
kubectl run nginx --image=nginx
Kaani direct Pod ki automatic **replica management, rollout, rollback** lanti Deployment features undavu.
Deployment use chesthe:
Deployment
↓
ReplicaSet
↓
Pods
Deployment desired replicas maintain cheyyadaniki ReplicaSet ni use chestundi.
Suppose 3 replicas kavali:
Deployment
↓
ReplicaSet
↓
Pod 1
Pod 2
Pod 3
Oka Pod delete ayithe ReplicaSet replacement Pod create cheyyadaniki work chestundi.
Simple ga:
Direct Pod = **simple workload**
Deployment = **production-style replicated application management**

---

18. Oka Pod delete ayithe Kubernetes automatic ga new Pod create chestunda?

Ans:
Always kaadu. Pod ni evaru manage chestunnaru ane dani meeda depend avuthundi.
Direct ga create chesina Pod:
kubectl run nginx --image=nginx
delete ayithe automatic replacement generally create avvadu.
Kaani Deployment/ReplicaSet manage chestunna Pod delete ayithe:
Deployment
↓
ReplicaSet
↓
Pod
ReplicaSet desired replica count maintain cheyyadaniki replacement Pod create chestundi.
Example:
Desired = 3
Current = 3
Pod 2 delete:
Desired = 3
Current = 2
ReplicaSet replacement create chestundi:
Desired = 3
Current = 3
Simple ga:
Controller-managed Pod → replacement possible
Standalone Pod → automatic replacement generally ledu.

---

19. Pod CrashLoopBackOff **ante enti?**
Ans:
CrashLoopBackOff ante container start avuthundi, **crash avuthundi, Kubernetes restart chestundi, malli crash avuthundi**, ila repeated cycle jarugutundi.
Repeated failures valla Kubernetes restart attempts madhya backoff delay increase chestundi.
Flow:
Container Start
↓
Application Crash
↓
Container Restart
↓
Application Crash
↓
Restart
↓
CrashLoopBackOff
Common reasons:

- Application startup error
- Wrong configuration
- Missing environment variable
- Incorrect command
- Application dependency unavailable
- Permission issue
- Wrong Secret/ConfigMap configuration

Troubleshooting:
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
--previous particularly useful because previous crashed container logs ni check cheyyachu.
Example:
Application database connection kosam required environment variable expect chestundi. Variable missing ayithe application startup lo crash avvachu.
Simple ga:
CrashLoopBackOff = **Container repeatedly start → crash → restart cycle lo undi.**

---

20. Real-time scenario: Pod Pending **lo undi. Interview lo ela troubleshoot chestav?**
Ans:
First direct ga Pod delete/create cheyyanu. **Reason identify** chestha.
First:
kubectl get pods
Pod status confirm chestha.
Then:
kubectl describe pod <pod-name>

Events section chustha.
Suppose event ila undi:
0/3 nodes are available:
3 Insufficient cpu
Ante problem application code lo kaadu. Scheduler ki suitable Node lo required CPU available ledu.
Appudu Nodes resources check chestha:
kubectl get nodes
kubectl describe node <node-name>
Pod resource requests check chestha:
resources:
  requests:
    cpu: "2"
    memory: "2Gi"
Suppose Pod 2 CPU request chestundi, kani available Nodes lo 2 CPU allocatable resources levu. Appudu Pod Pending lo undachu.
Inko scenario lo:
node(s) had untolerated taint
unte Taints & Tolerations check chestha.
Inko scenario:
node(s) didn't match Pod's node affinity
unte Node Affinity configuration check chestha.
Overall troubleshooting flow:
Pod Pending
  ↓
kubectl get pod
  ↓
kubectl describe pod
  ↓
Check Events
  ↓
Identify reason
  |
  |--- Insufficient resources
  |--- Taint/Toleration
  |--- Affinity
  |--- PVC
  |--- Scheduling issue
  ↓
Fix root cause
  ↓
Pod gets scheduled
  ↓
Running
Simple ga:
Pod Pending ayithe **blind ga restart cheyyakunda** kubectl describe pod → **Events → exact reason identify → root cause fix** cheyyali.

---

3. Deployment / ReplicaSet — 20 Questions

1. Kubernetes lo Deployment ante enti?
Ans:
Deployment anedi Kubernetes lo **stateless applications ni deploy, update, scale, and manage cheyyadaniki** use chese Kubernetes resource.
Deployment direct ga containers ni manage cheyyadu. Deployment oka **ReplicaSet** ni manage chestundi, and ReplicaSet Pods ni manage chestundi.
Example:
Deployment
  |
  ↓
ReplicaSet
  |
  |--- Pod 1
  |--- Pod 2
  |--- Pod 3
Suppose nee application ki 3 replicas kavali anuko.
Deployment lo:
replicas: 3
set chestav.
Deployment ReplicaSet ni create/manage chestundi. ReplicaSet required number of Pods maintain chestundi.
Oka Pod fail ayithe ReplicaSet replacement Pod create cheyyadaniki work chestundi.
Simple ga:
Deployment = **Application deployment, updates, scaling, and rollout ni manage chese higher-level Kubernetes resource.**

---

2. Kubernetes lo ReplicaSet ante enti?
Ans:
ReplicaSet anedi Kubernetes lo **specified number of identical Pods running lo undela maintain chese resource**.
Suppose ReplicaSet ki:
replicas: 3
ani configure chesav.
Ante Kubernetes ki 3 matching Pods kavali.

```
ReplicaSet
  |
  |--- Pod 1
  |--- Pod 2
  |--- Pod 3
```

Oka Pod fail/delete ayithe:
Desired = 3
Current = 2
ReplicaSet replacement Pod create chestundi.
Desired = 3
Current = 3
Simple ga:
ReplicaSet = **Required number of Pod replicas ni maintain chese resource.**

---

3. Deployment and ReplicaSet madhya difference enti?
Ans:
Deployment and ReplicaSet related resources, kani responsibilities different.
Deployment higher-level resource. Application rollout, update, rollback, scaling lanti operations manage chestundi.
ReplicaSet mainly required number of Pods maintain chestundi.
Architecture:

```
Deployment
  |
  ↓
ReplicaSet
  |
  ↓
Pods
```

Suppose application version v1 nunchi v2 ki update cheyyali.
Deployment new ReplicaSet create chesi new version Pods ni manage cheyyagaladu.

```
Deployment
  |
  |--- Old ReplicaSet → v1 Pods
  |
  └── New ReplicaSet → v2 Pods
```

Simple ga:
Deployment = **Rollouts + Updates + Rollbacks + Scaling**
ReplicaSet = **Pod replica count maintain cheyyadam**

---

4. Deployment enduku use chestam? Direct ga Pods create cheyyachu kada?
Ans:
Direct Pods create cheyyachu, kani production applications ki direct Pods manage cheyyadam difficult.
Deployment use chesthe:

- Multiple replicas maintain cheyyachu.
- Application updates controlled ga cheyyachu.
- Rollout status monitor cheyyachu.
- Previous version ki rollback cheyyachu.
- Scaling easy ga cheyyachu.
- Failed Pods ni ReplicaSet replacement tho maintain cheyyachu.

Example:
Direct Pods:
Pod 1
Pod 2
Pod 3
Pod 2 delete ayithe replacement management automatic ga undadu unless another controller manages them.
Deployment:

```
Deployment
  ↓
ReplicaSet
  ↓
Pod 1
Pod 2
Pod 3
```

Pod 2 delete ayithe ReplicaSet replacement create chestundi.
Simple ga:
Deployment = **Production application ni reliably manage cheyyadaniki preferred approach.**

---

5. Deployment lo replicas **ante enti?**
Ans:
replicas ante **simultaneously run avvali ani Kubernetes ki specify chese desired number of Pod instances**.
Example:
spec:
 replicas: 3
Ante:
Desired Pods = 3
Kubernetes approximately ila maintain cheyyadaniki work chestundi:
Deployment
 ↓
ReplicaSet
 ↓
Pod 1
Pod 2
Pod 3
Oka Pod fail ayithe:
Desired = 3
Current = 2
ReplicaSet replacement create chestundi.
Simple ga:
replicas: 3 = **Application ki 3 Pod instances kavali ani meaning.**

---

6. Deployment lo Pod fail ayithe em jaruguthundi?
Ans:
Suppose Deployment ki 3 replicas unnayi.
Pod 1 → Running
Pod 2 → Running
Pod 3 → Running
Pod 2 fail ayindi anuko.
Current state:
Desired = 3
Current = 2
Deployment manage chestunna ReplicaSet ee difference ni identify chesi replacement Pod create chestundi.
ReplicaSet
 |
 |--- Pod 1
 |--- Pod 3
 └── New Pod
Final ga:
Desired = 3
Current = 3
Simple ga:
Deployment → ReplicaSet → **Failed Pod ki replacement Pod maintain chestundi.**

---

7. Deployment and ReplicaSet relationship ela untundi?
Ans:
Deployment and ReplicaSet parent-child relationship laga work chestayi.
Deployment usually ReplicaSet ni create/manage chestundi. ReplicaSet Pods ni create/manage chestundi.
Deployment
 |
 ↓
ReplicaSet
 |
 ↓
Pods
Suppose Deployment version v1 undi:
Deployment
 ↓
ReplicaSet-v1
 ↓
3 Pods
Application image update chesthe Deployment new ReplicaSet create cheyyachu:
Deployment
 |
 |--- ReplicaSet-v1 → old Pods
 |

└── ReplicaSet-v2 → new Pods
Idi rolling update lo important.
Simple ga:
Deployment **ReplicaSets ni manage chestundi**, ReplicaSet **Pods ni manage chestundi.**

---

8. Deployment lo Rolling Update ante enti?
Ans:
Rolling Update anedi application new version ni **all old Pods ni okesari stop cheyyakunda gradually replace cheyyadam**.
Suppose:
v1 → 3 Pods
Nuvvu application ni v2 ki update chestav.
Kubernetes gradual ga:
Old v1 Pod → New v2 Pod
Old v1 Pod → New v2 Pod
Old v1 Pod → New v2 Pod
laga replace cheyyagaladu.
High-level:
Before:
v1 v1 v1

During:
v1 v1 v2
v1 v2 v2
v2 v2 v2

After:
v2 v2 v2
Correct configuration unte application availability maintain cheyyadaniki RollingUpdate strategy help chestundi.
Simple ga:
Rolling Update = **Old version Pods ni gradual ga new version Pods tho replace cheyyadam.**

---

9. Deployment lo Rollback ante enti?
Ans:
New application version deploy chesina tarvatha problems vasthe previous working version ki return avvadanni **Rollback** antam.
Suppose:
v1 → Working
Nuvvu update chesi:
v2 → Deployment
chestav.
Kaani v2 lo bug undi.
Rollback chesthe Deployment previous revision ki return avvadaniki Kubernetes rollout history use chestundi.
Example:
kubectl rollout undo deployment/myapp
Flow:
v1 → Working
↓
v2 → Bug
↓
Rollback
↓
v1 → Working
Simple ga:
Rollback = **Problematic new version nunchi previous stable version ki return avvadam.**

---

10. Kubernetes Deployment lo rollout status ela check chestam?
Ans:
Deployment update ayina tarvatha rollout successful ga complete ayinda leda check cheyyadaniki:
kubectl rollout status deployment/myapp
use cheyyachu.
Example:
Waiting for deployment "myapp" rollout to finish...
deployment "myapp" successfully rolled out
Deployment details kosam:
kubectl get deployment
Pods check cheyyadaniki:
kubectl get pods
ReplicaSets check cheyyadaniki:
kubectl get rs
Simple ga:
kubectl rollout status = **Deployment rollout complete ayinda ani check cheyyadaniki.**

---

11. Deployment rollout history ante enti?
Ans:
Deployment application updates ki related **previous revisions** ni track cheyyagaladu. Idi rollback and deployment history understand cheyyadaniki useful.
Check cheyyadaniki:
kubectl rollout history deployment/myapp
Example:
REVISION CHANGE-CAUSE
1 Initial deployment
2 Updated image
3 Updated configuration
Suppose Revision 3 lo issue vachindi. Previous revision information ni inspect chesi rollback decision teesukovachu.
Specific revision details kosam:
kubectl rollout history deployment/myapp --revision=2
Simple ga:
Rollout history = **Deployment previous revisions/updates ni track cheyyadaniki use ayye history.**

---

12. Deployment lo RollingUpdate **and** Recreate **strategies madhya difference enti?**
Ans:
Deployment lo commonly two strategies:

- **RollingUpdate**
- **Recreate**

RollingUpdate lo old Pods ni gradual ga replace chestu new Pods create chestaru.
v1 v1 v1
↓
v1 v1 v2
↓
v1 v2 v2
↓
v2 v2 v2
Application availability maintain cheyyadaniki idi common strategy.
Recreate lo old Pods ni first terminate chesi, tarvatha new Pods create chestaru.
v1 v1 v1
↓
No Pods
↓
v2 v2 v2
Temporary downtime occur avvachu.
Example:
strategy:
type: Recreate
Simple ga:
RollingUpdate = **Gradual replacement, usually less/no downtime**
Recreate = **Old Pods first remove, then new Pods create**

---

13. maxSurge **ante enti?**
Ans:
maxSurge anedi RollingUpdate time lo **desired replica count kanna temporarily entha extra Pods create cheyyavacho** define chestundi.
Suppose:
replicas: 4
maxSurge: 1
Ante normal desired count 4. Update time lo temporary ga maximum:
4 + 1 = 5 Pods
varaku undavachu.
Example:
Before:
v1 v1 v1 v1

During update:
v1 v1 v1 v1 v2
Tarvatha old Pod remove chestu new version increase chestundi.
Simple ga:
maxSurge = **Rolling update time lo extra ga create cheyyagalige Pod capacity.**

---

14. maxUnavailable **ante enti?**
Ans:
maxUnavailable anedi RollingUpdate time lo **desired replica count nunchi maximum enni Pods temporarily unavailable ga undavacho** define chestundi.
Suppose:
replicas: 4

maxUnavailable: 1
Ante update time lo maximum 1 Pod unavailable undadaniki allow chestundi.
So approximately:
Desired = 4
Minimum available during update = 3
This setting application availability and rollout speed madhya balance create cheyyadaniki useful.
Simple ga:
maxUnavailable = **Rolling update time lo temporarily unavailable ga undavachu ane maximum Pod count/percentage.**

---

15. ReplicaSet Pods ni ela identify chestundi?
Ans:
ReplicaSet ki **selector** untundi. Aa selector ki match ayye Pod labels ni base chesukoni ReplicaSet Pods ni identify/manage chestundi.
Example:

```
selector:
  matchLabels:
    app: backend
Pod:
labels:
  app: backend
```

Ippudu:
ReplicaSet selector → app=backend
Pod label → app=backend
Match avuthundi.
Suppose required replicas:
3
and matching Pods only 2 unnayi.
ReplicaSet replacement Pod create chestundi.
Simple ga:
ReplicaSet = **Selector + Pod labels match chesi required Pods ni manage chestundi.**

---

16. Deployment lo image update ela chestam?
Ans:
Suppose Deployment lo current image:
nginx:1.25
undi.
Nuvvu update chesi:
nginx:1.26
cheyyali anuko.
Command:
kubectl set image deployment/myapp nginx=nginx:1.26
Deployment new Pod template create/update chesi RollingUpdate strategy prakaram new version Pods ni gradually bring up cheyyachu.
Flow:
nginx:1.25
↓
Deployment update
↓
New ReplicaSet
↓
nginx:1.26 Pods
↓
Old ReplicaSet scale down
Simple ga:
Image update = **Deployment Pod template image change → new ReplicaSet → new version Pods rollout.**

---

17. Deployment lo selector **and Pod template labels enduku match avvali?**
Ans:
Deployment ki selector untundi. Aa selector Deployment manage cheyyalsina Pods ni identify chestundi.
Example:

```
selector:
  matchLabels:
    app: frontend

template:
  metadata:
    labels:
      app: frontend
```

Ikkada:
Selector → app=frontend
Pod Label → app=frontend

match avuthayi.
Selector and template labels proper relationship lo undali. Deployment create chestunnappudu invalid/mismatched configuration unte Kubernetes reject cheyyachu.
Simple ga:
Deployment selector = **Ye Pods ni manage cheyyalo identify chestundi.**
Template labels = **Create ayye Pods ki aa identity istayi.**

---

18. Deployment scale up and scale down ela chestam?
Ans:
Deployment lo replicas count change chesi application ni scale cheyyachu.
Suppose current:
replicas: 3
Undi.
Scale up:
kubectl scale deployment myapp --replicas=5
Appudu:
3 Pods → 5 Pods
Scale down:
kubectl scale deployment myapp --replicas=2
Appudu:
5 Pods → 2 Pods
Deployment ReplicaSet desired replica count update chestundi, ReplicaSet actual Pods ni accordingly adjust chestundi.
Flow:
Deployment
↓
replicas changed
↓
ReplicaSet
↓
Pods increase/decrease
Simple ga:
Deployment scaling = **Replica count change chesi application Pods ni increase/decrease cheyyadam.**

---

19. Deployment delete chesthe ReplicaSet and Pods ki em jaruguthundi?
Ans:
Normally Deployment ni delete chesthe, dani ownership relationship prakaram **associated ReplicaSets and Pods kuda delete avvadaniki Kubernetes garbage collection mechanism use chestundi**.
Example:
Deployment
↓
ReplicaSet
↓
Pods
Deployment delete:
Deployment ❌
↓
ReplicaSet ❌
↓
Pods ❌
Kaani exact behavior deletion propagation options batti change cheyyachu. Default cascading deletion lo dependent resources remove avvadam common behavior.
Simple ga:
Deployment delete chesthe normally **dani managed ReplicaSet and Pods kuda cascade ga delete avuthayi.**

---

20. Real-time scenario: Deployment update chesaka new Pods Running **lo unnayi, kani old Pods terminate avvatledu. Ela troubleshoot chestav?**
Ans:
First rollout status check chestha:
kubectl rollout status deployment/myapp
Tarvatha:
kubectl get pods
kubectl get rs
Old and new ReplicaSets identify chestha.
Example:
Old RS → 3 Pods
New RS → 3 Pods
Total 6 Pods unte rollout expected behavior prakaram temporary surge undochu, but continuously old Pods terminate avvakapothe reason investigate cheyyali.
Deployment details:
kubectl describe deployment myapp
Events check chestha.

Deployment strategy check chestha:
strategy:
 type: RollingUpdate
Then maxSurge and maxUnavailable values chustha.
Important ga **Readiness** kuda check chestha. New Pods Ready kakapothe Deployment old Pods ni immediately scale down cheyyakunda availability maintain cheyyadaniki wait cheyyachu.
kubectl describe pod <new-pod>
Readiness probe failure, image issue, application startup issue, resource problem lanti causes investigate chestha.
Overall flow:
Deployment Update
 ↓
New ReplicaSet created
 ↓
New Pods created
 ↓
Are new Pods Ready?
 |
 No ————→ Check Pod / Probe / Events
 |
 Yes
 ↓
Old ReplicaSet scaled down
 ↓
Old Pods terminate
Simple ga:
Old Pods terminate avvakapothe rollout status → get rs → describe deployment → **new Pod readiness/events → rollout strategy (**maxSurge **/** maxUnavailable**)** check chesi root cause identify chestha.

---

4. Services & Networking — 20 Questions

1. Kubernetes lo Networking ante enti?
Ans:
Kubernetes Cluster lo multiple Pods, Services, Nodes untayi. Oka application ki inko application tho communicate avvali. Alage outside nunchi users Kubernetes lo run avuthunna application ni access cheyyali.
Ee **Pod-to-Pod, Pod-to-Service, Service-to-Pod, and external-to-application communication** ni Kubernetes Networking antam.
Example:
User
 |
 ↓
Service
 |
 ↓
Frontend Pod
 |
 ↓
Backend Pod
 |
 ↓
Database Pod
Kubernetes networking proper ga work avvadaniki Pod networking, Services, DNS, CNI networking and external access mechanisms use avuthayi.
Simple ga:
Kubernetes Networking = **Pods, Services, Nodes and Users madhya network communication.**

---

2. Kubernetes lo Pod ki IP address untunda?
Ans:
Yes. Kubernetes lo normally prati Pod ki **unique IP address** assign avuthundi.
Example:
Pod 1 → 10.0.0.5
Pod 2 → 10.0.0.6
Pod 3 → 10.0.0.7
Ee Pod IP tho network communication possible.
Kaani Pod ephemeral. Pod delete ayithe replacement Pod ki different IP ravachu.
Old Pod → 10.0.0.5
 ↓
 Delete
 ↓
New Pod → 10.0.0.10
Anduke applications ni direct Pod IP meeda depend cheyyakunda Service use chestam.
Simple ga:
Pod ki IP untundi, **kani Pod IP permanent kaadu.**

3. Kubernetes Networking lo main basic rules enti?
Ans:
Kubernetes networking design lo few important principles untayi.
Main ga:

- Prati Pod ki own IP address untundi.
- Pod-to-Pod communication possible undali.
- Same Node lo unna Pods communicate cheyyagalali.
- Different Nodes lo unna Pods kuda communicate cheyyagalali.
- Pods NAT lekunda communicate cheyyagalige networking model common.
- Services stable access point provide chestayi.

Example:
Node 1 Node 2

Pod A Pod B
10.0.0.5 ——————→ 10.0.1.5
Pod A and Pod B different Nodes lo unna kuda Kubernetes networking implementation valla communicate cheyyagalavu.
Simple ga:
Kubernetes Networking goal = **Pods anywhere in the cluster can communicate reliably.**

---

4. Pod IP change avuthunte Service enduku use chestam?
Ans:
Suppose nee backend application ki 3 Pods unnayi.
Pod 1 → 10.0.0.5
Pod 2 → 10.0.0.6
Pod 3 → 10.0.0.7
User direct ga Pod IP ni access chesthe problem enti ante Pod delete/recreate ayinappudu IP change avvachu.
Anduke Service create chestam.

```
User
 |
 ↓
Service
 |
 |--- Pod 1
 |--- Pod 2
 └── Pod 3
```

Pod 2 fail ayina:

```
Service
 |
 |--- Pod 1
 |--- Pod 3
 └── New Pod
```

User ki backend Pod IP change ayina matter kaadu.
Simple ga:
Pod IP = **temporary**
Service = **stable access point**

---

5. Kubernetes lo Service ante enti?
Ans:
Service anedi Kubernetes lo **Pods ki stable network endpoint provide chese resource**.
Pods create/delete avvachu, Pod IPs change avvachu. Service matching Pods ni identify chesi traffic ni available Pods ki forward cheyyadaniki help chestundi.
Example:

```
Service: backend-service
    |
    |--- Backend Pod 1
    |--- Backend Pod 2
    └── Backend Pod 3
```

Service selector:

```
selector:
 app: backend
```

Matching label unna Pods ni Service backend endpoints ga use chestundi.
Simple ga:
Service = **Changing Pods ni stable network endpoint venaka expose chese resource.**

---

6. Kubernetes Service types enti?
Ans:
Kubernetes lo commonly four Service types gurinchi matladutham:

- **ClusterIP**

- **NodePort**
- **LoadBalancer**
- **ExternalName**

ClusterIP → Cluster lopala service access kosam.
NodePort → Node IP + port dwara service ni externally access cheyyadaniki.
LoadBalancer → Cloud environment lo external load balancer provision/integrate cheyyadaniki.
ExternalName → Kubernetes Service ni external DNS name ki map cheyyadaniki.
Example:
ClusterIP
Pod → Service → Internal Clients

NodePort
Internet → NodeIP:Port → Service → Pods

LoadBalancer
Internet → Cloud LB → Service → Pods
Simple ga:
Service type = **Service ni ekkadi nunchi and ela access cheyyalo determine cheyyadaniki use chestam.**

---

7. ClusterIP Service ante enti?
Ans:
ClusterIP Kubernetes Service ki **default Service type**. Idi Service ki cluster-internal virtual IP provide chestundi.
Cluster lopala unna applications ee Service ni access cheyyagalavu.
Example:
Backend Pods
|--- Pod 1
|--- Pod 2
|--- Pod 3
↑
|
backend-service
ClusterIP
Frontend Pod:
Frontend Pod
|
↓
backend-service
|
↓
Backend Pods
ClusterIP usually internet nunchi direct ga accessible kaadu.
Simple ga:
ClusterIP = **Cluster lopala applications madhya communication kosam stable Service.**

---

8. NodePort Service ante enti?
Ans:
NodePort Service type use chesthe Kubernetes **prati suitable Node meeda oka port expose** chestundi. External client Node IP + NodePort use chesi Service ni access cheyyagaladu.
Example:
Service Type = NodePort
NodePort = 30080
External request:
Client
|
↓
NodeIP:30080
|
↓
Service
|
↓
Pods
Suppose Node IP:
192.168.1.10
Appudu:
192.168.1.10:30080
through service access possible.
Simple ga:
NodePort = **Node IP + exposed port dwara Service ni access cheyyadam.**

---

9. LoadBalancer Service ante enti?
Ans:
LoadBalancer Service type mainly cloud environments lo application ni **external load balancer dwara expose cheyyadaniki** use chestam.
Example:
Internet
  |
  ↓
Cloud Load Balancer
  |
  ↓
Kubernetes Service
  |
  |--- Pod 1
  |--- Pod 2
  └── Pod 3
Cloud provider integration available unte Kubernetes LoadBalancer Service create chesinappudu external load balancer provision/configure avvachu.
Example:
type: LoadBalancer
Service external IP/address provide cheyyachu.
Simple ga:
LoadBalancer = **External users kosam cloud/external load balancer dwara Service expose cheyyadam.**

---

10. ExternalName Service ante enti?
Ans:
ExternalName Service anedi Kubernetes Service ni actual Kubernetes Pods ki route cheyyadam kanna, **external DNS name ki map cheyyadaniki** use chestam.
Example:
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: db.example.com
Kubernetes application:
Application
  |
  ↓
external-db
  |
  ↓
db.example.com
Ikkada Kubernetes lo backend Pod create cheyyadam ledu. DNS-level mapping provide chestundi.
Simple ga:
ExternalName = **Kubernetes Service name ni external DNS name ki map cheyyadam.**

---

11. Kubernetes Service selector ante enti?
Ans:
Service selector anedi **Service ki ye Pods backend ga undalo identify chese label matching rule**.
Example Service:
selector:
  app: backend
Pods:
Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=frontend
Service selector app=backend kabatti:
Service
  |
  |--- Pod 1 ✅
  |--- Pod 2 ✅
  └── Pod 3 ❌
Pod 3 Service backend lo include avvadu.
Simple ga:
Service selector = **Matching labels unna Pods ni Service backend ga select chese rule.**

---

12. Kubernetes Endpoints / EndpointSlices ante enti?
Ans:
Service ki traffic ekkadiki send cheyyalo Kubernetes ki backend endpoint information kavali.

Service selector matching Pods ni identify chestundi, and Kubernetes aa backend addresses ni **EndpointSlices** lo track chestundi.
Example:

```
Service
  |
  ↓
EndpointSlice
  |
  |--- 10.0.0.5:8080
  |--- 10.0.0.6:8080
  └── 10.0.0.7:8080
```

Service ki matching Pods change ayithe EndpointSlices information kuda update avuthundi.
Modern Kubernetes lo EndpointSlice scalable representation. Older environments/interviews lo Endpoints object gurinchi kuda mention cheyyachu.
Check cheyyadaniki:

```
kubectl get endpoints
kubectl get endpointslices
```

Simple ga:
EndpointSlice = **Service backend ga available unna Pod network endpoints ni track chese resource.**

---

13. Kubernetes Service DNS ante enti?
Ans:
Kubernetes cluster lo Services ki **DNS name** automatically provide cheyyachu. Anduke applications ki Service ClusterIP ni hardcode cheyyalsina avasaram usually undadu.
Example Service:

```
backend-service
```

Same namespace lo application:

```
backend-service
```

ani DNS name use cheyyachu.
Fully qualified form:

```
backend-service.my-namespace.svc.cluster.local
```

Flow:

```
Frontend Pod
  |
  ↓
backend-service
  |
  ↓
Kubernetes DNS
  |
  ↓
Service
  |
  ↓
Backend Pods
```

Simple ga:
Service DNS = **Service ni stable name dwara access cheyyadaniki Kubernetes DNS provide chese mechanism.**

---

14. Kubernetes lo CoreDNS ante enti?
Ans:
CoreDNS anedi Kubernetes cluster lo **DNS service** provide cheyyadaniki commonly used component.
Pods Service names ni resolve cheyyali ante cluster DNS important.
Example:

```
Frontend Pod
  |
  ↓
backend-service
  |
  ↓
CoreDNS
  |
  ↓
Service IP
```

Suppose backend Service:

```
backend-service
```

Frontend Pod:

```
curl http://backend-service:8080
```

ani call chesthe DNS resolution Service ni identify cheyyadaniki help chestundi.
Simple ga:
CoreDNS = **Kubernetes cluster lo Service/Pod DNS name resolution provide chese DNS component.**

---

15. Kubernetes lo Port, TargetPort and NodePort madhya difference enti?

Ans:
Idi Service interviews lo very common question.
port → Service expose chese port.
targetPort → Backend Pod/container application receive chese port.
nodePort → NodePort Service use chesthe Node meeda externally expose ayye port.
Example:

```
ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080
```

Flow:

```
Client
  |
  ↓
NodeIP:30080
  |
  ↓
Service:80
  |
  ↓
Pod:8080
```

Meaning:
nodePort = 30080
service port = 80
targetPort = 8080
Simple ga:
NodePort = **Node level external port**
Port = **Service port**
TargetPort = **Pod/application port**

---

16. Kubernetes lo Pod-to-Pod communication ela jaruguthundi?
Ans:
Kubernetes networking model prakaram Pods cluster network lo communicate cheyyagalali.
Suppose:
Node 1
Pod A → 10.0.0.5

Node 2
Pod B → 10.0.1.5
Pod A:
10.0.1.5
ni reach cheyyagalali, assuming cluster networking correctly configured.
High-level:

```
Pod A
  |
  ↓
CNI / Cluster Network
  |
  ↓
Node Networking
  |
  ↓
Pod B
```

Ee networking implementation ni Kubernetes lo commonly **CNI (Container Network Interface)** based plugins handle chestayi.
Simple ga:
Pod-to-Pod networking = **Cluster network/CNI through Pods different Nodes lo unna kuda communicate cheyyadam.**

---

17. CNI ante enti? Kubernetes lo CNI enduku use chestam?
Ans:
CNI = Container Network Interface.
Kubernetes itself complete Pod networking implementation provide cheyyadu. Container networking setup cheyyadaniki CNI-compatible network plugins use chestam.
CNI plugin responsibilities implementation batti vary avuthayi, but Pod network connectivity and IP allocation lanti functionality provide chestundi.
Examples:
Calico
Cilium
Flannel
High-level:

```
Kubernetes
  |
```

↓
CNI Plugin
|
↓
Pod Network
|
↓
Pod IP
Suppose new Pod create ayindi. CNI networking component Pod ki network connectivity and IP setup cheyyadaniki involved avuthundi.
Simple ga:
CNI = **Kubernetes Pods ki network connectivity provide/configure cheyyadaniki use ayye standard interface/plugin ecosystem.**

---

18. Service traffic ni backend Pods ki ela distribute chestundi?
Ans:
Service matching backend Pods ni identify chesi, available endpoints madhya traffic route cheyyadaniki Kubernetes networking mechanisms use chestundi.
Example:
Service
|
|--- Pod 1
|--- Pod 2
└── Pod 3
Requests multiple backend Pods ki distribute avvachu.
Request 1 → Pod 1
Request 2 → Pod 2
Request 3 → Pod 3
Exact traffic behavior networking implementation and configuration batti vary avvachu. Important point enti ante Service **stable virtual endpoint** provide chestundi and backend endpoints ni track chestundi.
Readiness state kuda important. Pod traffic receive cheyyadaniki ready kaakapothe, Service endpoints lo eligible ga undakapovachu.
Simple ga:
Service = **Stable endpoint + matching backend endpoints ki traffic routing.**

---

19. Kubernetes lo NetworkPolicy ante enti?
Ans:
NetworkPolicy anedi Pods madhya **network traffic ni control cheyyadaniki** use chese Kubernetes resource.
Default ga cluster networking connectivity allow ayina environment lo, specific security requirements kosam NetworkPolicy use cheyyachu.
Example:
Frontend Pod
|
↓
Backend Pod ✅

Frontend Pod
|
↓
Database Pod ❌
Suppose requirement:
Frontend → Backend = Allow
Frontend → Database = Deny
Backend → Database = Allow
NetworkPolicy dwara ee type traffic restrictions define cheyyachu.
Kaani important point: **NetworkPolicy work avvadaniki cluster network plugin/CNI NetworkPolicy support undali.**
Simple ga:
NetworkPolicy = **Pod-to-Pod network traffic ni allow/deny rules dwara control cheyyadaniki use chese security mechanism.**

---

20. Real-time scenario: Service create chesav, kani request Pods ki reach avvatledu. Ela troubleshoot chestav?
Ans:
First Service correct ga create ayinda check chestha:
kubectl get svc
kubectl describe svc <service-name>
Tarvatha Service selector and Pod labels compare chestha.
Service:
selector:
app: backend
Pods:
Pod 1 → app=frontend
Pod 2 → app=frontend

Ila mismatch unte Service ki matching backend Pods dorakavu.
Check:
kubectl get pods --show-labels
Next EndpointSlices check chestha:
kubectl get endpointslices
Service ki endpoints unnaya leda chustha.
Next target port check chestha.
Suppose:
port: 80
targetPort: 8080
Kaani application Pod lo actually 9090 lo listen chestunte connection fail avvachu.
Pod lopala application listening port verify chestha.
Next readiness check chestha:
kubectl get pods
kubectl describe pod <pod-name>
Pod Ready kakapothe Service backend eligibility affect avvachu.
NetworkPolicy unda check chestha:
kubectl get networkpolicy
DNS issue suspect chesthe:
Frontend Pod
  |
  ↓
Service DNS
resolution test chestha.
Overall troubleshooting flow:
Service not working
    ↓
Check Service
    ↓
Check Selector
    ↓
Check Pod Labels
    ↓
Check EndpointSlices
    ↓
Check targetPort
    ↓
Check Pod Ready status
    ↓
Check NetworkPolicy
    ↓
Check DNS
    ↓
Check application listening port
Simple ga:
Service work avvakapothe **Service → Selector → Pod Labels → Endpoints → targetPort → Readiness → NetworkPolicy → DNS → Application port** order lo troubleshoot chestha

---

5. Labels & Selectors — 20 Questions

1. Kubernetes lo Label ante enti?
Ans:
Label anedi Kubernetes resource ki attach chese **key-value pair metadata**. Resources ni identify, organize, group, and select cheyyadaniki Labels use chestam.
Example:
labels:
 app: backend
 environment: production
 tier: api
Ikkada:
app   = backend
environment = production
tier   = api
lanti key-value pairs Labels.
Suppose cluster lo 10 Pods unnayi. Andulo 5 backend Pods, 5 frontend Pods unnayi anuko.
Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=backend

Pod 4 → app=frontend
Pod 5 → app=frontend
Labels valla ee Pods ni easily identify/select cheyyachu.
Simple ga:
Label = **Kubernetes resource ni identify/group cheyyadaniki use chese key-value tag.**

---

2. Kubernetes lo Selector ante enti?
Ans:
Selector anedi **specific labels unna Kubernetes resources ni select cheyyadaniki use chese matching rule**.
Example:
selector:
app: backend
Cluster lo:
Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=frontend
Selector:
app=backend
kabatti Pod 1 and Pod 2 select avuthayi.
Selector
|
|--- Pod 1 ✅
|--- Pod 2 ✅
└── Pod 3 ❌
Simple ga:
Selector = **Matching labels unna resources ni select chese rule.**

---

3. Labels and Selectors madhya relationship enti?
Ans:
Labels resources ki **identity information** istayi. Selectors aa labels ni **match chesi required resources ni select** chestayi.
Example:
Pod:
app=backend
Selector:
app=backend
Match ayindi.
Label → app=backend
↑
|
Selector → app=backend
Kubernetes lo Services, ReplicaSets, Deployments lanti resources Labels and Selectors ni heavily use chestayi.
Simple ga:
Label = **Tag**
Selector = **Aa tag ni search/select chese rule**

---

4. Kubernetes lo Labels enduku use chestam?
Ans:
Labels Kubernetes resources ni **organize, identify, filter, group, and select** cheyyadaniki use chestam.
Example:
Pod 1 → app=frontend
Pod 2 → app=frontend
Pod 3 → app=backend
Pod 4 → app=backend
Ippudu frontend Pods kavali ante:
kubectl get pods -l app=frontend
Backend Pods:
kubectl get pods -l app=backend
Production Pods:
kubectl get pods -l environment=production
Large cluster lo hundreds/thousands resources unna situation lo Labels very useful.
Simple ga:
Labels = **Resources ni easy ga organize and filter cheyyadaniki.**

---

5. Kubernetes lo multiple Labels oka resource ki add cheyyacha?
Ans:
Yes. Oka Kubernetes resource ki **multiple labels** add cheyyachu.
Example:
metadata:
labels:
app: backend
environment: production

team: payments
tier: api
Ikkada same Pod ki four labels unnayi.
app = backend
environment = production
team = payments
tier = api
Different requirements kosam different selectors use cheyyachu.
Example:
kubectl get pods -l app=backend
or:
kubectl get pods -l environment=production
or multiple labels:
kubectl get pods -l app=backend,environment=production
Simple ga:
One resource ki **multiple labels** add cheyyachu.

---

6. Kubernetes lo Labels and Annotations madhya difference enti?
Ans:
Labels and Annotations rendu metadata, kani purpose different.
Labels → Resources ni identify/select/group cheyyadaniki.
Annotations → Resource gurinchi **additional non-identifying information** store cheyyadaniki.
Example Labels:
labels:
app: backend
environment: prod
Selectors ee labels ni use cheyyagalavu.
Annotations:
annotations:
description: "Payment backend application"
owner: "payments-team"
Annotations normally selection kosam use cheyyavu.
Example:
Label:
app=backend
↓
Service selector can use it

Annotation:
owner=payments-team
↓
Additional metadata
Simple ga:
Labels = **Selection/identification**
Annotations = **Additional information**

---

7. Kubernetes Service Labels and Selectors ni ela use chestundi?
Ans:
Service ki selector define chestam. Aa selector matching labels unna Pods ni backend endpoints ga select chestundi.
Example Pods:
Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=frontend
Service:
selector:
app: backend
Appudu:
Backend Service
|
|--- Pod 1 ✅
|--- Pod 2 ✅
└── Pod 3 ❌
Pod 1 or Pod 2 replace ayina matching labels unna new Pods automatically Service selection lo include avvachu.
Simple ga:
Service Selector = **Matching Pod labels ni base chesi backend Pods select cheyyadam.**

---

8. ReplicaSet Labels and Selectors ni ela use chestundi?
Ans:
ReplicaSet ki selector untundi. Aa selector matching labels unna Pods ni identify chesi, required replica count maintain chestundi.
Example:
selector:

matchLabels:
app: backend
Suppose:
Desired replicas = 3

Pod 1 → app=backend
Pod 2 → app=backend
Pod 3 → app=frontend
ReplicaSet ki matching Pods 2 matrame unnayi.
Desired = 3
Current matching = 2
So replacement Pod create cheyyadaniki ReplicaSet work chestundi.
Simple ga:
ReplicaSet = **Selector dwara matching Pods identify chesi replica count maintain chestundi.**

---

9. Deployment lo Selector enduku important?
Ans:
Deployment ki selector **Deployment manage cheyyalsina Pods ni identify cheyyadaniki** use avuthundi.
Example:
selector:
matchLabels:
app: frontend

template:
metadata:
labels:
app: frontend
Ikkada:
Deployment Selector → app=frontend
Pod Template Label → app=frontend
Match avuthunnayi.
Deployment ReplicaSet ni manage chestundi, ReplicaSet aa selector/Pod labels relationship dwara Pods ni manage chestundi.
Simple ga:
Deployment Selector = **Deployment ki related Pods ni identify cheyyadaniki key mechanism.**

---

10. kubectl **tho Labels use chesi Pods ni ela filter chestam?**
Ans:
kubectl lo -l or --selector option use chesi Labels based filtering cheyyachu.
Suppose Pods:
Pod 1 → app=backend
Pod 2 → app=frontend
Pod 3 → app=backend
Backend Pods:
kubectl get pods -l app=backend
Production Pods:
kubectl get pods -l environment=production
Multiple labels:
kubectl get pods -l app=backend,environment=production
Label display:
kubectl get pods --show-labels
Simple ga:
-l = **Label selector use chesi resources filter cheyyadaniki.**

---

11. Equality-based Label Selector ante enti?
Ans:
Equality-based selector lo label value **exact match** avvali.
Common operators:
=
==
!=
Example:
kubectl get pods -l app=backend
Idi:
app=backend
match ayye Pods ni select chestundi.
Not backend:
kubectl get pods -l app!=backend
Example:
Pod 1 → app=backend ❌
Pod 2 → app=frontend ✅

Simple ga:
Equality selector = **Exact label value match chese selector.**

---

12. Set-based Label Selector ante enti?
Ans:
Set-based selector lo label value **multiple possible values** lo unda ani check cheyyachu.
Common operators:
in
notin
exists
!
Example:
kubectl get pods -l 'environment in (production,staging)'
Ante:
environment=production ✅
environment=staging ✅
environment=dev ❌
notin example:
kubectl get pods -l 'environment notin (dev,test)'
Simple ga:
Set-based selector = **Multiple possible label values or label existence based on selection.**

---

13. matchLabels **ante enti?**
Ans:
matchLabels anedi Kubernetes selector lo **exact key-value matching rules** define cheyyadaniki use chestam.
Example:

```
selector:
  matchLabels:
    app: backend
    environment: production
```

Ante Pod ki rendu labels match avvali:
app=backend
environment=production
Example Pods:
Pod 1 → backend + production ✅
Pod 2 → backend + dev ❌
Pod 3 → frontend + production ❌
matchLabels multiple entries unte normally **AND relationship** laga work chestayi.
Simple ga:
matchLabels = **Exact key-value labels match cheyyadaniki selector definition.**

---

14. matchExpressions **ante enti?**
Ans:
matchExpressions anedi more flexible **set-based selector conditions** define cheyyadaniki use chestam.
Example:

```
selector:
  matchExpressions:
    - key: environment
      operator: In
      values:
        - production
        - staging
```

Meaning:
environment = production ✅
environment = staging ✅
environment = dev ❌
Other operators:
In
NotIn
Exists
DoesNotExist
Multiple expressions unte avi generally **AND** conditions laga evaluate avuthayi.
Simple ga:
matchExpressions = **Advanced/set-based conditions tho resources select cheyyadaniki.**

---

15. matchLabels **and** matchExpressions **madhya difference enti?**
Ans:
matchLabels simple exact matching kosam use chestam.
matchExpressions flexible/set-based matching kosam use chestam.
Example matchLabels:

matchLabels:
 app: backend
Meaning:
app must equal backend
matchExpressions:
matchExpressions:
 - key: environment
   operator: In
   values:
    - production
    - staging
Meaning:
environment = production OR staging
So:
matchLabels
  ↓
Simple exact match

matchExpressions
  ↓
Flexible condition
Simple ga:
matchLabels = **simple exact matching**
matchExpressions = **advanced/set-based matching**

---

16. Oka Pod ki Label add/change cheyyacha?
Ans:
Yes. Running resource ki labels add or change cheyyachu.
Example:
kubectl label pod mypod app=backend
Existing label change cheyyali ante --overwrite use cheyyachu:
kubectl label pod mypod app=frontend --overwrite
Appudu Pod label:
Before → app=backend
After → app=frontend
Important point: Service or ReplicaSet selector aa label meeda depend ayithe, label change valla aa resource selection behavior change avvachu.
Example:
Service selector → app=backend

Pod:
app=backend → Service selects it

Label changed:
app=frontend → Service no longer selects it
Simple ga:
Label change cheyyachu, **kani dependent selectors behavior ni affect cheyyachu.**

---

17. Service selector ki matching Labels unna Pods lekapothe em jaruguthundi?
Ans:
Service create avuthundi, kani matching backend Pods lekapothe Service ki **usable endpoints** undavu.
Example:
Service:
selector:
 app: backend
Pods:
Pod 1 → app=frontend
Pod 2 → app=frontend
Matching app=backend Pod ledu.
So:
Service
  |
  └── No matching backend endpoints
Client Service ni access chesthe request ki backend Pod dorakakapovachu.
Troubleshooting:
kubectl get svc
kubectl get endpointslices
kubectl get pods --show-labels
Service selector and Pod labels compare cheyyali.
Simple ga:
No matching labels = **Service ki backend endpoints undakapovachu.**

18. Labels change chesthe Service behavior immediate ga change avuthunda?
Ans:
Usually yes. Kubernetes control plane matching labels and Service endpoints ni observe/update chestundi.
Suppose:
Service selector:
app=backend
Pod:
app=backend
So Pod Service backend lo untundi.
Nuvvu label change chesav:
app=frontend
Appudu Pod Service selector ki match avvadu. Kubernetes EndpointSlice information update chesi aa Pod ni Service backend eligibility nunchi remove cheyyachu.
Flow:
Pod label change
↓
Selector no longer matches
↓
EndpointSlice update
↓
Service backend changes
Simple ga:
Label selector relationship change ayithe **Service backend membership kuda change avvachu.**

---

19. Labels use chesi real-time application environments ni ela manage chestaru?
Ans:
Real-time projects lo Labels ni application, environment, version, team, tier lanti information identify cheyyadaniki use chestaru.
Example:
labels:
app: payment
environment: production
tier: backend
version: v2
Cluster lo:
payment + production + backend + v1
payment + production + backend + v2
payment + staging + backend + v2
Ippudu production backend Pods select cheyyali ante:
app=payment
environment=production
tier=backend
lanti conditions use cheyyachu.
Service version-specific routing or operational filtering requirements lo labels useful.
Monitoring, troubleshooting, deployment organization, and workload management lo kuda Labels very useful.
Simple ga:
Labels = **Real-time production environment lo applications/resources ni logically organize and select cheyyadaniki powerful mechanism.**

---

20. Real-time scenario: Service ki Pods unnayi, kani traffic Pods ki vellatledu. Labels & Selectors angle lo ela troubleshoot chestav?
Ans:
First Service selector check chestha:
kubectl describe svc <service-name>
Suppose selector:
app=backend
Next Pods labels check chestha:
kubectl get pods --show-labels
Suppose output:
Pod 1 → app=frontend
Pod 2 → app=frontend
Pod 3 → app=backend
Service ki Pod 3 matrame matching.
Next EndpointSlices check chestha:
kubectl get endpointslices
Pod 3 endpoint lo unda check chestha.
Suppose Service selector:
app=backend
environment=production
Kaani Pod:
app=backend

environment=staging
Ikkada app match ayina environment match avvatledu. So Service Pod ni select cheyyadu.
Flow:
Service
↓
Check Selector
↓
Check Pod Labels
↓
Labels match avuthunnaya?
|
No
↓
Fix labels/selector
↓
EndpointSlice updates
↓
Service gets backend endpoint
↓
Traffic can reach Pod
Important ga label change chestunnappudu Deployment/ReplicaSet selectors tho conflicts create cheyyakunda careful ga handle cheyyali.
Simple ga:
Service traffic issue lo Labels & Selectors troubleshoot order = **Service Selector → Pod Labels → Match verify → EndpointSlices → Fix mismatch.**

---

6. Namespaces — 20 Questions

1. Kubernetes lo Namespace ante enti?
Ans:
Namespace anedi Kubernetes cluster lo resources ni **logical ga separate/group cheyyadaniki** use chese mechanism.
Oka Kubernetes cluster lo multiple teams, applications, environments unnayi anuko. Anni resources same place lo unte manage cheyyadam difficult avvachu.
Anduke Namespaces use chestam.
Example:
Kubernetes Cluster
|
|--- dev
|--- staging
└── production
Dev Namespace lo:
dev
├── frontend
├── backend
└── database
Production Namespace lo:
production
├── frontend
├── backend
└── database
Same cluster lo unna resources ni logical boundaries lo organize cheyyachu.
Simple ga:
Namespace = **Kubernetes cluster resources ni logical ga separate and organize cheyyadaniki use chese boundary.**

---

2. Kubernetes lo Namespaces enduku use chestam?
Ans:
Namespaces mainly **resource organization, isolation, access control, and resource management** kosam use chestam.
Suppose company lo:
Team A → payments
Team B → orders
Team C → analytics
Same cluster use chestunnaru.
Namespaces:
payments
orders
analytics
laga separate cheyyachu.
Namespaces valla:

- Resources easy ga organize cheyyachu.
- RBAC permissions namespace level lo apply cheyyachu.

- ResourceQuota namespace level lo set cheyyachu.
- LimitRange use cheyyachu.
- Same resource names different namespaces lo use cheyyachu.

Example:
dev/backend
prod/backend
Rendu namespaces lo backend ane resource name undochu.
Simple ga:
Namespaces = **Organization + access control + resource management + logical isolation kosam.**

---

3. Kubernetes cluster lo multiple Namespaces create cheyyacha?
Ans:
Yes. Oka Kubernetes cluster lo multiple Namespaces create cheyyachu.
Example:
Cluster
|
|--- dev
|--- test
|--- staging
|--- production
└── monitoring
Each Namespace lo different resources run avvachu.
Example:
dev
└── backend Deployment

production
└── backend Deployment
Rendu same cluster lo unna, Namespace different kabatti resources separate logical scope lo untayi.
Simple ga:
One cluster → **Many Namespaces** possible.

---

4. Kubernetes lo default ga unde Namespaces enti?
Ans:
Kubernetes cluster create chesinappudu konni standard/system Namespaces usually untayi.
Common Namespaces:
default
kube-system
kube-public
kube-node-lease
default → Namespace specify cheyyakapothe namespaced resources commonly ikkada create avuthayi.
kube-system → Kubernetes system components/resources kosam.
kube-public → Cluster-wide readable information kosam intended public namespace.
kube-node-lease → Nodes heartbeat/lease information kosam use chestaru.
Check cheyyadaniki:
kubectl get namespaces
or:
kubectl get ns
Simple ga:
Default/system namespaces Kubernetes cluster lo **different system and workload purposes** kosam use avuthayi.

---

5. Namespace create ela chestam?
Ans:
Namespace ni command line or YAML file dwara create cheyyachu.
Command:
kubectl create namespace dev
Appudu:
Namespace
↓
dev
YAML method:
apiVersion: v1
kind: Namespace
metadata:
name: dev
Apply:
kubectl apply -f namespace.yaml
Verify:
kubectl get namespaces
Simple ga:
Namespace create = kubectl create namespace <name> or **Namespace YAML apply**.

6. Kubernetes resource ni specific Namespace lo ela create chestam?
Ans:
Resource YAML lo metadata.namespace specify cheyyachu.
Example:
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: dev
Ippudu Deployment dev Namespace lo create avuthundi.
Alternatively command line lo:
kubectl apply -f deployment.yaml -n dev
Check:
kubectl get deployment -n dev
Simple ga:
Specific Namespace = **YAML lo** metadata.namespace **or command lo** -n <namespace> **use cheyyali.**

---

7. Namespace specify cheyyakapothe resource ekkada create avuthundi?
Ans:
kubectl command lo Namespace specify cheyyakapothe, current kubectl context lo configured namespace use avuthundi. Default context usually default Namespace ni use chestundi.
Example:
kubectl get pods
Current context Namespace default ayithe:
default
 ├── Pod 1
 └── Pod 2
Specific Namespace:
kubectl get pods -n dev
Current context Namespace check cheyyadaniki:
kubectl config view --minify --output 'jsonpath={..namespace}'
Simple ga:
Namespace omit chesthe **current kubectl context namespace** apply avuthundi; usually default context lo default.

---

8. Same resource name different Namespaces lo use cheyyacha?
Ans:
Yes. Namespaced resources different Namespaces lo **same name** use chesukovachu.
Example:
dev namespace:
Deployment → backend

production namespace:
Deployment → backend
Idi conflict kaadu because Namespace scope different.
dev/backend
production/backend
Different resources ga consider chestayi.
Same Namespace lo however same resource kind/name combination duplicate ga undadu.
Simple ga:
Different Namespaces → **Same resource name possible.**

---

9. Namespace resource isolation ante enti?
Ans:
Namespace resources ni **logical boundary** lo separate cheyyadaniki help chestundi.
Example:
dev namespace
 └── backend

production namespace
 └── backend
Dev resource and production resource logically separate scopes lo untayi.
Kaani important point: **Namespace complete security boundary kaadu.** Kubernetes networking, RBAC, admission policies, NetworkPolicy etc. separate controls provide chestayi.
Example:
Namespace
  +
RBAC
  +
NetworkPolicy
  +

ResourceQuota
combine chesi stronger isolation design cheyyachu.
Simple ga:
Namespace = **Logical isolation**, not automatically complete security isolation.

---

10. Namespace lo Service and Pod communication ela work chestundi?
Ans:
Same Namespace lo Service and Pods unte Service name ni simple DNS name ga use cheyyachu.
Example:
Namespace: dev

backend-service
|
↓
Backend Pods
Frontend Pod same Namespace lo:
curl http://backend-service:8080
ani access cheyyachu.
Different Namespace lo Service unte Namespace ni DNS name lo include cheyyachu.
Example:
backend-service.production
Full DNS form:
backend-service.production.svc.cluster.local
Simple ga:
Same Namespace → **Service name enough in many cases**
Different Namespace → **Namespace name include cheyyachu**

---

11. Different Namespaces lo unna Pods communicate cheyyagalava?
Ans:
Yes. Namespace different undadam valla Pods automatic ga network communication nunchi block avvavu.
Example:
dev namespace
Frontend Pod
|
↓
production namespace
Backend Service
|
↓
Backend Pod
Networking configuration allow chesthe communication possible.
Kaani NetworkPolicy restrictions unte traffic deny avvachu.
So:
Different Namespace
↓
Not automatically network isolated
↓
NetworkPolicy may restrict traffic
Simple ga:
Different Namespace = **Network communication automatically block avvadu.**

---

12. Namespace delete chesthe danilo unna resources ki em jaruguthundi?
Ans:
Namespace delete chesthe, normally aa Namespace scope lo unna **namespaced resources kuda deletion process lo ki velthayi**.
Example:
dev Namespace
|
|--- Deployment
|--- Service
|--- Pods
|--- ConfigMap
└── Secret
Namespace delete:
kubectl delete namespace dev
Appudu:
dev ❌
├── Deployment ❌
├── Service ❌
├── Pods ❌
├── ConfigMap ❌

└── Secret ❌
Deletion complete avvadaniki konchem time pattachu depending on resources/finalizers.
Simple ga:
Namespace delete = **Dani lopala unna namespaced resources deletion ki guravuthayi.**

---

13. Namespace and Node madhya difference enti?
Ans:
Namespace and Node completely different concepts.
Namespace = Logical grouping/isolation boundary.
Node = Kubernetes workloads run ayye worker machine.
Example:

```
Cluster
 |
 |--- Namespace: dev
 |     └── Pods
 |
 └── Namespace: prod
       └── Pods
```

Nodes:

```
Cluster
 |
 |--- Node 1
 |--- Node 2
 └── Node 3
```

Oka Namespace lo unna Pods multiple Nodes meeda run avvachu.

```
dev Namespace
 |
 |--- Pod → Node 1
 |--- Pod → Node 2
 └── Pod → Node 3
```

Simple ga:
Namespace = **Logical boundary**
Node = **Physical/virtual machine where Pods run**

---

14. Namespace ki ResourceQuota enduku use chestam?
Ans:
ResourceQuota anedi Namespace lo **overall resource usage or object count ki limits** set cheyyadaniki use chestam.
Suppose development team ki limited resources ivvali.
Example:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 8Gi
```

Meaning approximately:

```
dev Namespace
 |
  ├── Maximum Pods = 10
  ├── CPU requests = 4
  └── Memory requests = 8Gi
```

Team accidentally unlimited workloads deploy cheyyakunda control cheyyachu.
Simple ga:
ResourceQuota = **Namespace resource consumption ki overall limit.**

---

15. Namespace lo LimitRange ante enti?
Ans:
LimitRange anedi Namespace lo individual container/Pod ki **default or allowed resource requests/limits** define cheyyadaniki use chestam.
Example:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: limits
  namespace: dev
spec:
  limits:
```

```
  - type: Container
    default:
      cpu: "500m"
      memory: "512Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
```

Appudu developer resource values specify cheyyakapothe configured defaults apply avvachu.
Difference:

```
ResourceQuota
   ↓
Whole Namespace total limit

LimitRange
   ↓
Individual Container/Pod resource constraints/defaults
```

Simple ga:
LimitRange = **Namespace lo individual workload resource defaults/limits control cheyyadaniki.**

---

16. Namespace and RBAC madhya relationship enti?
Ans:
RBAC permissions ni Namespace scope lo assign cheyyachu.
Suppose developer team ki dev Namespace lo Deployments manage cheyyadaniki permission kavali, kani production lo permission ivvakudadhu.

```
Developer
  |
  ↓
RoleBinding
  |
  ↓
Role
  |
  ↓
dev Namespace
```

Developer:
dev → Access ✅
production → Access ❌
Role Namespace-scoped permissions define cheyyachu. RoleBinding aa permissions ni users/groups/service accounts ki bind chestundi.
Simple ga:
Namespace + RBAC = **Team ki specific Namespace lo required permissions matrame ivvadaniki.**

---

17. Namespace and NetworkPolicy madhya relationship enti?
Ans:
NetworkPolicy ni Namespace lo create chesi aa Namespace lo unna Pods ki **network traffic restrictions** apply cheyyachu.
Example:

```
production Namespace

Frontend Pod
   |
   ↓ allowed
Backend Pod

Frontend Pod
   |
   X denied
Database Pod
```

NetworkPolicy labels, namespace selectors, pod selectors lanti mechanisms use chesi traffic rules define cheyyachu.
Important point: Namespace itself network isolation provide cheyyadu.

```
Namespace
  +
NetworkPolicy
  ↓
Network traffic control
```

Simple ga:
Namespace = **logical grouping**
NetworkPolicy = **network traffic control**

---

18. Namespace lo default Service DNS and cross-Namespace Service DNS ela work chestayi?
Ans:
Same Namespace lo backend Service undi anuko:

Namespace: dev
Service: backend
Dev Namespace lo frontend:
http://backend:8080
ani access cheyyachu.
Different Namespace lo Service:
Service: backend
Namespace: production
Appudu:
http://backend.production:8080
lanti namespace-qualified DNS name use cheyyachu.
Full DNS:
backend.production.svc.cluster.local
Flow:
Pod
↓
Service DNS
↓
Namespace
↓
Service
↓
Backend Pods
Simple ga:
Cross-Namespace Service access lo **Service name + Namespace** use cheyyadam common.

---

19. Real-time scenario: Dev team ki dev **Namespace access kavali, production access ivvakudadhu. Ela design chestav?**
Ans:
First dev Namespace create chestha:
kubectl create namespace dev
Tarvatha developer permissions kosam Role create chestha.
Example:

```
kind: Role
metadata:
  name: developer-role
  namespace: dev
rules:
  - apiGroups: ["", "apps"]
    resources: ["pods", "services", "deployments"]
    verbs: ["get", "list", "watch", "create", "update", "patch"]
```

Tarvatha RoleBinding dwara developer group/user/service account ki bind chestha.
Developer
|
↓
RoleBinding
|
↓
Role
|
↓
dev Namespace
Production Namespace ki corresponding RoleBinding ivvakapothe same permission automatically production ki apply avvadu.
Additional protection kosam:
RBAC
+
ResourceQuota
+
LimitRange
+
NetworkPolicy
use cheyyachu.
Simple ga:
Developer ki **dev Namespace scoped RBAC permission** ivvali; production ki permission ivvakudadhu.

---

20. Real-time scenario: dev **Namespace lo Pod create chesav, kani** kubectl get pods **lo Pod kanipinchadam ledu. Ela troubleshoot chestav?**
Ans:
First Namespace issue suspect chestha.
Nuvvu:
kubectl get pods
run chesav anuko.

Idi current context Namespace lo Pods matrame chupinchachu.
First all Namespaces check chestha:
kubectl get pods -A
or:
kubectl get pods --all-namespaces
Dev Namespace specifically:
kubectl get pods -n dev
Pod actually dev lo create ayinda check chestha.
Current context check:
kubectl config view --minify
Specific Namespace set cheyyali ante:
kubectl config set-context --current --namespace=dev
Tarvatha:
kubectl get pods
If Pod still kanipinchakapothe:
kubectl get events -n dev
kubectl get deployment -n dev
kubectl describe pod <pod-name> -n dev
check chestha.
Overall troubleshooting flow:
Pod not visible
↓
kubectl get pods -A
↓
Correct Namespace?
|
No → kubectl get pods -n dev
|
Yes
↓
Check Deployment / Pod
↓
Check Events
↓
Check RBAC permissions
↓
Check Pod status
Simple ga:
Namespace issue lo first rule: kubectl get pods -A → **correct Namespace identify** → -n dev **tho check → events/RBAC/status investigate.**

---

7. ConfigMap & Secret — 20 Questions

1. Kubernetes lo ConfigMap ante enti?
Ans:
ConfigMap anedi Kubernetes lo application ki required ayye **non-sensitive configuration data** ni store cheyyadaniki use chese resource.
Application code lo configuration hardcode cheyyakunda ConfigMap lo separate ga maintain cheyyachu.
Example:
apiVersion: v1
kind: ConfigMap
metadata:
name: app-config
data:
APP_ENV: production
APP_PORT: "8080"
LOG_LEVEL: info
Application:
Pod
|
↓
ConfigMap
|
|--- APP_ENV
|--- APP_PORT
└── LOG_LEVEL
Password, API key, token lanti sensitive information kosam ConfigMap use cheyyakudadhu. Daaniki Secret use chestam.
Simple ga:
ConfigMap = **Non-sensitive application configuration ni external ga maintain cheyyadaniki use chese resource.**

---

2. Kubernetes lo Secret ante enti?

Ans:
Secret anedi Kubernetes lo **sensitive configuration data** ni store/pass cheyyadaniki use chese resource.
Examples:
Database password
API token
Username
TLS certificate/key
Registry credentials
Example:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  username: admin
  password: mypassword
```

Pod Secret ni environment variables or mounted files ga consume cheyyachu.

```
Pod
|
↓
Secret
|
|--- username
└── password
```

Important: Kubernetes Secret data default ga Base64 encoded representation lo kanipinchachu; **Base64 encryption kaadu**. Strong security kosam encryption at rest, RBAC, least privilege, and external secret-management solutions consider cheyyali.
Simple ga:
Secret = **Sensitive data ni Kubernetes workloads ki provide cheyyadaniki use chese resource.**

---

3. ConfigMap and Secret madhya difference enti?
Ans:
Main difference data sensitivity and intended usage.
ConfigMap:
Non-sensitive configuration
Examples:

```
APP_ENV=production
LOG_LEVEL=info
APP_PORT=8080
```

Secret:
Sensitive information
Examples:

```
DB_PASSWORD
API_TOKEN
TLS_KEY
```

Comparison:
ConfigMap → Normal configuration
Secret → Sensitive configuration
Both ni Pod ki:

- Environment variables ga
- Files ga

provide cheyyachu.
Simple ga:
ConfigMap = **Configuration**
Secret = **Sensitive configuration**

---

4. ConfigMap ni Pod ki environment variable ga ela pass chestam?
Ans:
ConfigMap lo value define chesi, Pod lo envFrom or individual env reference use cheyyachu.
ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
```

Pod:

```
envFrom:
  - configMapRef:
```

```
    name: app-config
```
Appudu container ki:
APP_ENV=production
LOG_LEVEL=info
environment variables available avuthayi.
Individual value:
```
env:
 - name: APP_ENV
   valueFrom:
    configMapKeyRef:
     name: app-config
     key: APP_ENV
```
Simple ga:
ConfigMap → envFrom / configMapKeyRef → **Container environment variables**

---

5. Secret ni Pod ki environment variable ga ela pass chestam?
Ans:
Secret value ni Pod environment variable ga inject cheyyachu.
Secret:
```
apiVersion: v1
kind: Secret
metadata:
 name: db-secret
stringData:
 DB_PASSWORD: mypassword
```
Pod:
```
env:
 - name: DB_PASSWORD
   valueFrom:
    secretKeyRef:
     name: db-secret
     key: DB_PASSWORD
```
Container lo:
DB_PASSWORD=mypassword
available avuthundi.
Multiple Secret values ki:
```
envFrom:
 - secretRef:
    name: db-secret
```
use cheyyachu.
Simple ga:
Secret → secretKeyRef / secretRef → **Container environment variable**

---

6. ConfigMap ni Volume laga mount cheyyacha?
Ans:
Yes. ConfigMap ni Pod volume ga mount chesi configuration ni **files laga** container ki provide cheyyachu.
ConfigMap:
```
data:
 app.properties: |
  environment=production
  log.level=info
```
Pod:
```
volumes:
 - name: config-volume
   configMap:
    name: app-config

containers:
 - name: app
   volumeMounts:
    - name: config-volume
      mountPath: /etc/config
```
Container lo:
/etc/config/app.properties
file available avvachu.
Flow:
ConfigMap
↓
Volume
↓
Container

↓
/etc/config/app.properties
Simple ga:
ConfigMap = **Environment variable ga or configuration file ga mount cheyyachu.**

---

7. Secret ni Volume laga mount cheyyacha?
Ans:
Yes. Secret ni volume ga mount chesi container lo **files laga sensitive data provide** cheyyachu.
Example Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
stringData:
  username: admin
  password: mypassword
```

Pod volume:

```
volumes:
  - name: secret-volume
    secret:
      secretName: db-secret
```

Mount:

```
volumeMounts:
  - name: secret-volume
    mountPath: /etc/secrets
```

Container lo approximately:
/etc/secrets/username
/etc/secrets/password
lanti files available avvachu.
Simple ga:
Secret Volume = **Secret values ni container filesystem lo files laga expose cheyyadam.**

---

8. ConfigMap lo data **and** binaryData **ante enti?**
Ans:
ConfigMap lo configuration values store cheyyadaniki data field commonly use chestam.
Example:

```
data:
  APP_ENV: production
  LOG_LEVEL: info
```

binaryData field binary data kosam use cheyyachu.
Example:

```
binaryData:
  some-file: <base64-encoded-data>
```

Normal text configuration kosam data enough.
data
↓
Text configuration

binaryData
↓
Binary data
Simple ga:
data = **Text configuration**
binaryData = **Binary data**

---

9. Kubernetes Secret lo stringData **and** data **madhya difference enti?**
Ans:
Secret lo stringData and data rendu values specify cheyyadaniki use cheyyachu, kani input format different.
stringData lo plain text value provide cheyyachu:

```
stringData:
  username: admin
  password: mypassword
```

Kubernetes Secret representation ni internally manage chestundi.
data field lo values **Base64 encoded** format lo provide cheyyali.
Example:

```
data:
  username: YWRtaW4=
```

Important:
Base64 ≠ Encryption
Base64 just encoding.

Simple ga:
stringData = **Plain text input convenience**
data = **Base64-encoded Secret data**

---

10. ConfigMap or Secret ni container lo envFrom **use chesi ela load chestam?**
Ans:
envFrom use chesthe ConfigMap/Secret lo unna multiple key-value pairs ni environment variables ga load cheyyachu.
ConfigMap:
data:
APP_ENV: production
LOG_LEVEL: info
APP_PORT: "8080"
Pod:
envFrom:
- configMapRef:
name: app-config
Appudu:
APP_ENV=production
LOG_LEVEL=info
APP_PORT=8080
Secret kuda:
envFrom:
- secretRef:
name: db-secret
use cheyyachu.
Simple ga:
envFrom = **Referenced ConfigMap/Secret lo multiple values ni environment variables ga load cheyyadam.**

---

11. ConfigMap update chesthe Pod ki new value immediate ga vastunda?
Ans:
Depends on **ela consume chestunnavo**.
ConfigMap ni environment variable ga inject chesthe, already running container lo environment variable automatic ga update avvadu. Usually new value use cheyyalante Pod restart/rollout required.
Example:
ConfigMap
APP_ENV=dev
↓
Pod starts
↓
ENV APP_ENV=dev
Later ConfigMap:
APP_ENV=production
chesthe existing container environment lo old value continue avvachu.
Kaani ConfigMap ni volume/file ga mount chesunte Kubernetes mounted content ni eventually update cheyyagaladu, with some timing/caching behavior.
Simple ga:
Environment variable = **Pod restart/rollout usually required**
Mounted ConfigMap file = **Kubernetes eventually updates mounted content**

---

12. Secret update chesthe Pod ki new value immediate ga vastunda?
Ans:
Idi kuda consumption method meeda depend avuthundi.
Secret environment variable ga consume chesthe:
Secret
↓
Pod Environment
Existing running process ki old environment value continue avvachu. New value use cheyyalante usually Pod restart/rollout required.
Secret volume ga mount chesthe Kubernetes mounted Secret content ni eventually update cheyyagaladu, subject to Kubernetes volume update behavior and timing.
Example:
Secret password=v1
↓
Mounted file
↓
Secret updated to v2
↓
Mounted content eventually updated
Kaani application already file content memory lo cache cheskunte application behavior different ga undochu.

Simple ga:
Secret ENV = **Restart/rollout usually needed**
Secret Volume = **Mounted file content can be updated eventually**

---

13. ConfigMap lo sensitive data store cheyyacha?
Ans:
Technically ConfigMap lo arbitrary configuration data store cheyyachu, kani **passwords, tokens, private keys lanti sensitive information kosam ConfigMap use cheyyadam recommended kaadu**.
Example wrong approach:
kind: ConfigMap
data:
DB_PASSWORD: mypassword
Better:
ConfigMap
↓
APP_ENV
LOG_LEVEL
APP_PORT

Secret
↓
DB_PASSWORD
API_TOKEN
TLS_KEY
Reason enti ante ConfigMap non-sensitive configuration kosam designed. Sensitive data ki Secret and proper access controls use cheyyali.
Simple ga:
ConfigMap = **Non-sensitive data only**
Secret = **Sensitive data**

---

14. Kubernetes Secret secure aa? Secret lo data encrypted untunda?
Ans:
Secret name chusi **automatically fully encrypted and secure** ani assume cheyyakudadhu.
Important points:

- Secret data API representation lo Base64 encoded ga undochu.
- Base64 encryption kaadu.
- Kubernetes API access RBAC permissions tho control cheyyali.
- Encryption at rest configure cheyyadam security kosam important.
- Least-privilege access follow cheyyali.
- Production environments lo external secret management systems kuda consider cheyyachu.

Example:
Secret
↓
RBAC
↓
Who can read?
And storage side:
Secret
↓
Encryption at Rest
↓
Protected storage
Simple ga:
Secret = **Sensitive data resource**, kani security kosam **RBAC + encryption at rest + least privilege** important.

---

15. ConfigMap and Secret ni immutable ga cheyyacha?
Ans:
Yes. ConfigMap and Secret ki immutable: true set cheyyachu.
Example ConfigMap:
apiVersion: v1
kind: ConfigMap
metadata:
name: app-config
immutable: true
data:
APP_ENV: production
Once immutable, data ni modify cheyyaleru. Change kavali ante new ConfigMap create cheyyadam common approach.
Why useful?
ConfigMap

↓
Immutable
↓
Accidental changes prevented
Large-scale environments lo unnecessary watches/update processing reduce cheyyadaniki kuda immutable objects useful.
Secret ki kuda:
immutable: true
use cheyyachu.
Simple ga:
Immutable = **Create ayyaka data change cheyyakunda prevent cheyyadam.**

---

16. ConfigMap and Secret lo key/value names application environment variables laga ela work chestayi?
Ans:
Suppose ConfigMap:

```
data:
 APP_NAME: payment
 APP_ENV: production
```

Secret:

```
stringData:
 DB_PASSWORD: secret123
```

Pod:

```
envFrom:
 - configMapRef:
    name: app-config
 - secretRef:
    name: db-secret
```

Container ki:
APP_NAME=payment
APP_ENV=production
DB_PASSWORD=secret123
available avuthayi.
Application code:
APP_NAME
APP_ENV
DB_PASSWORD
environment variables ni read chestundi.
Simple ga:
ConfigMap/Secret → **Environment variables → Application configuration**

---

17. ConfigMap or Secret key missing ayithe Pod ki em jaruguthundi?
Ans:
Idi reference ela define chesavo batti behavior change avvachu.
Suppose:

```
env:
 - name: DB_PASSWORD
   valueFrom:
    secretKeyRef:
     name: db-secret
     key: password
```

Kaani Secret lo password key lekapothe required reference resolve avvakapovachu and container start avvakapovachu.
Optional ga mark chesthe:
optional: true
use cheyyachu.
Example:

```
valueFrom:
 secretKeyRef:
   name: db-secret
   key: password
   optional: true
```

Appudu missing value ni tolerate cheyyagaladu, depending on usage.
Simple ga:
Required key missing = **Pod/container startup issue ravachu**
optional: true = **Missing reference tolerate cheyyachu**

---

18. Real-time application lo ConfigMap and Secret ni ela separate ga design chestav?
Ans:
Suppose production payment application undi.
Normal configuration:
APP_ENV=production
APP_PORT=8080
LOG_LEVEL=info

DB_HOST=mysql
Ivi ConfigMap lo:
ConfigMap
  |
  |--- APP_ENV
  |--- APP_PORT
  |--- LOG_LEVEL
  └── DB_HOST
Sensitive data:
DB_USERNAME
DB_PASSWORD
API_TOKEN
ivi Secret lo:
Secret
  |
  |--- DB_USERNAME
  |--- DB_PASSWORD
  └── API_TOKEN
Application:
         Pod
        / \
       /   \
   ConfigMap   Secret
     | |
  Normal config Sensitive data
Benefits:

- Configuration and secrets separate.
- Access permissions separately control cheyyachu.
- Environment-specific configuration easy ga change cheyyachu.
- Passwords application image lo hardcode cheyyalsina avasaram undadu.

Simple ga:
ConfigMap = **How application should run**
Secret = **Sensitive credentials application needs**

---

19. ConfigMap/Secret values ni Docker image lo hardcode cheyyadam kanna Kubernetes resources enduku better?
Ans:
Suppose Docker image build chestunnappudu:
DB_PASSWORD=mypassword
hardcode chesav.
Problem enti ante same image ni dev, staging, production environments lo use cheyyadam difficult avvachu.
Better approach:
Same Docker Image
    |
    +--- dev ConfigMap/Secret
    |
    +--- staging ConfigMap/Secret
    |
    └── production ConfigMap/Secret
Example:
Image → payment:v1

dev:
DB_HOST=dev-db

prod:
DB_HOST=prod-db
Image change cheyyakunda configuration change cheyyachu.
Sensitive credentials image lo bake cheyyakunda Secret/reference mechanism use cheyyadam safer design.
Simple ga:
Same image + environment-specific ConfigMap/Secret = **Better separation of application and configuration.**

---

20. Real-time scenario: Application Pod start avvatledu after adding Secret. Ela troubleshoot chestav?
Ans:
First Pod status check chestha:
kubectl get pods
If:
CreateContainerConfigError
or similar startup/configuration error unte:
kubectl describe pod <pod-name>
check chestha.

Next referenced Secret exists aa:
kubectl get secret db-secret
Secret keys check cheyyali:
kubectl describe secret db-secret
Sensitive values ni unnecessarily terminal lo expose cheyyakunda metadata/key names verify chestha.
Suppose Pod:
secretKeyRef:
  name: db-secret
  key: password
Secret:
username
db_password
only unnayi.
Ikkada:
Pod expects → password
Secret has → db_password
Key mismatch valla container configuration fail avvachu.
Next Namespace check chestha:
kubectl get secret db-secret -n dev
kubectl get pod <pod-name> -n dev
Important ga Secret **same Namespace** lo undali when referenced by a Pod.
Then YAML/config verify:
kubectl describe pod <pod-name>
kubectl get secret db-secret -o yaml
Finally application side environment/config expectation verify chestha.
Overall flow:
Pod not starting
↓
kubectl get pods
↓
kubectl describe pod
↓
Check Secret exists
↓
Check correct Namespace
↓
Check Secret key name
↓
Check secretKeyRef / secretRef
↓
Check optional setting
↓
Check application configuration
Simple ga:
Secret issue troubleshoot = **Pod status → Describe → Secret exists? → Same Namespace? → Key correct aa? → Reference correct aa? → Application config check.**

---

8. Volumes & Storage — 20 Questions

1. Kubernetes lo Volume ante enti?
Ans:
Kubernetes lo Volume anedi **Pod/container ki storage provide cheyyadaniki** use chese mechanism.
Container filesystem generally temporary nature lo untundi. Container restart/remove ayithe container filesystem lo unna data lose avvachu.
Suppose application container lo:
/app/data/users.db
file create chesindi anuko. Container recreate ayithe aa data disappear avvachu.
Anduke Volume use chestam.
Pod
│
└── Container
    │
    └── Volume
        │
        └── Data
Volume valla data container lifecycle kanna longer ga maintain cheyyagalamu, depending on volume type.
Simple ga:
Volume = **Pod/container ki persistent or shared storage provide cheyyadaniki use chese mechanism.**

---

2. Container filesystem and Kubernetes Volume madhya difference enti?

Ans:
Container filesystem container lifecycle ki closely tied untundi. Container recreate ayithe aa writable layer lo unna data normally preserve avvadu.
Volume separate storage mechanism.
Example:
Without Volume:

Container
↓
/data
↓
Container deleted
↓
Data lost
Volume use chesthe:
Container
↓
Volume
↓
Container deleted
↓
Volume remains*
↓
New Container
↓
Same data
* Volume type and lifecycle rules batti persistence behavior vary avuthundi.
Simple ga:
Container filesystem = **Temporary by nature**
Volume = **Data lifecycle ni container lifecycle nunchi separate cheyyadaniki**

---

3. emptyDir **Volume ante enti?**
Ans:
emptyDir anedi Pod start ayinappudu **empty directory create chese temporary Volume**.
Example:

```
volumes:
  - name: shared-data
    emptyDir: {}
```

Pod lo two containers unnayi anuko:

```
Pod
|
|--- Container 1
|      |
|      ↓
|    emptyDir
|
└--- Container 2
       |
       ↓
     emptyDir
```

Rendu containers same Volume mount chesthe files share chesukovachu.
Pod remove ayithe emptyDir data kuda delete avuthundi.
Simple ga:
emptyDir = **Pod lifetime varaku unde temporary/shared storage.**

---

4. emptyDir **ni real-time lo ekkada use chestam?**
Ans:
emptyDir temporary data kosam useful.
Examples:

- Application temporary files.
- Cache.
- Scratch space.
- Multiple containers madhya shared files.
- Processing intermediate data.

Example:

```
Main Application Container
     |
     ↓
  /shared/data
     ↑
```

|
Sidecar Container
Main container file create chesthe sidecar container same emptyDir mount chesunte aa file read cheyyachu.
Example:
Container 1
↓
writes log
↓
emptyDir
↓
Container 2
↓
processes log
Kaani Pod delete ayithe data preserve avvadu.
Simple ga:
emptyDir = **Temporary processing/cache/shared container data kosam.**

---

5. hostPath **Volume ante enti?**
Ans:
hostPath anedi **Node filesystem lo unna particular path ni Pod ki mount** cheyyadaniki use chese Volume type.
Example:

```
volumes:
 - name: host-data
   hostPath:
    path: /data
    type: DirectoryOrCreate
```

Flow:

```
Node
 |
 └── /data
      ↑
      |
     Pod
      |
  /app/data
```

Pod /app/data lo write chesthe underlying Node /data path lo data store avvachu.
Kaani hostPath portability and security concerns create cheyyachu because workload specific Node filesystem meeda depend avuthundi.
Simple ga:
hostPath = **Node filesystem path ni Pod ki mount cheyyadam.**

---

6. hostPath **and** emptyDir **madhya difference enti?**
Ans:
Main difference storage location and lifecycle.
emptyDir:
Pod starts
↓
emptyDir created
↓
Pod exists
↓
Pod deleted
↓
emptyDir deleted
hostPath:
Node filesystem
↓
/data
↑
|
Pod
hostPath data Node filesystem lo untundi, so Pod recreate ayina same Node meeda path available unte data remain avvachu.
But Pod different Node ki schedule ayithe aa data automatically available ani guarantee ledu.
Simple ga:
emptyDir = **Pod-scoped temporary storage**
hostPath = **Node filesystem storage**

---

7. PersistentVolume (PV) ante enti?
Ans:
PersistentVolume, short ga **PV**, Kubernetes cluster lo persistent storage ni represent chese resource.
PV ni cluster administrator or storage system provision cheyyachu.

Example:
Application Pod
|
↓
PersistentVolumeClaim
|
↓
PersistentVolume
|
↓
Actual Storage
Actual storage backend:
Cloud Disk
NFS
SAN
Local storage
etc.
Pod delete/recreate ayina storage lifecycle separate ga manage cheyyachu, depending on PV/storage configuration.
Simple ga:
PV = **Cluster lo persistent storage resource representation.**

---

8. PersistentVolumeClaim (PVC) ante enti?
Ans:
PVC anedi application/developer side nunchi **storage ni request cheyyadaniki** use chese Kubernetes resource.
Suppose application ki:
10Gi storage
ReadWriteOnce
kavali.
PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Flow:
Application
↓
PVC
↓
PV
↓
Actual Storage
Application direct ga storage backend details manage cheyyalsina avasaram takkuva.
Simple ga:
PVC = **Application storage kosam chese request.**

---

9. PV and PVC madhya difference enti?
Ans:
PV and PVC opposite sides represent chestayi.
PV:
Storage available
PVC:
Storage required
Example:
PV
↓
50Gi available

PVC
↓
10Gi requested
Kubernetes PVC request ki suitable PV ni bind cheyyagaladu.
Flow:
Storage
↓
PV

↑
PVC
↑
Pod
Simple ga:
PV = **Available/provisioned storage**
PVC = **Requested storage**

---

10. Pod lo PVC ni Volume ga ela use chestam?
Ans:
First PVC create chestam. Tarvatha Pod spec lo persistentVolumeClaim reference use chestam.
PVC:

```
kind: PersistentVolumeClaim
metadata:
 name: app-pvc
spec:
 accessModes:
  - ReadWriteOnce
 resources:
  requests:
   storage: 10Gi
```

Pod:

```
volumes:
 - name: app-storage
  persistentVolumeClaim:
   claimName: app-pvc
```

Container:

```
volumeMounts:
 - name: app-storage
  mountPath: /data
```

Flow:
Pod
↓
PVC: app-pvc
↓
PV
↓
Storage
Container /data lo store chesina data persistent storage backend ki vellachu.
Simple ga:
Pod → **PVC reference** → Persistent storage.

---

11. StorageClass ante enti?
Ans:
StorageClass anedi Kubernetes lo **different types of storage ni describe/provision cheyyadaniki** use chese resource.
Example:
fast-ssd
standard
premium
Application ki fast storage kavali ante specific StorageClass request cheyyachu.
PVC:

```
spec:
 storageClassName: fast-ssd
```

Flow:
PVC
↓
StorageClass
↓
Dynamic Provisioning
↓
PV
↓
Actual Storage
Cloud environments lo StorageClass use chesi storage volumes dynamically create cheyyadam common.
Simple ga:
StorageClass = **Storage type/provisioning policy define chese mechanism.**

---

12. Dynamic Provisioning ante enti?
Ans:
Dynamic Provisioning ante developer manually PV create cheyyakunda **PVC create chesinappudu Kubernetes/storage system required storage ni automatically provision cheyyadam**.

Example:
Developer creates PVC
↓
storageClassName: fast-ssd
↓
StorageClass
↓
Storage provisioned automatically
↓
PV created/bound
↓
Pod uses PVC
Without dynamic provisioning:
Admin creates PV manually
↓
Developer creates PVC
With dynamic provisioning:
Developer creates PVC
↓
Storage gets provisioned automatically
Simple ga:
Dynamic Provisioning = **PVC request batti storage automatically create/provision avvadam.**

---

13. Kubernetes lo Access Modes ante enti?
Ans:
Access Mode anedi oka Volume ni **ela access cheyyavachho** indicate chestundi.
Common modes:
ReadWriteOnce (RWO)
ReadOnlyMany (ROX)
ReadWriteMany (RWX)
ReadWriteOncePod (RWOP)
RWO → Volume ni read/write mode lo one node ki attach cheyyadaniki intended.
ROX → Multiple nodes read-only ga access cheyyagalige mode.
RWX → Multiple nodes read/write ga access cheyyagalige mode.
RWOP → Read/write access ni single Pod ki restrict cheyyadaniki.
Actual supported modes **storage backend/CSI driver** meeda depend avuthayi.
Simple ga:
Access Mode = **Storage ni one/multiple nodes or Pods ela access cheyyali ane rule.**

---

14. RWO, ROX, RWX madhya difference enti?
Ans:
RWO — ReadWriteOnce
Node 1
|
↓
Volume
↑
Read + Write
Typically one node ki read/write attachment.
ROX — ReadOnlyMany
Node 1 ─┐
Node 2 ─┼──→ Volume
Node 3 ─┘
Read Only
Multiple nodes read-only access cheyyagalavu, if backend supports it.
RWX — ReadWriteMany
Node 1 ─┐
Node 2 ─┼──→ Volume
Node 3 ─┘
Read + Write
Multiple nodes read/write access cheyyagalavu, if backend supports it.
Example use:
RWO → Single-node database workload
RWX → Shared filesystem
ROX → Shared read-only data
Simple ga:
RWO = **one node read/write**
ROX = **many nodes read-only**
RWX = **many nodes read/write**

---

15. Reclaim Policy ante enti?

Ans:
PersistentVolume release ayinappudu underlying storage resource ki **em behavior undali** ani Reclaim Policy define chestundi.
Common policies:
Retain
Delete
Recycle (legacy/deprecated)
Retain:
PVC deleted
↓
PV released
↓
Data preserved
Admin manually cleanup/recovery cheyyachu.
Delete:
PVC deleted
↓
PV released
↓
Provisioned storage deleted*
* behavior storage provisioner/backend meeda depend chestundi.
Production databases kosam accidental data deletion avoid cheyyadaniki reclaim policy carefully choose cheyyali.
Simple ga:
Reclaim Policy = **PVC/PV release ayyaka storage ki em cheyyalo define chese policy.**

---

16. Stateful applications ki persistent storage enduku important?
Ans:
Stateful applications data ni preserve cheyyali.
Examples:
MySQL
PostgreSQL
MongoDB
Kafka
Elasticsearch
Suppose database Pod:
Database Pod
↓
/var/lib/database
↓
PVC
↓
Persistent Storage
Pod crash ayina or recreate ayina, new Pod same PVC ni attach chesukunte stored data continue cheyyachu.
Without persistent storage:
Database Pod
↓
Container filesystem
↓
Pod recreated
↓
Data may be lost
Simple ga:
Stateful application = **Data important**, so persistent storage commonly required.

---

17. PersistentVolume and PersistentVolumeClaim Namespace-scoped aa?
Ans:
Important interview point.
PVC is Namespace-scoped.
Example:
dev Namespace
└── app-pvc
production Namespace lo same PVC name separate ga undochu.
PV is cluster-scoped.
Cluster
└── PV
PV ki Namespace undadu.
Flow:
Namespace
↓
PVC
↓
PV (cluster-scoped)

↓
Storage
Simple ga:
PVC = **Namespace-scoped**
PV = **Cluster-scoped**

---

18. Storage issue vachinappudu PVC Pending **state lo undi. Ela troubleshoot chestav?**
Ans:
First PVC status check chestha:
kubectl get pvc
Suppose:
NAME STATUS
app-pvc Pending
Next:
kubectl describe pvc app-pvc
Events lo reason chustha.
Then StorageClass check:
kubectl get storageclass
PVC lo:
storageClassName
correct ga unda check chestha.
Next:
kubectl get pv
manual PV provisioning use chestunte suitable PV available aa check chestha.
Dynamic provisioning ayithe CSI/storage provisioner healthy aa check cheyyali.
Troubleshooting flow:
PVC Pending
↓
kubectl describe pvc
↓
Check Events
↓
StorageClass check
↓
PV available?
↓
Dynamic provisioning working?
↓
CSI provisioner healthy?
↓
Capacity / AccessMode / topology check
Simple ga:
PVC Pending = **Events → StorageClass → PV → Provisioner/CSI → capacity/access mode check.**

---

19. Real-time scenario: Pod restart ayyaka application data disappear ayyindi. Ela troubleshoot chestav?
Ans:
First application data ekkada store chestundo check chestha.
Pod YAML:
kubectl describe pod <pod-name>
or Deployment:
kubectl get deployment <name> -o yaml
Check:
volumeMounts
volumes
Suppose application /data lo data store chestundi, kani volume mount ledu.
/data
↓
Container filesystem
↓
Container recreated
↓
Data lost
If emptyDir use chestunte:
emptyDir
↓
Pod recreated
↓
Data lost
Persistent data kosam:
Pod
↓

PVC
↓
PV
↓
Persistent Storage
use cheyyali.
Next checks:
kubectl get pvc
kubectl get pv
kubectl describe pvc <pvc-name>
Also application correct mountPath use chestunda check chestha.
Simple ga:
Data disappear issue = **Volume mount unda? emptyDir aa? PVC aa? Correct mountPath aa? PV/PVC healthy aa?**

---

20. Real-time interview scenario: Database Deployment ni 3 replicas ga scale chesav. Same PVC ni use chesthe em problem ravachu?
Ans:
Idi very important interview scenario.
Suppose:
Database Deployment
replicas = 3
All Pods:
Pod 1 ─┐
Pod 2 ─┼──→ Same PVC
Pod 3 ─┘
PVC access mode ReadWriteOnce ayithe, multiple Pods **same node vs different nodes and backend semantics** batti attachment/access issues ravachu. RWO ni "only one Pod" ani simple ga assume cheyyakudadhu; it is primarily a node-level access mode.
Even if storage supports shared access, database Pods same data directory ni simultaneously write cheyyadam application-level data corruption/consistency problems create cheyyachu.
Database workloads kosam usually:
StatefulSet
 |
 |--- Pod 0 → PVC 0
 |--- Pod 1 → PVC 1
 └── Pod 2 → PVC 2
lanti per-Pod persistent storage model use chestaru.
StatefulSet volumeClaimTemplates use chesi each Pod ki separate PVC create cheyyachu.
Flow:
StatefulSet
 ↓
Pod 0 → PVC 0 → Volume 0
Pod 1 → PVC 1 → Volume 1
Pod 2 → PVC 2 → Volume 2
Simple ga:
Database replicas ki **same PVC blindly share cheyyadam correct design kaadu**. Usually StatefulSet + **separate PVC per Pod** use chestam.

---

9. Requests & Limits — 20 Questions

1. Kubernetes lo Resource Requests ante enti?
Ans:
Resource Request anedi container ki **minimum required CPU and memory resources** entha kavalo Kubernetes ki cheppe value.
Example:
resources:
 requests:
  cpu: "500m"
  memory: "512Mi"
Meaning:
CPU → 500m = 0.5 CPU
Memory → 512Mi
Scheduler Pod ni Node ki place chestunnappudu requests ni consider chestundi.
Suppose Node lo:
Available CPU = 2 CPU
Pod request = 0.5 CPU
Kubernetes aa Pod ni schedule cheyyadaniki aa requested capacity available unda ani consider chestundi.
Simple ga:
Request = **Container ki required resources; scheduling decision lo mainly use chestaru.**

---

2. Kubernetes lo Resource Limits ante enti?

Ans:
Resource Limit anedi container **maximum CPU or memory usage entha varaku allow cheyyalo** define chestundi.
Example:
```
resources:
 limits:
   cpu: "1"
   memory: "1Gi"
```
Meaning:
CPU → Maximum 1 CPU
Memory → Maximum 1Gi
Memory limit cross ayithe container **OOMKilled** avvachu.
CPU limit cross ayithe CPU usage generally throttle avvachu.
Simple ga:
Limit = **Container use cheyyagalige maximum resource boundary.**

---

3. Requests and Limits madhya main difference enti?
Ans:
Requests and Limits rendu different purposes kosam.
Request → Scheduling kosam
Limit → Runtime maximum boundary kosam
Example:
```
resources:
 requests:
   cpu: "500m"
   memory: "512Mi"
 limits:
   cpu: "1"
   memory: "1Gi"
```
Meaning:
Request:
CPU = 0.5
Memory = 512Mi

Limit:
CPU = 1
Memory = 1Gi
Scheduler mainly **request** ni consider chesi Node select chestundi.
Container runtime lo configured **limit** resource consumption boundary ni enforce chestundi.
Simple ga:
Request = **Need**
Limit = **Maximum allowed**

---

4. CPU request 500m **ante enti?**
Ans:
Kubernetes lo CPU ni cores or millicores lo represent cheyyachu.
1 CPU = 1000m
So:
500m = 0.5 CPU
1000m = 1 CPU
250m = 0.25 CPU
Example:
```
resources:
 requests:
   cpu: "500m"
```
Meaning container ki scheduling purpose kosam **0.5 CPU worth capacity** request chestundi.
Example Node:
Node CPU = 4 CPU

Pod A request = 1 CPU
Pod B request = 0.5 CPU
Pod C request = 1 CPU
Total requests:
1 + 0.5 + 1 = 2.5 CPU
Remaining allocatable capacity approximately:
4 - 2.5 = 1.5 CPU
Other system reservations/details valla actual allocatable capacity different undochu.
Simple ga:
500m = 0.5 CPU

---

5. Memory request 512Mi **ante enti?**

Ans:
512Mi ante container ki scheduling kosam **512 Mebibytes memory request** chesindi ani meaning.
Example:

```
resources:
  requests:
    memory: "512Mi"
```

Suppose Node allocatable memory:
8Gi
Already Pods requests:
Pod A → 2Gi
Pod B → 1Gi
Pod C → 512Mi
Total:
3.5Gi
New Pod request:
512Mi
Scheduler Node lo requested capacity available unda ani check chestundi.
Important: Kubernetes memory quantities lo Mi, Gi binary units; M, G decimal units.
Simple ga:
512Mi = **512 Mebibytes memory request.**

---

6. Requests scheduler ki enduku important?
Ans:
Kubernetes Scheduler Pod ni Node ki assign chestunnappudu **Pod resource requests** ni consider chestundi.
Example:
Node 1 → 2 CPU available
Node 2 → 500m CPU available
New Pod:
CPU request = 1 CPU
Appudu:
Node 1 → Can fit ✅
Node 2 → Cannot fit ❌
Scheduler request ni base chesi suitable Node select chestundi.
Important point:
Request ≠ Actual usage
Container actual ga 100m CPU use chestunna, request 500m unte scheduling lo 500m consider avuthundi.
Simple ga:
Scheduler question:
"Ee Pod ki requested resources fit ayye Node edi?"

---

7. Limits runtime lo ela work chestayi?
Ans:
Container ki limit set chesthe runtime aa resource usage ni configured boundary lo control chestundi.
Example:

```
resources:
  limits:
    cpu: "1"
    memory: "512Mi"
```

CPU:
Application wants → 1.5 CPU
Limit → 1 CPU
CPU usage throttle avvachu.
Memory:
Application uses → >512Mi
Limit → 512Mi
Memory limit exceed ayithe container OOMKilled avvachu.
CPU limit exceed
↓
Throttling

Memory limit exceed
↓
OOM kill possible
Simple ga:
CPU limit → **throttling**
Memory limit → **OOMKilled possible**

---

8. CPU limit and Memory limit behavior same aa?
Ans:
No. CPU and Memory behavior different.
Suppose:

limits:
 cpu: "1"
 memory: "512Mi"
Application CPU ekkuva use cheyyali anukunte:
CPU > 1
 ↓
CPU throttling
Memory ekkuva allocate cheyyali anukunte:
Memory > 512Mi
 ↓
Memory limit violation
 ↓
Container may be OOMKilled
Memory resource **compressible kaadu**. CPU generally **compressible** resource.
Simple ga:
CPU → Throttle
Memory → OOMKill possible

---

9. Pod ki resources define chestama or Container ki define chestama?
Ans:
resources.requests and resources.limits **container level lo define chestam**.
Example:
containers:
 - name: backend
  resources:
   requests:
    cpu: "500m"
    memory: "512Mi"
   limits:
    cpu: "1"
    memory: "1Gi"
Oka Pod lo multiple containers unte each container ki separate requests/limits untayi.
Example:
Pod
|
|--- Container 1
| CPU request = 500m
|
└--- Container 2
  CPU request = 200m
Pod-level effective request/limit calculations generally container values ni aggregate chesina result meeda depend chestayi, plus special Pod-level resource features where supported.
Simple ga:
Requests/Limits = **Container specification lo define chestam.**

---

10. Oka Pod lo 2 containers unte requests ela calculate chestam?
Ans:
Suppose Pod lo two containers unnayi:
Container 1:
CPU request = 500m
Memory request = 512Mi

Container 2:
CPU request = 200m
Memory request = 256Mi
Pod scheduling kosam total resource requests approximately:
CPU:
500m + 200m = 700m

Memory:
512Mi + 256Mi = 768Mi
So scheduler Pod ni Node ki place chestunnappudu combined resource requirement consider chestundi.
Same principle limits ki kuda container limits aggregate ayye scenarios untayi.
Simple ga:
Pod request = **Containers resource requests total** for normal scheduling calculation.

---

11. Requests set chesi Limits set cheyyakapothe em jaruguthundi?
Ans:
Request matrame set chesi limit set cheyyakapothe container ki explicit maximum CPU/memory limit configured undakapovachu, unless Namespace lo **LimitRange/defaults** apply avuthunnayi.
Example:

resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
Scheduler:
CPU request = 500m
Memory request = 512Mi
consider chestundi.
Kaani:
CPU limit → Not explicitly specified
Memory limit → Not explicitly specified
Namespace LimitRange defaults unte Kubernetes avi apply cheyyachu.
Production lo requests and limits intentional ga define cheyyadam predictable resource behavior kosam useful.
Simple ga:
Request only = **Scheduling requirement undi; maximum boundary explicit ga undakapovachu.**

---

12. Limits set chesi Requests set cheyyakapothe em jaruguthundi?
Ans:
Container ki limit set chesi request explicitly specify cheyyakapothe Kubernetes behavior lo **request defaults/derivation rules** apply avvachu, including LimitRange settings.
Common situation lo:
resources:
  limits:
    cpu: "1"
    memory: "1Gi"
If no separate request and no overriding LimitRange default, Kubernetes can use the limit as the request for that resource.
So:
Limit:
CPU = 1
Memory = 1Gi

Request may become:
CPU = 1
Memory = 1Gi
Exact behavior ni cluster LimitRange/default configuration tho verify cheyyali.
Interview point:
Limit-only configuration often causes **request to be set equal to limit** when no other request default overrides it.
Simple ga:
Limit-only = **Request may default to the limit.**

---

13. CPU request and CPU limit same ga petti advantage enti?
Ans:
Example:
resources:
  requests:
    cpu: "500m"
  limits:
    cpu: "500m"
Meaning:
Guaranteed scheduling request = 500m
Maximum CPU = 500m
Application ki predictable CPU boundary kavali anukunte useful.
Kaani limit same ga unte application burst avvadaniki extra CPU use cheyyaledu.
Compare:
Request = 500m
Limit = 1 CPU
Application available CPU unte 1 CPU varaku burst cheyyagaladu, subject to node/runtime conditions.
So:
Same:
500m → 500m
  ↓
No extra CPU burst beyond limit

Different:
500m → 1 CPU
  ↓
Can use more CPU when available
Simple ga:
Same request/limit = **Predictable CPU allocation, but less bursting flexibility.**

---

14. Memory request and limit same ga petti advantage enti?

Ans:
Example:
resources:
  requests:
    memory: "512Mi"
  limits:
    memory: "512Mi"
Meaning:
Requested memory = 512Mi
Maximum memory = 512Mi
Scheduler 512Mi request consider chestundi.
Container memory usage 512Mi limit ni exceed chesthe OOMKill risk untundi.
Memory cannot be safely "bursted" beyond limit like CPU.
Memory request and limit same ga define cheyyadam predictable behavior kosam useful, but application actual memory requirement accurately estimate cheyyali.
Simple ga:
Memory request = limit → **Predictable memory boundary.**

---

15. Kubernetes lo QoS Classes ante enti?
Ans:
Kubernetes Pods ki resource configuration batti **QoS — Quality of Service** class assign chestundi.
Main classes:
Guaranteed
Burstable
BestEffort
Guaranteed:
All containers ki CPU and memory requests/limits specified and each resource lo request = limit.
Burstable:
At least one container ki requests or limits configured unnayi, but Pod fully Guaranteed criteria meet avvadu.
BestEffort:
Pod containers ki CPU/memory requests and limits none.
Example:
Guaranteed
CPU request = limit
Memory request = limit

Burstable
Request < Limit

BestEffort
No request
No limit
QoS class resource pressure time lo Pod eviction behavior priority ni influence chestundi.
Simple ga:
QoS = **Resource configuration batti Pod ki Kubernetes assign chese service quality class.**

---

16. BestEffort Pod ante enti?
Ans:
Pod lo **CPU and memory requests/limits none** unte aa Pod BestEffort QoS class lo untundi.
Example:
containers:
  - name: app
    image: nginx
Resources section ledu.
So:
CPU request → None
CPU limit → None
Memory request → None
Memory limit → None
QoS:
BestEffort
Resource pressure vachinappudu BestEffort Pods generally higher eviction risk lo untayi compared with Burstable/Guaranteed Pods.
Production critical workloads ki resources intentionally define cheyyadam usually better.
Simple ga:
BestEffort = **No CPU/memory requests and limits.**

---

17. Burstable Pod ante enti?
Ans:
Pod Guaranteed criteria meet avvakapoyina, at least one container ki CPU/memory request or limit configured unte usually Burstable QoS class lo untundi.

Example:
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "1"
    memory: "512Mi"
Here:
Request < Limit
So Pod:
QoS = Burstable
Burstable workload ki baseline resource request untundi, and configured limits varaku burst capacity undochu.
Simple ga:
Burstable = **Some resource requests/limits configured, but not fully Guaranteed.**

---

18. Resource pressure vachinappudu Requests & Limits ela help chestayi?
Ans:
Node lo memory/CPU pressure vachinappudu Kubernetes resource management and eviction decisions lo **requests, QoS, actual usage, and other factors** consider chestundi.
Example:
Node memory pressure
↓
Kubernetes evaluates Pods
↓
QoS + requests + actual usage
↓
Eviction decisions
BestEffort Pods generally higher eviction priority lo untayi.
Guaranteed Pods generally better protection pondutayi, though not an absolute guarantee against every type of eviction/failure.
Requests accurate ga set cheyyadam important because Kubernetes resource planning and eviction behavior ki requests relevant.
Simple ga:
Requests/Limits = **Scheduling + runtime control + resource-pressure behavior ki important inputs.**

---

19. Real-time scenario: Pod Pending **lo undi. Node lo CPU free ga kanipistundi. Requests & Limits angle lo ela troubleshoot chestav?**
Ans:
First:
kubectl get pods
Pod Pending lo unda check chestha.
Next:
kubectl describe pod <pod-name>
Events lo scheduler reason chustha.
Suppose:
0/3 nodes are available:
3 Insufficient cpu
ani vachindi.
Application actual CPU usage low ayina, request high undochu.
Example:
Pod CPU request = 4 CPU
Node allocatable = 4 CPU
Already requested = 3 CPU
New Pod ki:
3 + 4 = 7 CPU
needed, so fit avvadu.
Actual usage:
Existing Pods actual usage = 1 CPU
ayina scheduler primarily **requests** ni consider chestundi.
Check:
kubectl get pod <pod-name> -o yaml
kubectl describe node <node-name>
Then verify:
Pod requests
Node allocatable
Existing Pods requests
Possible fixes:

- Request accurate ga reduce cheyyadam.
- Larger node add cheyyadam.
- More nodes add cheyyadam.
- Workload distribute cheyyadam.

- HPA/VPA or other capacity strategy consider cheyyadam.

Simple ga:
Node lo actual CPU free unna, **requested CPU fit kakapothe Pod Pending lo undochu.**

---

20. Real-time interview scenario: Production application ki OOMKilled **vastundi. Requests & Limits angle lo ela troubleshoot chestav?**

Ans:
First Pod status:
kubectl get pods
Suppose:
backend-xxx
OOMKilled
Next:
kubectl describe pod <pod-name>
Container state/reason check chestha.
Next resource configuration:
kubectl get deployment <deployment-name> -o yaml
Check:
resources:
 requests:
  memory: "512Mi"
 limits:
  memory: "512Mi"
Suppose application actual memory usage:
Normal → 400Mi
Peak → 700Mi
But limit:
512Mi
So:
Application
 ↓
Memory usage = 700Mi
 ↓
Limit = 512Mi
 ↓
OOMKill
Next questions:

1. Application genuinely more memory need chestunda?
2. Memory leak unda?
3. Traffic suddenly increase ayyinda?
4. Limit too low aa?
5. Multiple containers unte correct container edi?
6. Node memory pressure kuda unda?

Commands:
kubectl describe pod <pod-name>
kubectl top pod <pod-name>
kubectl top pod <pod-name> --containers
kubectl get pod <pod-name> -o yaml
Fix blindly limit increase cheyyakudadhu.
If application legitimately needs 1Gi:
resources:
 requests:
  memory: "1Gi"
 limits:
  memory: "2Gi"
lanti values workload behavior batti tune cheyyachu.
But memory leak unte limit increase only temporary workaround avvachu.
Overall flow:
OOMKilled
 ↓
Check Pod status
 ↓
Describe Pod
 ↓
Check memory limit
 ↓
Check actual memory usage
 ↓
Check application behavior/leak
 ↓
Tune request/limit

↓
Monitor again
Simple ga:
OOMKilled = **Container memory limit exceed ayinda, actual usage entha, application leak/traffic issue unda ani first verify chesi appudu request/limit tune cheyyali.**

---

10. Liveness / Readiness / Startup Probes — 20 Questions

1. Kubernetes lo Probe ante enti?
Ans:
Probe anedi Kubernetes ki **container/application current health and state ni check cheyyadaniki** use chese mechanism.
Kubernetes application container running lo undani matrame chusi healthy ani assume cheyyadu.
Suppose application process running undi, kani application requests process cheyyalekapothundi.
Container process → Running
Application → Not responding
Ilaanti situations ni detect cheyyadaniki probes use chestam.
Kubernetes lo main probes:
Liveness Probe
Readiness Probe
Startup Probe
Simple ga:
Probe = **Application healthy ga unda, traffic accept cheyyagalada, startup complete ayyinda ani Kubernetes check cheyyadaniki use chese mechanism.**

---

2. Liveness Probe ante enti?
Ans:
Liveness Probe anedi application **alive/healthy ga unda** ani check chestundi.
Application process running lo unna, but internally stuck/deadlock state lo undi anuko.
Container → Running
Application → Stuck
Liveness probe fail ayithe Kubernetes container ni **restart** cheyyagaladu.
Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
```

Flow:
Kubernetes
↓
/health check
↓
Fail repeatedly
↓
Container unhealthy
↓
Container restart
Simple ga:
Liveness = **"Application still alive aa?"**

---

3. Readiness Probe ante enti?
Ans:
Readiness Probe anedi container/application **traffic accept cheyyadaniki ready ga unda** ani check chestundi.
Readiness fail ayithe Kubernetes usually Pod ni Service endpoints nunchi temporarily remove chestundi.
Example:
Pod
↓
Readiness check
↓
FAIL
↓
Not Ready
↓
Service traffic stop
Container restart avvalsina avasaram ledu.
Example application:
Application starting
Database connection establishing

Cache loading
App ready ayye varaku user traffic pampakudadhu.
Simple ga:
Readiness = **"Traffic accept cheyyadaniki ready aa?"**

---

4. Liveness and Readiness probes madhya difference enti?
Ans:
Main difference:
Liveness
↓
Application alive/healthy aa?
↓
Fail → Container restart possible

Readiness
↓
Traffic accept cheyyadaniki ready aa?
↓
Fail → Service traffic nunchi remove
Example:
Application database connection temporary ga lose ayindi.
Readiness fail chesthe:
Pod remains running
↓
No new traffic
Liveness fail chesthe:
Container considered unhealthy
↓
Restart possible
Anduke rendu same endpoint ki blindly configure cheyyadam correct kaadu.
Simple ga:
Liveness = **Restart decision**
Readiness = **Traffic decision**

---

5. Startup Probe ante enti?
Ans:
Startup Probe anedi application **startup complete ayyinda leda** ani check cheyyadaniki use chestam.
Some applications start avvadaniki chaala time tiskuntayi.
Example:
Application starts
↓
Loads large configuration
↓
Initializes cache
↓
Connects to database
↓
Startup complete
Startup probe use chesthe startup phase lo liveness/readiness probes ni **startup probe successful ayye varaku delay/suppress** cheyyachu.
Example:

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

30 × 10 = up to roughly 300 seconds of startup checking window, subject to probe timing semantics.
Simple ga:
Startup = **"Application first time startup complete ayyinda?"**

---

6. Liveness, Readiness, Startup probes ni oka simple example tho explain cheyyi.
Ans:
Suppose nee application:
Backend API
Pod start ayindi.
First:
Startup Probe
↓
"Application startup complete aa?"
Startup complete ayyaka:

Readiness Probe
↓
"Requests accept cheyyagalava?"
Application running time lo:
Liveness Probe
↓
"Application healthy/alive ga unda?"
Suppose application stuck ayindi:
Liveness → FAIL
↓
Container restart
Suppose application healthy but DB dependency ready kaadu:
Readiness → FAIL
↓
Pod gets no Service traffic
Simple flow:
Startup → Start avvadaniki ready aa?
Readiness → Traffic ki ready aa?
Liveness → Still healthy/alive aa?

---

7. HTTP Probe ante enti?
Ans:
HTTP Probe anedi Kubernetes container lo HTTP endpoint ni call chesi response successful aa ani check chestundi.
Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
```

Kubernetes:
Pod
↓
http://PodIP:8080/health
↓
HTTP response
↓
Success / Failure
Application:
GET /health
→ 200 OK
ante probe successful.
Suppose:
GET /health
→ timeout / error
repeated failures configured threshold cross chesthe probe fail avvachu.
Simple ga:
HTTP Probe = **HTTP endpoint ni call chesi application health check cheyyadam.**

---

8. TCP Probe ante enti?
Ans:
TCP Probe anedi specified port meeda **TCP connection establish cheyyagalama** ani check chestundi.
Example:

```
readinessProbe:
  tcpSocket:
    port: 8080
```

Flow:
Kubernetes
↓
TCP connection → Pod:8080
↓
Connection successful
↓
Probe success
Port listen cheyyakapothe:
Connection failed
↓
Probe failed
TCP probe application business logic correct ga work chestunda ani prove cheyyadu.
Example:
Port open → TCP success
Database connection broken → Application may still be unhealthy

Simple ga:
TCP Probe = **Port connection establish avuthunda ani check cheyyadam.**

---

9. Exec Probe ante enti?
Ans:
Exec Probe anedi container lopala **command execute chesi result check** chestundi.
Example:

```
livenessProbe:
  exec:
    command:
     - cat
     - /tmp/healthy
```

Command exit code:
0 → Success
Non-zero → Failure
Flow:

```
Kubernetes
   ↓
Execute command inside container
   ↓
Exit code
   ↓
0 → Healthy
non-zero → Failed
```

Useful when application ki HTTP endpoint lekapoyina local command/script tho health check cheyyagaliginappudu.
Kaani complex/heavy commands probes lo run cheyyadam avoid cheyyali.
Simple ga:
Exec Probe = **Container lopala command execute chesi health check cheyyadam.**

---

10. initialDelaySeconds **ante enti?**
Ans:
initialDelaySeconds anedi container start ayyaka **probe first time execute cheyyadaniki mundu wait cheyyalsina seconds**.
Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
```

Meaning:

```
Container starts
     ↓
Wait 30 seconds
     ↓
First liveness probe
```

Application startup slow ayithe immediate liveness check valla unnecessary failures avoid cheyyadaniki historically use chese setting.
Kaani slow-starting applications kosam modern configuration lo **Startup Probe** usually better approach.
Simple ga:
initialDelaySeconds = **Container start ayyaka first probe mundu wait time.**

---

11. periodSeconds **ante enti?**
Ans:
periodSeconds anedi probe **entha interval ki once execute cheyyalo** define chestundi.
Example:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  periodSeconds: 10
```

Meaning approximately:
10 sec → Probe
10 sec → Probe
10 sec → Probe
...
So Kubernetes regular intervals lo application readiness check chestundi.
Very small periodSeconds pettadam valla unnecessary probe traffic and load increase avvachu.
Simple ga:
periodSeconds = **Probe frequency/interval.**

---

12. timeoutSeconds **ante enti?**

Ans:
timeoutSeconds anedi oka probe response kosam **maximum wait time**.
Example:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  timeoutSeconds: 2
```

Meaning:
Probe request
↓
Wait maximum 2 sec
↓
Response vachindi → Success
Response time out → Failure
Suppose endpoint normally 100ms lo response istundi. Appudu 2 seconds reasonable avvachu.
Kaani endpoint sometimes 5 seconds tiskunte timeoutSeconds: 2 valla failures ravachu.
Simple ga:
timeoutSeconds = **Probe response kosam wait chese maximum time.**

---

13. failureThreshold **ante enti?**
Ans:
failureThreshold anedi probe fail ayinappudu **final action consider cheyyadaniki enni consecutive failures required** ani indicate chestundi.
Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 3
```

Suppose:
Probe 1 → FAIL
Probe 2 → FAIL
Probe 3 → FAIL
Configured threshold reach ayithe liveness failure action trigger avvachu.
Single temporary network hiccup valla immediate restart avoid cheyyadaniki threshold useful.
Simple ga:
failureThreshold = **Enni consecutive failures tarvatha probe failed ga consider cheyyali.**

---

14. successThreshold **ante enti?**
Ans:
successThreshold anedi probe successful state ni establish cheyyadaniki **enni consecutive successful probes required** ani define chestundi.
Example:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  successThreshold: 2
```

Suppose Pod currently NotReady:
Probe 1 → Success
Probe 2 → Success
Appudu readiness state successful ga consider cheyyavachu.
Important interview point:

- Readiness probe ki successThreshold > 1 useful.
- Liveness and startup probes ki successThreshold generally **1** ga undali.

Simple ga:
successThreshold = **Ready/healthy state ki enni consecutive successes kavalo.**

---

15. livenessProbe **fail ayithe Pod delete avuthunda?**
Ans:
Direct ga "Pod delete avuthundi" ani cheppadam correct kaadu.
Liveness probe repeated ga fail ayithe Kubernetes **container ni restart** chestundi.
Example:
Pod
|
└── Container
↓
Liveness FAIL
↓

Container restart
Pod itself same object ga continue avvachu.
Suppose Pod Deployment manage chestunte:
Deployment
↓
Pod
↓
Container unhealthy
↓
Container restart
Pod replacement and deletion scenarios separate controller/scheduling events batti jaragachu.
Simple ga:
Liveness fail → **Container restart**, not automatically "Pod delete".

---

16. Readiness Probe fail ayithe container restart avuthunda?
Ans:
Normally **no**.
Readiness failure purpose:
Pod running
↓
Readiness FAIL
↓
Pod NotReady
↓
Service traffic nunchi endpoint remove
Container continue running chestundi.
Suppose application temporary ga database connection establish chestundi.
Readiness FAIL
↓
No new Service traffic
↓
Application recovers
↓
Readiness SUCCESS
↓
Service traffic resumes
Idi zero/minimal disruption rolling deployments and dependency readiness kosam very useful.
Simple ga:
Readiness fail = **Traffic stop/remove**
Container restart = **No**

---

17. Startup Probe use chesthe Liveness and Readiness probes ela behave chestayi?
Ans:
Startup Probe configure chesthe application startup phase lo **startup probe successful ayye varaku** liveness/readiness probes ki startup protection provide cheyyachu.
Example:

```
startupProbe:
 httpGet:
  path: /startup
  port: 8080
 failureThreshold: 30
 periodSeconds: 10

livenessProbe:
 httpGet:
  path: /health
  port: 8080

readinessProbe:
 httpGet:
  path: /ready
  port: 8080
```

Flow:
Container starts
↓
Startup Probe
↓
FAIL → Keep waiting/retry
↓
SUCCESS
↓

Liveness/Readiness take effect
Slow-starting applications ki idi very useful.
Simple ga:
Startup Probe = **Startup phase ni protect chestundi; startup success ayyaka normal health checks take over chestayi.**

---

18. Liveness probe ki /health **and Readiness probe ki** /ready **separate endpoints enduku use chestam?**
Ans:
Because **healthy** and **ready to receive traffic** are not always same condition.
Example:
Application process healthy
↓
Liveness = SUCCESS
Kaani:
Database connection unavailable
↓
Readiness = FAIL
Application completely dead kaadu, so restart cheyyalsina avasaram ledu.
Separate endpoints:
/health
↓
Basic application health

/ready
↓
Can accept production traffic?
This prevents unnecessary restarts.
Bad design:
Same deep dependency check
↓
Liveness
↓
Temporary DB failure
↓
Restart container unnecessarily
Simple ga:
Liveness = **Should I restart?**
Readiness = **Should I send traffic?**

---

19. Real-time scenario: Application startup ki 3 minutes padutundi, but Liveness Probe 10 seconds lo fail ayi container restart loop lo ki velthundi. Problem enti? Ela fix chestav?
Ans:
Problem enti ante application **slow ga start avuthundi**, but liveness probe startup complete avvakundane health check chestundi.
Flow:
Container starts
↓
Application needs 3 minutes
↓
Liveness starts too early
↓
Probe FAIL
↓
Container restart
↓
Application starts again
↓
Liveness FAIL
↓
Restart loop
Fix ki **Startup Probe** use cheyyadam best approach.
Example:

```
startupProbe:
 httpGet:
  path: /startup
  port: 8080
 periodSeconds: 10
 failureThreshold: 30

livenessProbe:
 httpGet:
  path: /health
  port: 8080
```

periodSeconds: 10
Here startup window roughly:
30 × 10 sec = 300 sec
which gives approximately 5 minutes of startup checking window.
Once startup probe succeeds:
Startup SUCCESS
↓
Liveness becomes active
Also verify that the startup endpoint accurately represents startup completion.
Simple ga:
Slow startup + early liveness failure = **Startup Probe use cheyyali.**

---

20. Real-time interview scenario: Pod Running **lo undi, but users ki requests vellatledu. Liveness success, Readiness fail. Ela troubleshoot chestav?**
Ans:
First important point:
Pod Running
+
Liveness SUCCESS
+
Readiness FAIL
ante container alive ga undi, but Kubernetes aa Pod ni **traffic receive cheyyadaniki ready ga consider cheyyatledu**.
First:
kubectl get pods
Next:
kubectl describe pod <pod-name>
Readiness probe failure reason check chestha.
Suppose:
Readiness probe failed:
HTTP probe failed with statuscode: 503
App /ready endpoint 503 enduku return chestundo application logs check chestha:
kubectl logs <pod-name>
Then Service endpoints check chestha:
kubectl get endpoints
kubectl get endpointslices
If Pod Ready condition false ayithe Service ki usable endpoint ga appear avvakapovachu.
Next check:
Readiness path correct aa?
Port correct aa?
Named port correct aa?
Application listening aa?
Dependency available aa?
Example:
Pod
↓
Readiness /ready
↓
503
↓
Pod NotReady
↓
Service endpoint unavailable
↓
User traffic doesn't reach Pod
If application intentionally readiness fail chestunte dependency issue fix cheyyali.
If probe configuration wrong ayithe YAML fix cheyyali.
Commands:
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get svc
kubectl get endpointslices
Overall troubleshooting flow:
Users can't reach application
↓
Pod Running?
↓
Liveness OK?
↓
Readiness FAIL?
↓
Describe Pod

↓
Readiness failure reason
↓
Check application logs
↓
Check Service + EndpointSlices
↓
Check probe path/port
↓
Check application dependencies
↓
Fix + verify Ready
Simple ga:
Running ≠ Ready.
Liveness success + Readiness failure ante:
Application process alive undi, kani Service traffic accept cheyyadaniki currently ready kaadu.

---

11. Deployment Strategies — 20 Questions

1. Kubernetes lo Deployment Strategy ante enti?
Ans:
Deployment Strategy anedi application ki **new version release chestunnappudu old Pods ni new Pods tho ela replace cheyyali** ane approach.
Suppose application:
Version 1 → app:v1
New version:
Version 2 → app:v2
Release time lo Kubernetes:
Old Pods → New Pods
ni ela transition cheyyali anedi Deployment Strategy decide chestundi.
Common approaches:
Rolling Update
Recreate
Advanced deployment patterns:
Blue-Green
Canary
Simple ga:
Deployment Strategy = **Old application version nunchi new version ki traffic/workload ni ela move cheyyalo define chese approach.**

---

2. Kubernetes Deployment lo default strategy enti?
Ans:
Kubernetes Deployment ki default strategy **RollingUpdate**.
Example:

```
spec:
  strategy:
    type: RollingUpdate
```

Manam strategy explicitly mention cheyyakapoyina default ga RollingUpdate use avuthundi.
Flow:
Old Pods
↓
New Pod create
↓
New Pod Ready
↓
Old Pod terminate
↓
Next New Pod
↓
...
Advantage:
Application usually continuously available
provided enough replicas and proper readiness configuration exist.
Simple ga:
Deployment default strategy = **RollingUpdate**

---

3. Rolling Update strategy ante enti?
Ans:
Rolling Update anedi old version Pods ni **one-by-one or controlled batches lo new version Pods tho replace cheyyadam**.

Suppose 4 replicas unnayi:
Old:
v1 v1 v1 v1
New deployment start:
v1 v1 v1 v2
Then:
v1 v1 v2 v2
Then:
v1 v2 v2 v2
Finally:
v2 v2 v2 v2
Actual number of old/new Pods maxSurge and maxUnavailable settings batti vary avuthundi.
Simple ga:
Rolling Update = **Old Pods ni gradually new Pods tho replace cheyyadam.**

---

4. Rolling Update valla main advantage enti?
Ans:
Main advantage **application availability maintain chestu new version deploy cheyyadam**.
Suppose:
4 replicas
Deployment time lo:
Old Pods + New Pods
temporarily coexist avvachu.
New Pod Ready ayyaka old Pod remove chestam.
So:
Users
↓
Service
↓
Available Pods
↓
Old + New during rollout
All replicas ni okesari stop cheyyalsina requirement ledu.
Kaani zero downtime guarantee automatic kaadu. Proper:

- Readiness probes
- Replica count
- maxUnavailable
- Application compatibility
- Graceful shutdown

important.
Simple ga:
Rolling Update = **Gradual replacement + usually better availability.**

---

5. Recreate strategy ante enti?
Ans:
Recreate strategy lo Kubernetes **old version Pods ni first remove chesi, tarvatha new version Pods create** chestundi.
Example:
Before:
v1 v1 v1
Deployment:
Delete old Pods
Temporary:
No application Pods
Then:
Create new Pods
v2 v2 v2
Flow:
v1 Pods
↓
Delete
↓
No Pods
↓
Create v2 Pods
During transition application temporarily unavailable avvachu.
Simple ga:
Recreate = **Old all Pods remove → New all Pods create.**

---

6. RollingUpdate and Recreate madhya difference enti?
Ans:
RollingUpdate:

Old Pods
  +
New Pods
  ↓
Gradual replacement
Usually downtime avoid cheyyadaniki use chestam.
Recreate:
Old Pods
  ↓
Delete all
  ↓
New Pods
Temporary downtime possible.
Comparison:
RollingUpdate
→ Gradual
→ Better availability
→ Old + New may coexist

Recreate
→ All old removed first
→ Downtime possible
→ Then new version
If old and new versions simultaneously run avvakudadhu ane strict requirement unte Recreate useful avvachu.
Simple ga:
RollingUpdate = **Gradual**
Recreate = **Stop → Start**

---

7. maxUnavailable **ante enti?**
Ans:
maxUnavailable anedi rolling update time lo **desired replicas count lo maximum enni Pods unavailable ga undavachho** define chestundi.
Example:
```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
```
Suppose:
Desired replicas = 5
maxUnavailable: 1 ante rollout time lo normally at most one desired replica unavailable ga undela controller manage chestundi.
Flow:
5 old Pods
  ↓
1 old Pod replaced
  ↓
4 old + 1 new
Actual rollout behavior readiness and other conditions batti vary avvachu.
Simple ga:
maxUnavailable = **Rolling update lo simultaneously unavailable ga undagalige Pod limit.**

---

8. maxSurge **ante enti?**
Ans:
maxSurge anedi rolling update time lo **desired replica count kanna extra ga enni Pods create cheyyavachho** define chestundi.
Example:
```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
```
Suppose:
Desired replicas = 5
Rollout time lo temporary ga:
5 desired
+
1 surge
=
6 Pods
undavachu.
New Pod ready ayyaka old Pod terminate cheyyachu.
Simple ga:
maxSurge = **Rollout time lo desired replicas kanna extra Pods entha create cheyyavachho.**

---

9. maxUnavailable **and** maxSurge **ni example tho explain cheyyi.**
Ans:
Suppose:
replicas = 4
Configuration:
maxUnavailable: 1
maxSurge: 1
Meaning:
Desired = 4
Maximum temporary Pods = 5
Maximum unavailable = 1
Rollout:
Initial:
v1 v1 v1 v1
New Pod create:
v1 v1 v1 v1 v2
New Pod Ready ayyaka old Pod remove:
v1 v1 v1 v2
Next:
v1 v1 v2 v2
Eventually:
v2 v2 v2 v2
Exact ordering can vary, but idea:
maxSurge → Extra capacity
maxUnavailable → Allowed unavailable capacity
Simple ga:
Surge = **Extra Pods**
Unavailable = **Allowed missing Pods**

---

10. Blue-Green Deployment ante enti?
Ans:
Blue-Green deployment lo **two complete application environments** maintain chestam.
Blue → Current version
Green → New version
Suppose:
Blue = v1
Green = v2
Green environment fully test chesaka Service selector/traffic ni Green side ki switch chestam.
Before:
User
↓
Service
↓
Blue v1
After switch:
User
↓
Service
↓
Green v2
Rollback:
Service
↓
Blue v1
ki switch cheyyachu.
Simple ga:
Blue-Green = **Old complete environment + New complete environment, then traffic switch.**

---

11. Canary Deployment ante enti?
Ans:
Canary deployment lo new version ni **small percentage of traffic/users ki first release** chestam.
Suppose:
100% traffic
Initially:
v1 → 100%
v2 → 0%
Canary:
v1 → 90%
v2 → 10%
Metrics good unte:
v1 → 50%

v2 → 50%
Finally:
v2 → 100%
Monitor:
Error rate
Latency
CPU
Memory
Business metrics
issue unte traffic v1 ki reduce/rollback cheyyachu.
Simple ga:
Canary = **Small traffic → Monitor → Gradually increase → Full rollout.**

---

12. Rolling Update lo Readiness Probe enduku important?
Ans:
Rolling update lo new Pod create ayindani matrame Kubernetes traffic ki ready ani assume cheyyakudadhu.
Suppose new Pod:
Container started
but application:
Still initializing
Database connection not ready
Readiness probe:
FAIL
So Service new Pod ki traffic avoid cheyyagaladu.
Once:
Readiness SUCCESS
appudu Pod traffic receive cheyyadaniki eligible avvachu.
Flow:
New Pod
↓
Readiness check
↓
FAIL → No traffic
↓
SUCCESS
↓
Traffic
↓
Old Pod can be replaced
Simple ga:
Readiness Probe = **New Pod truly ready ayyaka traffic/rollout progression ki help chestundi.**

---

13. Rolling update stuck ayithe common reasons enti?
Ans:
Rolling update stuck avvadaniki multiple reasons unnayi.
Common reasons:

- New Pods Pending.
- Image pull failure.
- CrashLoopBackOff.
- Readiness probe failing.
- Insufficient CPU/memory.
- Scheduling constraints.
- PVC/storage issue.
- Application startup problem.
- Deployment progress deadline exceeded.

Troubleshooting:
kubectl rollout status deployment/<name>
Then:
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
Deployment:
kubectl describe deployment <name>
Flow:
Rollout stuck
↓
rollout status
↓
Check Pods
↓

Describe failed Pod
↓
Logs
↓
Fix root cause
Simple ga:
Rollout stuck = **Deployment status → Pod status → Events → Logs → Root cause.**

---

14. kubectl rollout status **command enduku use chestam?**
Ans:
Deployment rollout **complete ayyinda, in-progress lo unda, failure/stuck condition unda** ani check cheyyadaniki use chestam.
Command:
kubectl rollout status deployment/my-app
Successful example:
deployment "my-app" successfully rolled out
Rollout ongoing unte progress information chupinchachu.
Useful during CI/CD:
Deploy
↓
kubectl rollout status
↓
Success → Pipeline continues
Failure/timeout → Investigate/stop
Simple ga:
kubectl rollout status = **Deployment rollout status monitor cheyyadaniki.**

---

15. Kubernetes Deployment rollback ela chestam?
Ans:
Deployment previous revision ki rollback cheyyadaniki:
kubectl rollout undo deployment/my-app
History check:
kubectl rollout history deployment/my-app
Specific revision ki:
kubectl rollout undo deployment/my-app --to-revision=2
Example:
v1 → Good
v2 → Good
v3 → Bug
Rollback:
v3
↓
rollback
↓
v2
Rollback success ayinda check:
kubectl rollout status deployment/my-app
Simple ga:
Rollback = **Problematic new version nunchi previous working Deployment revision ki return avvadam.**

---

16. Real-time scenario: New version deploy chesaka errors increase ayyayi. Immediate ga em chestav?
Ans:
First deployment rollout status and application health check chestha.
kubectl rollout status deployment/my-app
Pods:
kubectl get pods
Logs:
kubectl logs <pod-name>
If clearly new version issue ani confirm ayithe rollback:
kubectl rollout undo deployment/my-app
Then:
kubectl rollout status deployment/my-app
Health verify:
Error rate ↓
Pods Ready
Application healthy
After stabilization, root cause investigate chestha instead of simply assuming rollback is permanent fix.
Flow:
New release
↓
Errors increase
↓

Confirm issue
↓
Rollback
↓
Verify health
↓
Root-cause analysis
↓
Fix
↓
Redeploy
Simple ga:
Production issue = **Stabilize first → Rollback if appropriate → Investigate → Fix → Redeploy.**

---

17. Deployment strategy choose chestappudu em factors consider chestav?
Ans:
Strategy application requirements batti select cheyyali.
Consider:
1. Downtime acceptable aa?
2. Old + New versions coexist avvacha?
3. Rollback speed entha important?
4. Extra infrastructure cost entha?
5. Traffic gradually test cheyyala?
6. Database backward compatibility unda?
7. Application stateful aa?
8. Risk tolerance entha?
Examples:
RollingUpdate
→ Normal stateless applications

Recreate
→ Old/new coexist avvakudadhu

Blue-Green
→ Fast switch + easy rollback

Canary
→ Gradual risk-controlled release
Simple ga:
Strategy = **Availability + Risk + Cost + Rollback + Application compatibility batti choose cheyyali.**

---

18. Real-time scenario: Database schema new version tho incompatible undi. Simple RollingUpdate use chesthe problem enti?
Ans:
Idi important real-time interview question.
Suppose:
Old App v1 → Old DB schema
New App v2 → New DB schema
Rolling update time lo:
v1 Pods + v2 Pods
↓
Same Database
simultaneously run avvachu.
If v1 and v2 same DB schema ni support cheyyakapothe:
v1 → expects old schema
v2 → expects new schema
↓
Conflicts / errors
Possible solution:
Backward-compatible database migration pattern:
1. Add compatible DB changes
2. Deploy application version that supports both
3. Migrate data if needed
4. Switch fully to new behavior
5. Remove old schema later
Alternative Blue-Green kuda use cheyyachu, but DB compatibility problem completely disappear avvadu if both environments share same database.
Simple ga:
RollingUpdate lo **old + new versions coexist** avvachu, so DB changes backward compatible ga design cheyyali.

---

19. Real-time interview scenario: Zero downtime deployment kavali. Kubernetes lo em configuration important?

Ans:
"Zero downtime" kosam strategy matrame enough kaadu.
Consider:
Deployment
↓
RollingUpdate
↓
Multiple replicas
↓
Readiness Probe
↓
Proper maxUnavailable
↓
Graceful shutdown
↓
Service
Example:
replicas: 3

strategy:
type: RollingUpdate
rollingUpdate:
maxUnavailable: 0
maxSurge: 1
Readiness:
readinessProbe:
httpGet:
path: /ready
port: 8080
Meaning:
New Pod
↓
Ready?
↓
Yes
↓
Traffic eligible
↓
Old Pod can terminate
Also application graceful shutdown properly handle cheyyali.
Important:
RollingUpdate
alone ≠ Guaranteed zero downtime
Traffic handling, probes, replicas, termination behavior, dependencies—all matter.
Simple ga:
Zero downtime = **RollingUpdate + enough replicas + readiness + proper rollout settings + graceful shutdown + healthy application design.**

---

20. Real-time interview scenario: Production lo risky version deploy cheyyali. RollingUpdate vs Blue-Green vs Canary — edi choose chestav?
Ans:
Requirement batti choose chestha.
RollingUpdate
v1 → gradually → v2
Best when:
Normal stateless application
Controlled gradual replacement
Infrastructure simple ga maintain cheyyali
Blue-Green
Blue v1
Green v2
↓
Traffic switch
Best when:
Fast rollback important
Two environments maintain cheyyagalige capacity undi
Full new environment validate cheyyali
Canary
v1 → 90%
v2 → 10%
↓

Monitor
↓
v2 → 50%
↓
Monitor
↓
v2 → 100%
Best when:
High-risk release
Real user traffic tho gradually validate cheyyali
Metrics/observability strong ga unnayi
Interview lo nenu ila answer chestha:
Normal application
→ RollingUpdate

Fast rollback + parallel environments
→ Blue-Green

High-risk release + gradual traffic testing
→ Canary
But final decision lo **database compatibility, cost, traffic management, observability, rollback requirements** kuda consider chestha.
Simple ga:
RollingUpdate = **Gradual Pod replacement**
Blue-Green = **Two environments + traffic switch**
Canary = **Small traffic + gradual rollout**

---

12. Scaling — HPA — 20 Questions

1. Kubernetes lo Scaling ante enti?
Ans:
Scaling anedi application workload periginaappudu or tagginaappudu **application run chestunna Pods count ni change cheyyadam**.
Suppose:
Normal traffic
↓
3 Pods
Traffic suddenly increase ayindi:
High traffic
↓
3 Pods → 6 Pods
Traffic taggindi:
Low traffic
↓
6 Pods → 3 Pods
Kubernetes lo scaling manually kuda cheyyachu, automatic ga kuda cheyyachu.
Manual Scaling
→ kubectl scale

Automatic Scaling
→ HPA
Simple ga:
Scaling = **Workload requirement batti application Pods count ni increase/decrease cheyyadam.**

---

2. Horizontal Scaling ante enti?
Ans:
Horizontal Scaling ante existing Pod ki more CPU/memory ivvadam kaadu. Instead **more Pods create cheyyadam**.
Example:
Before:
Pod 1
Pod 2

After:
Pod 1
Pod 2
Pod 3
Pod 4
Pod 5
Traffic multiple Pods madhya distribute cheyyadaniki Service use avvachu.

Flow:
Users
↓
Service
↓
Pod 1
Pod 2
Pod 3
Pod 4
Idi especially stateless applications ki useful.
Simple ga:
Horizontal Scaling = **More Pods add cheyyadam.**

---

3. Vertical Scaling ante enti?
Ans:
Vertical Scaling ante existing Pod/container ki **more CPU or memory resources provide cheyyadam**.
Example:
Before:
CPU = 500m
Memory = 512Mi

After:
CPU = 1 CPU
Memory = 1Gi
Pod count same ga undochu.
Before:
3 Pods × 500m CPU

After:
3 Pods × 1 CPU
Vertical scaling ki Kubernetes ecosystem lo VPA (Vertical Pod Autoscaler) use cheyyavachu.
Simple ga:
Vertical Scaling = **Same Pods ki more resources ivvadam.**

---

4. HPA ante enti?
Ans:
HPA full form **Horizontal Pod Autoscaler**.
HPA application workload ni monitor chesi configured metrics batti **Pod replicas count ni automatically increase/decrease** chestundi.
Example:
Normal:
3 Pods
CPU = 40%

Traffic increase
CPU = 80%
↓
HPA
↓
5 Pods
Traffic decrease:
CPU = 20%
↓
HPA
↓
3 Pods
HPA usually Deployment, ReplicaSet, StatefulSet lanti scalable workload resources tho work cheyyagaladu.
Simple ga:
HPA = **Metrics batti Pod replicas ni automatic ga scale chese Kubernetes controller.**

---

5. HPA ki Deployment enduku commonly use chestam?
Ans:
HPA ki desired replica count ni manage cheyyadaniki **scalable workload target** kavali.
Common setup:
HPA
↓
Deployment
↓
ReplicaSet
↓
Pods

Suppose Deployment:
replicas: 3
HPA CPU target based on scaling chesthe:
CPU high
↓
HPA changes desired replicas
↓
Deployment
↓
ReplicaSet
↓
More Pods
Deployment actual Pod creation/deletion ni directly handle cheyyadu; ReplicaSet layer through Pods maintain chestundi.
Simple ga:
HPA → **Target workload replicas ni adjust chestundi → Deployment/ReplicaSet Pods ni create/delete chestayi.**

---

6. HPA CPU based scaling ela work chestundi?
Ans:
Suppose HPA configuration:
Min replicas = 2
Max replicas = 10
CPU target = 60%
Current:
2 Pods
Average CPU = 30%
No need to scale up.
Traffic increase:
Average CPU = 90%
HPA target 60% tho compare chesi desired replica count increase cheyyachu.
Flow:
CPU usage
↓
Metrics API
↓
HPA
↓
Calculate desired replicas
↓
Deployment
↓
More Pods
Exact replica count HPA algorithm and available metrics batti calculate avuthundi.
Simple ga:
CPU target cross ayithe **HPA more Pods create cheyyadaniki scale up cheyyachu.**

---

7. HPA lo minReplicas **ante enti?**
Ans:
minReplicas anedi HPA **minimum maintain cheyyalsina Pod replicas count**.
Example:
minReplicas: 2
maxReplicas: 10
Meaning:
Minimum → 2 Pods
Maximum → 10 Pods
Traffic very low unna:
CPU low
↓
HPA scale down
↓
2 Pods
But:
2 → 1
ki HPA scale down cheyyadu because minimum = 2.
Simple ga:
minReplicas = **HPA allow chese lowest replica count.**

---

8. HPA lo maxReplicas **ante enti?**
Ans:
maxReplicas anedi HPA **maximum ga enni Pods varaku scale cheyyagalado** define chestundi.
Example:
minReplicas: 2

maxReplicas: 10
High traffic:
2 → 4 → 7 → 10
HPA 10 reach ayyaka configured metric inka high unna:
10 → 11 ❌
scale cheyyadu because maximum = 10.
Important: maxReplicas infrastructure capacity guarantee cheyyadu. Nodes/resources saripokapothe requested replicas Pending lo undochu.
Simple ga:
maxReplicas = **HPA scale chese upper limit.**

---

9. HPA lo CPU utilization percentage ela calculate chestaru?
Ans:
CPU utilization based HPA commonly **current CPU usage ni CPU request tho compare** chestundi.
Simplified ga:
CPU utilization =
Actual CPU usage / CPU request × 100
Example:
CPU request = 500m
Actual usage = 250m
Then:
250 / 500 × 100
= 50%
If target:
Target = 50%
application around target lo undi.
Important point:
CPU utilization percentage based HPA use cheyyali ante CPU requests accurately defined undali.
Requests wrong ga unte utilization calculation and scaling behavior unexpected ga undochu.
Simple ga:
CPU utilization = **Actual CPU usage compared with CPU request.**

---

10. HPA ki Metrics Server enduku important?
Ans:
CPU/memory utilization based HPA ki current resource usage data kavali.
Kubernetes cluster lo common setup:
Pod
↓
Metrics Server
↓
Metrics API
↓
HPA
↓
Scale decision
Metrics Server current CPU/memory usage ni provide chestundi.
Check:
kubectl top pods
If:
kubectl top pods
data ivvakapothe metrics collection/setup problem undochu.
HPA:
kubectl get hpa
and:
kubectl describe hpa <hpa-name>
tho inspect cheyyachu.
Simple ga:
Metrics Server = **CPU/memory usage metrics provide chesi HPA ki scaling information available cheyyadaniki common component.**

---

11. HPA CPU and Memory rendu based ga scale cheyyagalada?
Ans:
Yes. HPA multiple metrics ni use cheyyagaladu.
Example:
CPU target = 60%
Memory target = 70%
Application:
CPU = 40%
Memory = 80%
Memory target exceed ayindi.
HPA memory metric based desired replicas calculate chestundi.

Multiple metrics unte HPA **each metric ki desired replica count calculate chesi highest desired count ni choose cheyyagaladu**.
Example:
CPU calculation:
3 Pods

Memory calculation:
5 Pods
Then:
Desired replicas = 5
Simple ga:
HPA **multiple resource/custom metrics** use chesi scaling decision teesukovachu.

---

12. HPA custom metrics ante enti?
Ans:
CPU/memory కాకుండా application-specific metrics base chesi scale cheyyadam **Custom Metrics based scaling**.
Example application:
Queue requests = 100
or:
Requests per second = 500
CPU 30% unna kuda queue huge ga unte application ki more Pods kavali.
Custom metric:
Queue length
↓
Metrics system
↓
HPA
↓
More Pods
Typical custom metrics systems Prometheus Adapter lanti components tho integrate cheyyachu.
Simple ga:
Custom Metric = **Application behavior ki meaningful metric base chesi autoscale cheyyadam.**

---

13. HPA scale up and scale down immediate ga jaruguthaya?
Ans:
Usually HPA every metric change ki immediate ga replicas jump/drop cheyyadu.
Kubernetes HPA lo **sync period, tolerance, stabilization/window behavior, scaling policies** lanti mechanisms untayi.
Especially scale down lo fluctuations valla:
5 Pods
↓
4 Pods
↓
5 Pods
↓
4 Pods
ila constant oscillation avvakunda stabilization help chestundi.
Example:
Traffic spike
↓
Scale up
↓
Traffic drops temporarily
↓
Don't immediately scale down
↓
Observe/stabilize
Exact timing cluster/controller configuration batti vary avuthundi.
Simple ga:
HPA = **Metric fluctuations ni consider chesi controlled scaling chestundi; every small change ki immediate scale down avvalsina avasaram ledu.**

---

14. HPA scale avvatledu ante first em check chestav?
Ans:
First HPA status:
kubectl get hpa
Then:
kubectl describe hpa <hpa-name>
Check:
Current metrics?
Desired replicas?
Min/max?
Conditions?

Events?
CPU/memory based HPA ayithe:
kubectl top pods
check chestha.
Next Pod resource requests:
kubectl get deployment <name> -o yaml
Check:
resources:
  requests:
    cpu: "500m"
Metrics unavailable ayithe HPA scaling work avvakapovachu.
Also:
HPA target correct aa?
maxReplicas reached aa?
Metric actually target cross chestunda?
TargetRef correct aa?
Simple ga:
HPA not scaling → **HPA describe + metrics + requests + target + min/max check.**

---

15. HPA current CPU utilization **target kanna high unna kuda replicas increase avvatledu. Why?**
Ans:
Possible reasons several unnayi.
Example:
CPU = 90%
Target = 60%
Still no scale-up.
Check:

1. maxReplicas already reached aa?
2. Metrics stale/unavailable aa?
3. HPA conditions/events em cheptunnayi?
4. TargetRef correct aa?
5. CPU requests properly configured aa?
6. HPA controller metrics calculation lo issue unda?
7. Pods recently started and metrics behavior stabilize avuthunda?
8. Scaling policy/rate limitation apply avuthunda?

Commands:
kubectl describe hpa <hpa-name>
kubectl get hpa
kubectl top pods
kubectl get deployment <name>
Example:
Current replicas = 10
maxReplicas = 10
CPU = 90%
Appudu HPA scale up cheyyaledu because:
maxReplicas = 10
already reached.
Simple ga:
High CPU ≠ always more Pods. **HPA constraints and metric/configuration status check cheyyali.**

---

16. HPA and Cluster Autoscaler madhya difference enti?
Ans:
Rendu different levels lo scaling chestayi.
HPA:
Application workload
  ↓
More Pods
Cluster Autoscaler:
Not enough Node capacity
  ↓
More Nodes
Example:
Traffic increases
  ↓
HPA
  ↓
3 Pods → 10 Pods
Kaani nodes lo capacity ledu:
New Pods → Pending
Cluster Autoscaler available unte:
Insufficient Node capacity

↓
Add Node
↓
Pending Pods schedule
Flow:
Traffic
↓
HPA → More Pods
↓
Insufficient Nodes
↓
Cluster Autoscaler → More Nodes
Simple ga:
HPA = **Pod scaling**
Cluster Autoscaler = **Node scaling**

---

17. Real-time scenario: Traffic suddenly 10x ayyindi. HPA exactly em chestundi?
Ans:
Suppose initial state:
Deployment replicas = 3
HPA:
min = 3
max = 15
CPU target = 60%
Traffic 10x:
CPU increases
↓
Metrics collected
↓
HPA calculates desired replicas
↓
3 → more replicas
Suppose HPA calculates:
Desired = 10
Deployment desired replicas 10 ki update avvachu.
Then:
Deployment
↓
ReplicaSet
↓
Creates additional Pods
But new Pods schedule avvadaniki Node capacity undali.
If capacity insufficient:
Some Pods → Pending
Cluster Autoscaler available unte nodes add cheyyachu.
Important: HPA is **not instant traffic magic**. Metrics collection, controller sync, Pod startup, scheduling, image pulling, readiness, and application warm-up all take time.
Simple ga:
Traffic spike → **Metrics → HPA → More Pods → Scheduling → Ready → Traffic handling.**

---

18. Real-time scenario: HPA 10 Pods create chesindi, but 5 Pods Pending **lo unnayi. Problem enti?**
Ans:
HPA desired replicas increase chesindi, but cluster lo **enough scheduling capacity ledu**.
Flow:
High traffic
↓
HPA
↓
10 desired Pods
↓
Only 5 can schedule
↓
5 Pending
Check:
kubectl get pods
Pending Pod:
kubectl describe pod <pending-pod>
Suppose event:
Insufficient cpu
Then check Nodes:
kubectl describe nodes

Potential solutions:

- Add/scale Nodes.
- Cluster Autoscaler configure/use cheyyadam.
- Resource requests correct ga tune cheyyadam.
- Workload placement constraints review cheyyadam.
- Existing workloads resource usage/capacity review cheyyadam.

Important:
HPA can request more Pods
but
HPA does NOT create Node capacity.
Simple ga:
HPA Pods increase chestundi; **Nodes capacity lekapothe new Pods Pending lo undochu.**

---

19. Real-time scenario: HPA constantly 3 → 10 → 3 → 10 Pods ani oscillate avuthundi. Enduku? Ela troubleshoot chestav?
Ans:
Idi **flapping/oscillation** type problem.
Possible reasons:
Traffic fluctuates heavily
CPU target too aggressive
Requests incorrectly configured
Pods startup slow
Metrics lag/noise
Application warm-up behavior
Scale-down stabilization insufficient
Example:
3 Pods
↓
CPU high
↓
10 Pods
↓
CPU drops sharply
↓
3 Pods
↓
CPU high again
↓
10 Pods
Troubleshoot:
kubectl describe hpa <hpa-name>
kubectl get hpa -w
kubectl top pods
kubectl get pods
Check:
Target utilization
Requests
Scale behavior/policies
Stabilization configuration
Pod startup/readiness time
Traffic pattern
HPA behavior tuning lo scale down stabilization and policies useful avvachu.
Also application ki CPU request too low unte calculated utilization artificially high ga kanipinchachu.
Simple ga:
HPA oscillation = **Metrics + requests + target + stabilization + application behavior investigate cheyyali.**

---

20. Real-time interview scenario: Production API ki autoscaling design cheyyali. HPA ela configure chestav and em things consider chestav?
Ans:
First application **stateless** and horizontally scalable aa ani confirm chestha.
Example:
API Deployment
replicas = 3
HPA:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
  name: api
 minReplicas: 3
 maxReplicas: 20
 metrics:
  - type: Resource
    resource:
     name: cpu
     target:
      type: Utilization
      averageUtilization: 60
```

Flow:

User traffic
↓
Service
↓
API Pods
↓
CPU/metrics
↓
Metrics API
↓
HPA
↓
3 → 5 → 10 → ...

But production lo CPU okkate metric ga use cheyyadam always ideal kaadu.

Consider:

- CPU request accurate ga unda?
- Memory important aa?
- Requests per second better metric aa?
- Queue length important aa?
- minReplicas entha?
- maxReplicas entha?
- Cluster capacity saripothunda?
- Cluster Autoscaler unda?
- Readiness Probe correct aa?
- Startup time entha?
- Graceful shutdown unda?
- Scale-up/scale-down behavior suitable aa?
- Database downstream capacity handle cheyyagalada?

Example:

API scales:
3 → 20 Pods
Kaani database:
DB max connections = 100
20 Pods వల్ల database overload avvachu.
So autoscaling design application **entire architecture** ni consider cheyyali.
Simple ga:
Production HPA = **Correct metric + correct requests + min/max + readiness + cluster capacity + downstream capacity + stabilization.**

---

13. DaemonSet — 20 Questions

1. Kubernetes lo DaemonSet ante enti?
Ans:
DaemonSet anedi Kubernetes lo **prati eligible Node meeda oka Pod copy run cheyyadaniki** use chese workload resource.
Suppose cluster lo:
Node 1
Node 2
Node 3
DaemonSet create chesthe:
Node 1 → Daemon Pod
Node 2 → Daemon Pod
Node 3 → Daemon Pod
New Node cluster lo join ayithe:
New Node
↓
DaemonSet automatically
↓

New Daemon Pod
Node remove ayithe aa Node meeda unna DaemonSet Pod kuda normally aa Node tho disappear avuthundi.
Common use cases:
Logging agent
Monitoring agent
Node-level security agent
Network plugin components
Simple ga:
DaemonSet = **Prati eligible Node meeda Pod run cheyyadaniki use chese controller.**

---

2. DaemonSet and Deployment madhya main difference enti?
Ans:
Deployment generally **application replicas count** ni manage chestundi.
DaemonSet **Node-based Pod placement** ni manage chestundi.
Deployment:
replicas: 3

Node 1 → Pod
Node 2 → Pod
Node 3 → Pod
DaemonSet:
3 Nodes
↓
3 Daemon Pods
New Node add ayithe:
Deployment
→ replicas count based

DaemonSet
→ eligible Node count based
Example:
Cluster = 5 Nodes

DaemonSet → approximately 5 Pods
Deployment replicas = 3 → 3 Pods
DaemonSet ki exact Pod count scheduling constraints, taints, selectors etc. batti eligible Nodes count tho change avvachu.
Simple ga:
Deployment = **How many replicas?**
DaemonSet = **Which eligible Nodes should have one?**

---

3. DaemonSet ki replicas **field enduku undadu?**
Ans:
Deployment lo:
replicas: 3
ani explicit ga cheptham.
DaemonSet lo manually replica count specify cheyyam because **Pod count eligible Nodes count batti determine avuthundi**.
Suppose:
Eligible Nodes = 4
DaemonSet:
4 Nodes
↓
4 Daemon Pods
New Node:
5th eligible Node
↓
5th Daemon Pod
One Node meeda one DaemonSet Pod ane basic model.
Simple ga:
DaemonSet replicas = **Eligible Nodes count batti automatically determine avuthayi.**

---

4. DaemonSet ni real-time lo ekkada use chestaru?
Ans:
DaemonSet mainly **node-level functionality** kosam use chestam.
Common examples:
1. Log collection agent
2. Monitoring agent
3. Node security agent
4. Network agent
5. Storage-related node agent
Example logging:
Node 1 → App Pods → Logs

↓
Log Agent

Node 2 → App Pods → Logs
↓
Log Agent
DaemonSet:
Node 1 → Fluent Bit Pod
Node 2 → Fluent Bit Pod
Node 3 → Fluent Bit Pod
Each node local logs ni collect cheyyadaniki useful.
Simple ga:
DaemonSet = **Node-level agent/workload ki ideal.**

---

5. New Node cluster lo add ayithe DaemonSet em chestundi?
Ans:
New Node DaemonSet rules ki eligible ayithe Kubernetes automatically **DaemonSet Pod ni aa Node meeda schedule cheyyadaniki try chestundi**.
Before:
Node 1 → DS Pod
Node 2 → DS Pod
New Node:
Node 3 joins
DaemonSet:
Node 3 → DS Pod
So manually:
kubectl create pod...
cheyyalsina avasaram ledu.
Simple ga:
New eligible Node → **DaemonSet automatically places its Pod there.**

---

6. Node remove ayithe DaemonSet Pod ki em avuthundi?
Ans:
Node cluster nunchi remove/drain/delete ayithe aa Node meeda unna DaemonSet Pod kuda aa Node tho normally disappear avuthundi.
Suppose:
Node 1 → DS Pod
Node 2 → DS Pod
Node 3 → DS Pod
Node 2 removed:
Node 1 → DS Pod
Node 3 → DS Pod
DaemonSet ki remaining eligible Nodes meeda one Pod maintain cheyyadam goal.
Node replacement vachindi:
New Node 4
DaemonSet:
Node 4 → New DS Pod
Simple ga:
DaemonSet Pod **Node lifecycle tho closely tied** untundi.

---

7. DaemonSet lo nodeSelector **use cheyyacha?**
Ans:
Yes. DaemonSet Pod ni **specific Nodes meeda matrame run cheyyadaniki** nodeSelector use cheyyachu.
Example Nodes:
Node 1 → disk=ssd
Node 2 → disk=hdd
Node 3 → disk=ssd
DaemonSet:

```
spec:
  template:
    spec:
      nodeSelector:
        disk: ssd
```

Result:
Node 1 → DS Pod ✅
Node 2 → No Pod ❌
Node 3 → DS Pod ✅
So all Nodes meeda kaadu, **matching Nodes meeda only** run avuthundi.
Simple ga:
nodeSelector = **DaemonSet Pod ni selected Nodes ki restrict cheyyadam.**

---

8. DaemonSet and Node Affinity ni kalipi use cheyyacha?
Ans:
Yes. DaemonSet Pod template lo **Node Affinity** use cheyyachu.
Example:

```
affinity:
 nodeAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
   nodeSelectorTerms:
    - matchExpressions:
      - key: workload
        operator: In
        values:
         - monitoring
```

Meaning:
workload=monitoring
↓
DaemonSet Pod eligible
Other Nodes:
workload=application
↓
DaemonSet Pod not scheduled
Node affinity complex rules express cheyyadaniki nodeSelector kanna flexible.
Simple ga:
DaemonSet + Node Affinity = **Specific category of Nodes meeda node-level agent run cheyyadam.**

---

9. Taint unna Node meeda DaemonSet Pod ela run avuthundi?
Ans:
Taint unna Node meeda normal Pods schedule avvakunda Kubernetes prevent cheyyachu.
Example:
Node:
dedicated=infra:NoSchedule
Normal Pod:
→ Cannot schedule ❌
DaemonSet Pod ki matching **toleration** unte aa taint ni tolerate chesi schedule avvachu.
Example:

```
tolerations:
 - key: "dedicated"
   operator: "Equal"
   value: "infra"
   effect: "NoSchedule"
```

Flow:
Tainted Node
↓
DaemonSet Pod
↓
Matching Toleration
↓
Can schedule
Important: Toleration alone doesn't force scheduling; other scheduling rules also must permit placement.
Simple ga:
Taint = **Node restriction**
Toleration = **Pod can tolerate that restriction**

---

10. DaemonSet Pod ki toleration enduku important?
Ans:
Some infrastructure Nodes intentionally tainted untayi.
Example:
Node:
node-role=infra:NoSchedule
Logging agent DaemonSet aa Node meeda kuda run avvali anuko.
Without toleration:
DaemonSet Pod
↓
Taint doesn't tolerate
↓
Not scheduled
With toleration:
DaemonSet Pod
↓
Matching toleration
↓

Can schedule
Infrastructure agents ki cluster-wide coverage kavali ante appropriate taints/tolerations configuration important.
Simple ga:
Toleration = **Tainted Nodes meeda DaemonSet Pod run avvadaniki permission/tolerance provide cheyyadam.**

---

11. DaemonSet Pod ni manually delete chesthe em avuthundi?
Ans:
DaemonSet desired state prakaram eligible Node meeda Pod undali.
Suppose:
Node 1 → DS Pod
Manually delete:
kubectl delete pod <daemon-pod>
DaemonSet controller observe chestundi:
Expected DS Pod → 1
Actual DS Pod → 0
So:
DaemonSet
↓
Creates replacement Pod
This is similar to how controllers continuously reconcile desired vs actual state.
Simple ga:
DaemonSet Pod manually delete chesthe **controller usually replacement Pod create chestundi.**

---

12. DaemonSet rolling update ante enti?
Ans:
DaemonSet update chestunnappudu old Pod version ni **gradually new Pod version tho replace cheyyadam** RollingUpdate strategy.
Example:
Node 1 → v1
Node 2 → v1
Node 3 → v1
New image:
Node 1 → v2
Node 2 → v1
Node 3 → v1
Then:
Node 1 → v2
Node 2 → v2
Node 3 → v1
Finally:
Node 1 → v2
Node 2 → v2
Node 3 → v2
DaemonSet strategy:
updateStrategy:
type: RollingUpdate
is commonly used.
Simple ga:
DaemonSet RollingUpdate = **Node-by-node Daemon Pod version replacement.**

---

13. DaemonSet lo OnDelete **strategy ante enti?**
Ans:
DaemonSet update strategy lo OnDelete use chesthe DaemonSet Pod template change ayina **existing Pods automatically replace avvakapovachu**.
New template:
v1 → v2
Existing Pods:
Node 1 → v1
Node 2 → v1
Node 3 → v1
Manually old Pods delete chesthe:
Delete Node 1 DS Pod
↓
New Pod created
↓
Node 1 → v2
So update control manual ga untundi.
Compare:
RollingUpdate
→ Automatic gradual replacement

OnDelete
→ Existing Pods update when deleted
Simple ga:
OnDelete = **Template change ayina existing DaemonSet Pods ni manually delete/replace cheyyalsi ravachu.**

---

14. DaemonSet status lo desiredNumberScheduled **ante enti?**
Ans:
desiredNumberScheduled anedi **DaemonSet Pod run avvalsina eligible Nodes count** ni indicate chestundi.
Suppose:
Total Nodes = 5
Eligible Nodes = 4
Then:
desiredNumberScheduled = 4
Actual Pods:
currentNumberScheduled = 4
If one Pod not scheduled:
desired = 4
current = 3
ante DaemonSet desired coverage complete kaaledu.
Check:
kubectl get daemonset
More details:
kubectl describe daemonset <name>
Simple ga:
desiredNumberScheduled = **Enni eligible Nodes meeda DaemonSet Pod undali ani expected count.**

---

15. DaemonSet and StatefulSet madhya difference enti?
Ans:
Rendu different purposes kosam.
DaemonSet:
Node 1 → Agent
Node 2 → Agent
Node 3 → Agent
Focus:
Node coverage
StatefulSet:
Pod-0
Pod-1
Pod-2
Focus:
Stable identity
Stable storage
Ordered deployment/scaling
Example:
DaemonSet → Log collector on every node
StatefulSet → Database cluster
Simple ga:
DaemonSet = **One Pod per eligible Node**
StatefulSet = **Stable identity/storage for stateful workloads**

---

16. DaemonSet Pod ki resource requests/limits enduku important?
Ans:
DaemonSet Pods prati Node meeda run avvachu kabatti, **each Node capacity meeda resource impact** untundi.
Suppose:
10 Nodes
DaemonSet:
CPU request = 200m
Memory request = 256Mi
Approximate total cluster request:
10 × 200m = 2000m CPU
10 × 256Mi = 2560Mi memory
If resource requests too high:
DaemonSet
↓
Consumes significant node capacity
↓
Application Pods may not fit
So node-level agents lightweight and properly sized ga undali.
Simple ga:
DaemonSet = **Many Nodes meeda run avuthundi, so small per-Pod resource request kuda cluster-wide significant avvachu.**

---

17. Real-time scenario: Cluster lo 5 Nodes unnayi, but DaemonSet only 3 Pods create chesindi. Enduku?
Ans:
DaemonSet always "all Nodes" ani meaning kaadu. **Eligible Nodes** meeda matrame Pods run chestayi.
Possible reasons:
1. nodeSelector mismatch
2. Node affinity mismatch
3. Taint without matching toleration
4. Node unschedulable
5. Resource shortage
6. Other scheduling constraints
Check:
kubectl get nodes --show-labels
DaemonSet:
kubectl describe daemonset <name>
Pending/missing Pod reasons kosam:
kubectl get pods -o wide
Node:
kubectl describe node <node-name>
Example:
5 Nodes

Node 1 → Eligible ✅
Node 2 → Eligible ✅
Node 3 → Eligible ✅
Node 4 → Tainted ❌
Node 5 → nodeSelector mismatch ❌
Result:
3 Daemon Pods
Simple ga:
DaemonSet Pod count = **Eligible + schedulable Nodes count**, not necessarily total Nodes.

---

18. Real-time scenario: DaemonSet update chesav, but old version Pods inka konni Nodes meeda unnayi. Ela troubleshoot chestav?
Ans:
First DaemonSet rollout status check chestha:
kubectl rollout status daemonset/<name>
Then:
kubectl get daemonset <name>
Pods:
kubectl get pods -o wide
Old/new image identify cheyyadaniki:
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{" "}{.spec.containers[*].image}{"\n"}{end}'
Then problematic Node/POD:
kubectl describe pod <pod-name>
kubectl logs <pod-name>
Common reasons:
Pod not Ready
Pod stuck terminating
Image pull issue
Node unavailable
Resource shortage
Taints/tolerations
Update rollout constraints
Also check:
kubectl describe daemonset <name>
Simple ga:
DaemonSet rollout stuck = **Rollout status → DS status → Pod versions → Pod events/logs → Node issue check.**

---

19. Real-time scenario: Logging agent ni DaemonSet ga deploy chesav. New Node add ayindi, but logging agent Pod aa Node meeda create avvaledu. Ela troubleshoot chestav?
Ans:
First new Node details:
kubectl get nodes --show-labels
kubectl describe node <new-node>
DaemonSet:
kubectl get daemonset
kubectl describe daemonset <daemonset-name>
Check whether Node matches:
nodeSelector?
nodeAffinity?

taints?
tolerations?
Example:
New Node:
environment=prod

DaemonSet:
environment=monitoring
Selector mismatch → DaemonSet won't place Pod there.
Or:
Node taint:
dedicated=infra:NoSchedule
and DaemonSet lacks matching toleration.
Then scheduling cannot happen.
Also check resource capacity:
kubectl describe node <new-node>
Potential flow:
New Node
↓
Is Node eligible?
↓
Labels?
Affinity?
Taints?
Resources?
↓
DaemonSet scheduling
↓
Pod created
Simple ga:
New Node meeda DaemonSet Pod missing ante **Node eligibility + taints/tolerations + affinity + resources** first check cheyyali.

---

20. Real-time interview scenario: "Every Node meeda log collector run cheyyali." Deployment use chestava or DaemonSet? Complete design explain cheyyi.
Ans:
I would use **DaemonSet**, because requirement is:
Every eligible Node meeda one log collector instance run avvali.
Architecture:
Node 1
├── Application Pods
└── Log Collector

Node 2
├── Application Pods
└── Log Collector

Node 3
├── Application Pods
└── Log Collector
DaemonSet:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: log-agent
spec:
 selector:
  matchLabels:
   app: log-agent
 template:
  metadata:
   labels:
    app: log-agent
  spec:
   containers:
    - name: log-agent
      image: example/log-agent:1.0
      resources:
       requests:
        cpu: "100m"
        memory: "128Mi"
       limits:
```

```
        cpu: "500m"
        memory: "256Mi"
```
If agent needs host log files, appropriate host-mounted volume configure chestha.
Conceptual flow:
Application containers
↓
Node log files
↓
DaemonSet Log Agent
↓
Central logging system
New Node:
New Node
↓
DaemonSet
↓
Log Agent automatically
Production considerations:

- Resource requests/limits set cheyyali.
- Required host paths/volumes carefully configure cheyyali.
- Taints unna infrastructure Nodes kosam tolerations evaluate cheyyali.
- Node affinity only required Nodes ki use cheyyali.
- RollingUpdate use chesi agent versions safely update cheyyali.
- Logging backend unavailable ayithe agent behavior understand cheyyali.
- Agent itself excessive CPU/memory consume cheyyakunda monitor cheyyali.

Simple ga:
Every Node meeda one log collector → DaemonSet is the natural choice.

---

14. StatefulSet — 20 Questions

1. Kubernetes lo StatefulSet ante enti?
Ans:
StatefulSet anedi Kubernetes lo **stateful applications ni manage cheyyadaniki** use chese workload resource.
Stateful application ante application ki **stable identity, stable network name, or persistent storage** avasaram undadam.
Examples:
Pod 0 → Database
Pod 1 → Database
Pod 2 → Database
StatefulSet lo Pods ki stable identity untundi.
Example:
mysql-0
mysql-1
mysql-2
Pod mysql-1 delete ayina replacement vachinappudu usually same ordinal identity:
mysql-1
ani maintain cheyyadaniki StatefulSet design chestundi.
Common use cases:
Database
Kafka
ZooKeeper
Distributed storage systems
Simple ga:
StatefulSet = **Stable identity + stable storage/network identity kavalsina stateful applications kosam.**

---

2. StatefulSet and Deployment madhya main difference enti?
Ans:
Deployment mostly **stateless applications** kosam use chestam.
StatefulSet **stateful applications** kosam use chestam.
Deployment:
app-7d8f9
app-3k2m1
app-5p6x4
Pods replace ayithe identities change avvachu.
StatefulSet:
db-0
db-1
db-2
Pods ki stable ordinal identity untundi.
Comparison:

Deployment
→ Stateless
→ Replica identity usually interchangeable
→ General web/API applications

StatefulSet
→ Stateful
→ Stable Pod identity
→ Stable storage/network identity
→ Ordered operations available
Example:
Frontend API → Deployment
Database cluster → StatefulSet
Simple ga:
Deployment = **Interchangeable Pods**
StatefulSet = **Identifiable, stable Pods**

---

3. StatefulSet Pod names ela untayi?
Ans:
StatefulSet Pods ki **stable ordinal-based names** untayi.
Suppose StatefulSet name:
redis
3 replicas:
redis-0
redis-1
redis-2
4th replica add chesthe:
redis-3
Suppose redis-1 delete ayindi.
Replacement Pod:
redis-1
same ordinal identity ni reclaim chestundi.
Simple ga:
StatefulSet Pod naming = **StatefulSet name + ordinal number.**

---

4. StatefulSet lo stable identity ante enti?
Ans:
Stable identity ante Pod restart/delete/recreate ayina **same logical identity ni maintain cheyyadam**.
Example:
db-0
db-1
db-2
db-1 delete ayindi.
Replacement:
db-1
Instead of random name.
This is useful because distributed applications lo individual members ni identify cheyyalsi untundi.
Example:
db-0 = database member 0
db-1 = database member 1
db-2 = database member 2
Application cluster topology lo each member ki identity important avvachu.
Simple ga:
Stable identity = **Pod recreate ayina same ordinal-based identity continue avvadam.**

---

5. StatefulSet lo stable network identity ante enti?
Ans:
StatefulSet Pods ki stable network identity provide cheyyadaniki usually **Headless Service** use chestam.
Example:

```
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  clusterIP: None
```

StatefulSet:
db-0
db-1
db-2
DNS identities available avvachu conceptually:
db-0.db

db-1.db
db-2.db
Exact FQDN namespace/domain configuration batti complete hostname vary avvachu.
This is useful for distributed systems where Pods need to discover individual members.
Simple ga:
StatefulSet + Headless Service = **Stable per-Pod DNS/network identity.**

---

6. Headless Service StatefulSet tho enduku use chestam?
Ans:
Normal Service usually **one stable Service endpoint** provide chestundi.
Kaani StatefulSet lo individual Pods ni identify cheyyali anukunte Headless Service useful.
Headless Service:
clusterIP: None
Suppose:
db-0
db-1
db-2
DNS/service discovery through individual Pod identities possible.
Conceptual flow:
Application
↓
Headless Service DNS
↓
db-0
db-1
db-2
Database cluster lanti systems ki individual members discover cheyyadam useful.
Simple ga:
Headless Service = **Individual StatefulSet Pods ni service discovery/DNS through identify cheyyadaniki useful.**

---

7. StatefulSet lo volumeClaimTemplates **ante enti?**
Ans:
volumeClaimTemplates anedi StatefulSet lo **each Pod ki separate PersistentVolumeClaim automatically create cheyyadaniki** use chestam.
Example:

```
volumeClaimTemplates:
 - metadata:
    name: data
   spec:
    accessModes:
     - ReadWriteOnce
    resources:
     requests:
      storage: 10Gi
```

Suppose:
replicas = 3
Then conceptually:
Pod db-0 → PVC data-db-0
Pod db-1 → PVC data-db-1
Pod db-2 → PVC data-db-2
Each Pod ki separate storage identity untundi.
Simple ga:
volumeClaimTemplates = **StatefulSet each Pod kosam PVC template create cheyyadaniki.**

---

8. StatefulSet Pod delete ayithe data em avuthundi?
Ans:
StatefulSet Pod delete ayina **associated PVC usually automatically delete avvadu** just because Pod was deleted.
Example:
db-0
↓
PVC
↓
PV
↓
Data
Pod delete:
db-0 ❌
PVC:
PVC ✅
Replacement:
db-0

↓
Same PVC
↓
Existing data
So StatefulSet persistent storage use case lo Pod replacement taruvatha data preserve avvadaniki PVC important.
However, PVC/PV lifecycle and retention behavior storage configuration and StatefulSet policies batti vary avvachu.
Simple ga:
Pod delete ≠ **PVC/data automatically delete**.

---

9. StatefulSet lo storageClassName **enduku use chestam?**
Ans:
storageClassName anedi PVC ki **which StorageClass use cheyyalo** specify chestundi.
Example:
storageClassName: fast-storage
Flow:
StatefulSet
↓
volumeClaimTemplate
↓
PVC
↓
StorageClass
↓
Dynamic provisioning
↓
PV
Suppose database ki fast SSD storage kavali:
StorageClass = fast-ssd
Then PVC aa StorageClass use cheyyachu.
Simple ga:
storageClassName = **PVC kosam which storage provisioning class use cheyyalo specify cheyyadam.**

---

10. StatefulSet scaling ela work chestundi?
Ans:
StatefulSet replicas increase chesthe new Pods **ordinal order lo** create avuthayi.
Suppose:
replicas = 2

db-0
db-1
Scale to 4:
db-0
db-1
db-2
db-3
New Pods usually ordinal order lo add avuthayi.
Scale down:
db-0
db-1
db-2
db-3
to:
replicas = 2
Higher ordinals remove avvadaniki StatefulSet design chestundi:
db-3 ❌
db-2 ❌

db-0
db-1
Simple ga:
Scale up = **Higher ordinal Pods add**
Scale down = **Higher ordinal Pods remove**

---

11. StatefulSet lo OrderedReady and Parallel pod management policy ante enti?
Ans:
StatefulSet lo Pod management policy behavior ni control cheyyachu.
Default:
podManagementPolicy: OrderedReady
Meaning Pods **ordered manner lo** create/scale cheyyadaniki Kubernetes wait chestundi.
Example:
db-0

↓ Ready
db-1
↓ Ready
db-2
Parallel:
podManagementPolicy: Parallel
Allows Pods to be started/stopped without waiting for strict readiness ordering.
Conceptually:
db-0 ─┐
db-1 ─┼→ Start independently
db-2 ─┘
Parallel useful when application members don't require strict startup order.
Simple ga:
OrderedReady = **Order important**
Parallel = **Pods independent ga manage cheyyachu**

---

12. StatefulSet lo update strategy enti?
Ans:
StatefulSet lo main update strategies:
RollingUpdate
OnDelete
RollingUpdate:
New Pod template apply ayyaka Pods ni controlled order lo update chestundi.
Example:
db-2 → update
db-1 → update
db-0 → update
StatefulSet RollingUpdate usually highest ordinal nunchi lower ordinal direction lo proceed avvachu.
OnDelete:
Template update ayina existing Pods automatically replace avvakapovachu. Pod delete/recreate ayinappudu new template apply avvachu.
Template v1 → v2
Existing Pods remain
↓
Delete Pod
↓
Replacement uses v2
Simple ga:
RollingUpdate = **Automatic controlled update**
OnDelete = **Pod deletion triggers replacement with new template**

---

13. StatefulSet lo podManagementPolicy: OrderedReady **default enduku useful?**
Ans:
Distributed stateful applications lo **startup/shutdown order** important avvachu.
Example database cluster:
db-0 → Primary/bootstrap member
db-1 → Replica
db-2 → Replica
Strict ordering requirement unte:
db-0
↓ Ready
db-1
↓ Ready
db-2
more predictable startup behavior provide cheyyachu.
But every stateful application ki strict ordering required ani kaadu.
If members independent ga start avvagalige application ayithe:
Parallel
better fit avvachu.
Simple ga:
OrderedReady = **Stateful members startup order important ayinappudu useful.**

---

14. StatefulSet lo updateStrategy: OnDelete **eppudu use chestav?**
Ans:
Existing Pods ni **automatic rollout cheyyakunda update timing ni manually control cheyyali** anukunnappudu OnDelete useful.
Example:
StatefulSet template
v1 → v2
Existing:
db-0 → v1
db-1 → v1

db-2 → v1
Manually db-2 delete chesthe:
db-2 replacement → v2
Then test chesi next Pod:
db-1 → v2
This gives operator more control, but manual operations increase operational responsibility.
Simple ga:
OnDelete = **Stateful Pods ni controlled/manual manner lo update cheyyadaniki useful.**

---

15. StatefulSet Pod ki stable storage ela maintain avuthundi?
Ans:
StatefulSet lo volumeClaimTemplates use chesthe each Pod ki **dedicated PVC** create avuthundi.
Example:
db-0 → PVC-0
db-1 → PVC-1
db-2 → PVC-2
db-1 Pod delete ayindi:
db-1 ❌
PVC-1 ✅
Replacement:
db-1
↓
PVC-1
So same logical member ki same persistent storage attach avvadaniki Kubernetes storage workflow help chestundi, subject to volume attachment and storage constraints.
This is one of StatefulSet's major differences from ordinary stateless Deployments.
Simple ga:
StatefulSet = **Pod identity + dedicated PVC identity** maintain cheyyadaniki designed.

---

16. Real-time scenario: StatefulSet Pod db-1 **crash ayindi. New Pod create ayyaka data ela recover avuthundi?**
Ans:
Suppose:
db-0 → PVC-0
db-1 → PVC-1
db-2 → PVC-2
db-1 crash/delete:
db-1 ❌
PVC-1 ✅
StatefulSet controller replacement create chestundi:
db-1
Storage attach/mount successful ayithe:
db-1
↓
PVC-1
↓
PV
↓
Existing data
Application itself database recovery/replay/replication logic use cheyyachu.
Important:
StatefulSet alone database-level data consistency guarantee cheyyadu.
Database replication, backup, WAL/journal, quorum etc. application-specific.
Simple ga:
Pod fail ayina **same PVC attach ayithe persistent data available avvachu; database recovery logic separate concern.**

---

17. Real-time scenario: StatefulSet lo db-0 **delete chesav.** db-0 **badulu** db-3 **create avuthunda?**
Ans:
No.
Suppose:
db-0
db-1
db-2
db-0 delete chesthe StatefulSet desired replicas still 3.
Controller observe:
Expected:
db-0
db-1
db-2

Actual:
db-1

db-2
So missing ordinal 0 ni replace cheyyadaniki:
db-0
create chestundi.
It doesn't simply create db-3, because db-3 would imply a fourth ordinal/member.
Simple ga:
StatefulSet missing Pod identity ni **same ordinal tho recreate chestundi.**

---

18. Real-time scenario: StatefulSet scale down chesthe PVCs em avuthayi?
Ans:
Suppose:
replicas = 3

db-0 → PVC-0
db-1 → PVC-1
db-2 → PVC-2
Scale down:
replicas = 1
StatefulSet higher ordinal Pods remove chestundi:
db-2 ❌
db-1 ❌
db-0 ✅
PVC behavior ni StatefulSet PVC retention policy and storage configuration determine chestayi. Modern Kubernetes lo PVC retention policy explicitly control cheyyachu.
So important interview point:
Pod delete/scale-down
≠ automatically assume PVC delete
Storage retention policy ni verify cheyyali.
Simple ga:
Scale down ayina **PVCs ni automatically delete chestayi ani blindly assume cheyyakudadhu. Retention policy check cheyyali.**

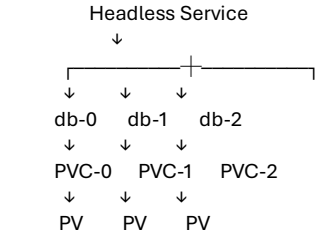
---

19. Real-time scenario: StatefulSet Pod Pending **lo undi. Ela troubleshoot chestav?**
Ans:
First:
kubectl get pods -o wide
Then:
kubectl describe pod <pod-name>
Events lo reason check chestha.
Common reasons:
1. Insufficient CPU/memory
2. PVC Pending
3. StorageClass problem
4. Volume provisioning failure
5. Node affinity issue
6. Taint/toleration issue
7. Node unavailable
PVC check:
kubectl get pvc
kubectl describe pvc <pvc-name>
StorageClass:
kubectl get storageclass
Nodes:
kubectl get nodes
kubectl describe node <node-name>
Flow:
Pod Pending
↓
Describe Pod
↓
Is PVC Pending?
↓
Is scheduling failing?
↓
Check StorageClass / Node / Events
↓
Fix root cause
Simple ga:
StatefulSet Pending Pod = **Pod scheduling + PVC + PV + StorageClass + Node constraints check cheyyali.**

---

20. Real-time interview scenario: Production lo 3-node database cluster deploy cheyyali. StatefulSet ela design chestav?

Ans:
First, database application **StatefulSet requirements ki actually suitable aa**, and database vendor/operator recommended deployment method enti ani verify chestha.
Basic Kubernetes architecture:



StatefulSet:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: db
spec:
 serviceName: db
 replicas: 3
 selector:
  matchLabels:
   app: db
 template:
  metadata:
   labels:
    app: db
  spec:
   containers:
    - name: db
      image: example/db:1.0
      volumeMounts:
       - name: data
         mountPath: /data
 volumeClaimTemplates:
  - metadata:
     name: data
    spec:
     accessModes:
      - ReadWriteOnce
     resources:
      requests:
       storage: 20Gi
```

Conceptual result:
db-0 → stable identity + PVC-0
db-1 → stable identity + PVC-1
db-2 → stable identity + PVC-2
But production database design lo StatefulSet YAML okkate saripodu.
Need consider:
1. Headless Service
2. Persistent storage
3. StorageClass
4. Backup/restore
5. Database replication
6. Quorum
7. Pod anti-affinity / topology spread
8. Readiness and startup probes
9. Resource requests/limits
10. PodDisruptionBudget
11. Security and RBAC
12. Upgrade/rollback procedure
13. Monitoring and alerting
14. Failure/recovery testing
Especially 3 replicas unte possible ga **same physical Node meeda all replicas land avvakunda** topology rules use cheyyadam important.
Interview lo strong answer:
StatefulSet
↓
Stable Pod identities
↓

Headless Service
↓
Stable DNS
↓
Per-Pod PVC
↓
Persistent storage

+
Database-level replication,
backup, quorum and recovery
Simple ga:
3-node database cluster kosam StatefulSet **stable identity + stable DNS + dedicated persistent storage** provide chestundi, but **actual database HA/replication guarantee StatefulSet alone ivvadu.**

---

15. Job & CronJob — 20 Questions

1. Kubernetes lo Job ante enti?
Ans:
Kubernetes lo **Job** anedi oka task ni run chesi **successfully complete ayye varaku Pods ni manage cheyyadaniki** use chese workload resource.
Deployment continuous ga application run cheyyadaniki use chestam.
Job:
Task Start
↓
Pod Run
↓
Task Complete
↓
Job Complete
Example:
Database migration
Data processing
Batch calculation
One-time script
Suppose:
Job → database-migration
Pod:
Migration script run
↓
Success
↓
Job Completed
Simple ga:
Job = **Oka task ni run chesi successfully complete cheyyadaniki use chese Kubernetes resource.**

---

2. Job and Deployment madhya main difference enti?
Ans:
Deployment **long-running application** kosam.
Job **finite task** kosam.
Deployment:
Pod start
↓
Running continuously
↓
Failure → Replacement
Example:
Web API
Frontend
Backend
Job:
Pod start
↓
Task execute
↓
Complete
↓
Stop
Example:

Database migration
Backup script
Batch processing
Comparison:
Deployment → Long-running service
Job → One-time/finite task
Simple ga:
Deployment = **Application run avuthu undali**
Job = **Task complete ayyaka stop avvali**

---

3. Job lo Pod successful ga complete ayithe em avuthundi?
Ans:
Job target task successful ga complete ayithe Job **Completed** state ki vellachu.
Example:
Job
↓
Pod
↓
Script runs
↓
Exit code 0
↓
Success
Then:
Job → Complete
Pod → Completed
Job automatically same task ni endlessly run cheyyadu.
Check:
kubectl get jobs
Pod:
kubectl get pods
Detailed:
kubectl describe job <job-name>
Simple ga:
Job task success ayithe **Job complete state ki move avuthundi.**

---

4. Job fail ayithe Kubernetes em chestundi?
Ans:
Job Pod failure ayithe, Job configuration batti Kubernetes **another Pod attempt create cheyyachu**.
Example:
Job
↓
Pod 1
↓
Application error
↓
Failed
If retries allowed unte:
Pod 2
↓
Retry
Again fail ayithe:
Pod 3
↓
Retry
Retries ni backoffLimit control chestundi.
Example:
backoffLimit: 4
Meaning Job failure retries ki configured limit untundi; exact Pod attempts and terminal behavior depend on Job configuration.
Simple ga:
Job Pod fail ayithe **Job retry cheyyagaladu**, backoffLimit retry behavior ni control cheyyadaniki use chestam.

---

5. Job lo completions **ante enti?**
Ans:
completions anedi **Job successfully complete cheyyalsina number of successful Pod executions**.
Example:
completions: 3
Meaning:
Successful completion 1
Successful completion 2
Successful completion 3

↓
Job Complete
Suppose:
Pod 1 → Success
Pod 2 → Success
Pod 3 → Success
Appudu Job complete avvachu.
If a Pod fails:
Pod 1 → Failed
successful completion count increase avvadu; retry/another execution may be created based on Job settings.
Simple ga:
completions = **Task successful ga enni times complete avvali.**

---

6. Job lo parallelism **ante enti?**
Ans:
parallelism anedi **same time lo maximum enni Job Pods run cheyyagalavo** specify chestundi.
Example:
completions: 10
parallelism: 3
Meaning total 10 successful completions kavali, but approximately max 3 active Pods at a time.
Flow:
Pod 1 ┐
Pod 2 ─┼→ Run
Pod 3 ┘
↓
Some complete
↓
Next Pods
Eventually:
10 successful completions
↓
Job Complete
Simple ga:
parallelism = **At one time enni Job Pods run cheyyavachho.**

---

7. completions **and** parallelism **madhya difference enti?**
Ans:
Suppose:
completions: 10
parallelism: 3
completions:
Total successful tasks = 10
parallelism:
Simultaneously active tasks ≈ 3
Flow:
10 total tasks
↓
3 run
↓
3 complete
↓
Next 3
↓
...
↓
10 successful
↓
Job Complete
So:
completions = How many successful executions?
parallelism = How many can run simultaneously?
Simple ga:
Completions = **Total work**
Parallelism = **Concurrent work**

---

8. Job lo backoffLimit **ante enti?**
Ans:
backoffLimit anedi Job failure ayinappudu **failed attempts ki retry behavior limit** cheyyadaniki use chestam.
Example:
backoffLimit: 3

Application repeatedly fail ayithe Kubernetes indefinite ga retry cheyyakunda configured limit reach ayyaka Job fail/terminal state ki vellachu.
Conceptual flow:
Attempt 1 → Failed
Attempt 2 → Failed
Attempt 3 → Failed
...
Limit reached
↓
Job Failed
Exact number of Pods created can be affected by other Job settings and failure modes.
Simple ga:
backoffLimit = **Job failure retries ki upper limit.**

---

9. Job lo activeDeadlineSeconds **ante enti?**
Ans:
activeDeadlineSeconds anedi Job **maximum active execution time** ni limit cheyyadaniki use chestam.
Example:
activeDeadlineSeconds: 300
Meaning Job execution 300 seconds active ga continue ayithe deadline reach ayyaka Kubernetes Job ni terminate/fail cheyyachu.
Example:
Job start
↓
0 sec
↓
100 sec
↓
250 sec
↓
300 sec
↓
Deadline
Useful when task accidentally hang ayithe indefinitely run avvakunda stop cheyyadaniki.
Simple ga:
activeDeadlineSeconds = **Job maximum active runtime limit.**

---

10. CronJob ante enti?
Ans:
CronJob anedi **scheduled time ki Jobs ni automatically create cheyyadaniki** use chese Kubernetes resource.
Example:
Every day 12 AM
↓
CronJob
↓
Creates Job
↓
Job creates Pod
↓
Task executes
↓
Complete
Example use cases:
Daily backup
Hourly cleanup
Periodic report generation
Scheduled data processing
Architecture:
CronJob
↓
Job
↓
Pod
↓
Task
Simple ga:
CronJob = **Schedule batti Jobs ni automatically create chese resource.**

---

11. CronJob lo cron schedule expression ante enti?
Ans:
CronJob lo schedule field **Job eppudu create avvali** ani define chestundi.
Example:

schedule: "0 0 * * *"
Meaning commonly:
Every day at 00:00
Another:
schedule: "*/5 * * * *"
Meaning:
Every 5 minutes
Cron expression general format:
minute
hour
day-of-month
month
day-of-week
CronJob timezone behavior should be considered explicitly; modern Kubernetes supports a timezone field, and otherwise schedule interpretation depends on controller configuration.
Simple ga:
Cron schedule = **Job eppudu trigger avvali ani define chese time expression.**

---

12. CronJob and Job madhya relation enti?
Ans:
CronJob direct ga application task run cheyyadu. CronJob **Job objects create chestundi**.
Flow:
CronJob
↓
Scheduled time
↓
Job created
↓
Pod created
↓
Task runs
↓
Job completes
Example:
CronJob = daily-backup
Today:
daily-backup-001 → Job
Tomorrow:
daily-backup-002 → Job
So each schedule execution generally creates a Job.
Simple ga:
CronJob = **Scheduler**
Job = **Actual task execution manager**

---

13. CronJob lo concurrencyPolicy **ante enti?**
Ans:
concurrencyPolicy anedi **previous Job still running unte next scheduled Job em cheyyali** ani control chestundi.
Main options:
Allow
Forbid
Replace
Allow:
Job 1 running
↓
Schedule again
↓
Job 2 also starts
Forbid:
Job 1 running
↓
Next schedule
↓
Skip new Job
Replace:
Job 1 running
↓
Next schedule
↓
Stop/replace previous Job
↓
New Job

Use case:
Long-running backup
overlap avvakudadhu ante Forbid useful avvachu.
Simple ga:
concurrencyPolicy = **Scheduled Jobs overlap ayinappudu behavior.**

---

14. CronJob lo startingDeadlineSeconds **ante enti?**
Ans:
Schedule time miss ayina Job ni **entha late varaku start cheyyadaniki allow cheyyalo** startingDeadlineSeconds control chestundi.
Example:
startingDeadlineSeconds: 300
Suppose Job:
Scheduled → 10:00
Controller temporarily unavailable undi and schedule miss ayindi.
If within allowed deadline:
10:04
↓
Can still start
Deadline exceed ayithe missed run ni skip cheyyachu.
Useful for jobs where stale execution is no longer useful after some time.
Simple ga:
startingDeadlineSeconds = **Missed schedule ni entha late varaku catch-up cheyyavachho.**

---

15. CronJob lo successfulJobsHistoryLimit **and** failedJobsHistoryLimit **enduku use chestam?**
Ans:
CronJob regularly Jobs create chestundi. Old completed Jobs indefinitely retain chesthe cluster lo unnecessary objects accumulate avvachu.
Example:
successfuJobsHistoryLimit: 3
failedJobsHistoryLimit: 1
Meaning old successful/failed Job history ni limited ga retain cheyyadaniki use chestam.
Conceptual:
Successful Jobs:
Job 10
Job 11
Job 12
→ Keep

Older:
Job 7
Job 8
Job 9
→ Cleanup according to policy
This is useful for operational cleanliness.
Simple ga:
History limits = **Old completed Job objects entha retain cheyyalo control cheyyadam.**

---

16. Real-time scenario: Daily backup CronJob create avuthundi, but previous backup 2 hours ga running undi. Next schedule vachinappudu em jaragali?
Ans:
First requirement:
Backup Jobs overlap avvakudadhu.
Appudu concurrencyPolicy: Forbid consider chestha.
Example:
concurrencyPolicy: Forbid
Flow:
10:00
↓
Backup Job 1 starts
↓
Still running
↓
11:00 schedule
↓
Existing Job still active
↓
New Job skipped
This prevents two backup processes simultaneously running.
But business requirement batti Replace or Allow kuda valid avvachu.
For backup specifically, overlap prevent cheyyadam often safer because:
Two backups

↓
Same data/storage
↓
Resource contention
↓
Possible conflicts
Simple ga:
Long-running backup overlap avvakudadhu ante Forbid **is a common choice.**

---

17. Real-time scenario: Job repeatedly fail avuthundi and hundreds of Pods create avuthunnayi. Em check chestav?
Ans:
First Job configuration inspect chestha:
kubectl describe job <job-name>
Check:
backoffLimit
completions
parallelism
activeDeadlineSeconds
Pod failure reason
Pods:
kubectl get pods
Failed Pod logs:
kubectl logs <pod-name>
Possible problem:
parallelism too high
Example:
parallelism: 100
If every Pod immediately fails, many attempts/resources may be consumed.
Also check:
ImagePullBackOff
CrashLoopBackOff
Application error
Permission/RBAC issue
Missing Secret/ConfigMap
Resource shortage
backoffLimit and appropriate parallelism/retry behavior tune cheyyali.
Simple ga:
Job runaway behavior = **parallelism + retries + Pod failure reason + resource consumption check cheyyali.**

---

18. Real-time scenario: CronJob expected time ki Job create cheyyadam ledu. Ela troubleshoot chestav?
Ans:
First CronJob status:
kubectl get cronjob
Detailed:
kubectl describe cronjob <cronjob-name>
Check:
Schedule correct aa?
Last schedule eppudu?
Suspended aa?
Concurrency policy emundi?
Starting deadline?
Controller issues?
Then Jobs:
kubectl get jobs
If Job create ayindi but Pod run avvatledu:
kubectl get pods
kubectl describe pod <pod-name>
Also verify cron timezone expectations.
Possible reasons:
Wrong cron expression
CronJob suspended
Missed schedule/deadline
ConcurrencyPolicy
Controller/control-plane issue
Job creation issue
Simple ga:
CronJob not triggering = **CronJob status → schedule → suspend → deadline/concurrency → created Jobs → Pods.**

---

19. Job lo restartPolicy **enduku important?** Always **use cheyyacha?**
Ans:
Job Pod template lo restartPolicy typically:

Never
OnFailure
use cheyyali.
Example:
restartPolicy: Never
Meaning container fail ayithe Pod container restart కాకుండా Job controller **another Pod attempt create cheyyagaladu**, depending on Job behavior.
OnFailure:
restartPolicy: OnFailure
Container failure ayithe same Pod lo container restart avvachu.
Deployment-style:
restartPolicy: Always
Job Pod template ki valid option kaadu.
Important difference:
Pod restart behavior
+
Job retry behavior
rendu separate concepts.
Simple ga:
Job Pods ki restartPolicy = Never **or** OnFailure**;** Always **use cheyyakudadhu.**

---

20. Real-time interview scenario: Every night database backup run cheyyali. Production-ready CronJob design ela chestav?
Ans:
Requirement:
Every night
↓
Database backup
↓
Upload to backup storage
↓
Complete
Basic design:

```
apiVersion: batch/v1
kind: CronJob
metadata:
 name: db-backup
spec:
 schedule: "0 0 * * *"
 concurrencyPolicy: Forbid
 successfulJobsHistoryLimit: 3
 failedJobsHistoryLimit: 3
 jobTemplate:
  spec:
   backoffLimit: 3
   activeDeadlineSeconds: 3600
   template:
    spec:
     restartPolicy: OnFailure
     containers:
      - name: backup
        image: example/backup:1.0
        envFrom:
         - secretRef:
            name: db-backup-secret
```

Flow:
Midnight
↓
CronJob
↓
Job
↓
Backup Pod
↓
Read DB credentials
↓
Create backup
↓
Upload backup
↓
Success
↓

Job Complete
Production lo important considerations:

- concurrencyPolicy: Forbid — overlapping backups avoid cheyyadaniki.
- backoffLimit — transient failures ki controlled retries.
- activeDeadlineSeconds — backup hang ayithe bounded runtime.
- Secrets — DB credentials ni image/YAML lo hard-code cheyyakunda Secret use cheyyali.
- Resource requests/limits — backup Pod cluster resources ni control cheyyali.
- Backup storage — durable external/persistent destination.
- Monitoring/alerting — failed backup immediately detect cheyyali.
- Retention — old backups lifecycle policy maintain cheyyali.
- Restore testing — **backup successful ani matrame enough kaadu; restore test kuda important.**
- Timezone — backup exact local time requirement unte CronJob timezone behavior explicitly configure/verify cheyyali.

Interview lo strong answer:
CronJob
↓
Scheduled Job
↓
Controlled retry
↓
Backup Pod
↓
Secure credentials
↓
Durable backup storage
↓
Monitoring + alerting
↓
Periodic restore testing
Simple ga:
Production CronJob = **Schedule + controlled concurrency + retries + timeout + secure credentials + durable storage + monitoring + restore validation.**

---

16. Ingress — 20 Questions

1. Kubernetes lo Ingress ante enti?
Ans:
Ingress anedi Kubernetes lo **external HTTP/HTTPS traffic ni cluster lo unna Services ki route cheyyadaniki** use chese API resource.
Suppose nee cluster lo:
Frontend Service
Backend Service
Payment Service
External users:
User
↓
Ingress
↓
Service
↓
Pods
Ingress rules use chesi hostname/path based routing cheyyachu.
Example:
example.com → Frontend Service
example.com/api → Backend Service
example.com/payment → Payment Service
Ingress itself usually **traffic process chese proxy/load balancer kaadu**. Ingress resource ni actual traffic handling kosam **Ingress Controller** implement chestundi.
Simple ga:
Ingress = **External HTTP/HTTPS traffic ni Services ki route cheyyadaniki rules define chese Kubernetes resource.**

---

2. Ingress and Service madhya difference enti?
Ans:
Service mainly **Pods ni expose cheyyadaniki stable network endpoint** provide chestundi.
Ingress mainly **external HTTP/HTTPS traffic ni multiple Services ki route cheyyadaniki** use chestam.
Example:
Internet
↓
Ingress
├— / → Frontend Service

├── /api → Backend Service
└── /pay → Payment Service
Services:
Frontend Service → Frontend Pods
Backend Service → Backend Pods
Payment Service → Payment Pods
So:
Service
→ Pod access / exposure

Ingress
→ HTTP/HTTPS routing to Services
Simple ga:
Service = **Pods ki stable endpoint**
Ingress = **External HTTP/HTTPS routing layer**

---

3. Ingress Controller ante enti?
Ans:
Ingress resource lo rules define chestam. Aa rules ni actually **watch chesi traffic handle chese component** ni Ingress Controller antam.
Architecture:
User
↓
Load Balancer
↓
Ingress Controller
↓
Ingress Rules
↓
Service
↓
Pods
Popular implementations examples:
NGINX Ingress Controller
HAProxy
Traefik
Cloud-provider-specific controllers
Important point:
Ingress Resource
≠
Ingress Controller
Ingress = configuration/routing rules.
Ingress Controller = those rules ni implement chesi traffic handle chese software.
Simple ga:
Ingress Resource = **Rules**
Ingress Controller = **Rules ni execute chese component**

---

4. Ingress Controller lekunda Ingress work avuthunda?
Ans:
Normally **Ingress resource create cheyyadam matrame sufficient kaadu**.
Example:
Ingress Resource
↓
Rules defined
But controller lekapothe:
Who will process these rules?
↓
No Ingress Controller
↓
Traffic routing won't be implemented
So cluster lo suitable Ingress Controller deploy/configure cheyyali.
Flow:
Ingress YAML
↓
Ingress Controller watches it
↓
Controller configures proxy/load balancer
↓
External traffic routed
Simple ga:
Ingress resource create cheyyadam alone saripodu; **Ingress Controller required to implement it.**

5. Ingress lo Host-based routing ante enti?
Ans:
Host-based routing lo **different domain/hostname batti traffic ni different Services ki route chestam**.
Example:
app.example.com
↓
Frontend Service

api.example.com
↓
Backend Service

admin.example.com
↓
Admin Service
Ingress:
rules:
- host: app.example.com
...
- host: api.example.com
...
- host: admin.example.com
...
Flow:
User → api.example.com
↓
Ingress
↓
Backend Service
↓
Backend Pods
Simple ga:
Host-based routing = **Domain/hostname batti Service select cheyyadam.**

---

6. Ingress lo Path-based routing ante enti?
Ans:
Path-based routing lo **URL path batti different Services ki traffic route chestam**.
Example:
example.com/
↓
Frontend Service

example.com/api
↓
Backend Service

example.com/admin
↓
Admin Service
Ingress rules:
paths:
- path: /
...
- path: /api
...
- path: /admin
...
Flow:
User → example.com/api/users
↓
Ingress
↓
Backend Service
↓
Backend Pod
Simple ga:
Path-based routing = **URL path batti backend Service select cheyyadam.**

---

7. Ingress lo TLS ante enti?

Ans:
TLS use chesi external client → Ingress traffic ni **HTTPS encrypted communication** ga provide cheyyachu.
Example:
https://example.com
↓
Ingress
↓
Service
↓
Pods
Ingress TLS configuration lo Secret reference untundi:

```
tls:
 - hosts:
    - example.com
   secretName: example-tls
```

Secret lo normally:
TLS certificate
Private key
untayi.
Flow:
Client
↓ HTTPS
Ingress
↓
Service
↓
Pod
Simple ga:
Ingress TLS = **External HTTPS traffic kosam certificate/private key configuration.**

---

8. Ingress lo TLS Secret enduku use chestam?
Ans:
HTTPS establish cheyyadaniki server ki **TLS certificate and private key** avasaram.
Kubernetes lo avi commonly Secret lo store chestam.
Example:
Secret

```
├── tls.crt
└── tls.key
```

Ingress:
Ingress
↓
TLS Secret
↓
Certificate
↓
HTTPS
User:
https://example.com
Ingress TLS terminate chesthe client connection encryption Ingress daggara terminate avvachu.
After that:
Ingress → Service → Pod
cluster-internal communication HTTP or HTTPS ela undali anedi separate architecture/security decision.
Simple ga:
TLS Secret = **Ingress ki HTTPS certificate/private key provide cheyyadaniki.**

---

9. Ingress TLS termination ante enti?
Ans:
TLS termination ante **HTTPS encryption ni Ingress/load-balancer layer daggara decrypt cheyyadam**.
Flow:
Client
↓ HTTPS
Ingress
↓ TLS termination
↓ HTTP
Service
↓
Pod
Example:
Client → https://example.com
Ingress certificate use chesi encrypted connection terminate chestundi.
Then backend communication:

Ingress → Backend Service → Pod
HTTP or HTTPS ga undochu, requirement batti.
If end-to-end encryption kavali:
Client
↓ HTTPS
Ingress
↓ HTTPS
Service
↓ HTTPS
Pod
configure cheyyachu, provided backend supports it.
Simple ga:
TLS termination = **HTTPS ni Ingress daggara decrypt/terminate cheyyadam.**

---

10. Ingress lo default backend ante enti?
Ans:
Ingress rules ki match kaani request ki **default backend** response ivvadaniki use cheyyachu.
Suppose:
example.com/
→ Frontend

example.com/api
→ Backend
User:
example.com/unknown
No matching rule unte configured default backend behavior apply avvachu.
Conceptually:
Request
↓
Ingress
↓
Matching rule?
├── Yes → Specific Service
└── No → Default backend
Exact default behavior depends on Ingress Controller implementation/configuration.
Simple ga:
Default backend = **No Ingress rule match ayina request kosam fallback backend.**

---

11. Ingress lo annotations enduku use chestam?
Ans:
Annotations ni Ingress Controller-specific **additional configuration** kosam historically use chestaru.
Example:
metadata:
annotations:
nginx.ingress.kubernetes.io/proxy-body-size: "10m"
Controller behavior ni configure cheyyadaniki annotations use avvachu.
Examples:
Request size
Rewrite behavior
Redirect behavior
Rate limiting
Proxy settings
But important point:
Annotations are usually controller-specific.
So NGINX-specific annotation Traefik controller lo work avvakapovachu.
Modern Kubernetes lo some capabilities ki dedicated API fields/CRDs better option avvachu.
Simple ga:
Ingress annotations = **Controller-specific extra configuration.**

---

12. Ingress lo path rewrite ante enti?
Ans:
Sometimes external URL path and backend application expect chese path different ga untayi.
Example:
User:
example.com/api/users
Backend application maybe expects:
/users
Ingress Controller rewrite configuration use chesi:
Client path:
/api/users

↓ rewrite

Backend:
/users
This behavior controller-specific configuration batti vary avvachu.
Simple ga:
Path rewrite = **Client request path ni backend expected path ki transform cheyyadam.**

---

13. Ingress and LoadBalancer Service madhya difference enti?
Ans:
Service type: LoadBalancer generally **one Service ni external load balancer through expose cheyyadaniki** use chestam.
Example:
Internet
↓
LoadBalancer Service
↓
Backend Pods
Ingress:
Internet
↓
Ingress
├── / → Frontend Service
├── /api → Backend Service
└── /admin → Admin Service
So one Ingress can route multiple HTTP/HTTPS applications based on host/path.
Common architecture:
Internet
↓
External Load Balancer
↓
Ingress Controller
↓
Ingress rules
↓
Multiple Services
Simple ga:
LoadBalancer Service = **Service exposure**
Ingress = **HTTP/HTTPS routing across Services**

---

14. Ingress lo IngressClass **ante enti?**
Ans:
Cluster lo multiple Ingress Controllers unte **which controller should handle this Ingress** ani identify cheyyadaniki IngressClass use chestam.
Example:
IngressClass:
nginx
Ingress:
spec:
ingressClassName: nginx
Flow:
Ingress
↓
ingressClassName = nginx
↓
NGINX Ingress Controller
↓
Handles rules
Another controller:
traefik
unte correct IngressClass select cheyyali.
Simple ga:
IngressClass = **Ee Ingress ni ye Ingress Controller handle cheyyalo identify cheyyadam.**

---

15. Ingress lo pathType **ante enti?**
Ans:
Ingress path rule lo pathType **path matching behavior** define chestundi.
Common values:
Exact
Prefix
ImplementationSpecific
Exact:

path: /api
pathType: Exact
/api exact match.
/api/users automatically same exact path kaadu.
Prefix:
path: /api
pathType: Prefix
/api, /api/users, /api/orders lanti paths match avvachu according to Kubernetes path matching semantics.
ImplementationSpecific:
Controller implementation batti behavior.
Production lo expected behavior clear ga undali.
Simple ga:
pathType = **Ingress URL path ela match avvali ani define cheyyadam.**

---

16. Real-time scenario: example.com **frontend ki vellali,** example.com/api **backend ki vellali. Ingress ela design chestav?**
Ans:
First Services:
frontend-service
backend-service
Ingress routing:
example.com/
↓
frontend-service

example.com/api
↓
backend-service
Example:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: app-ingress
spec:
 ingressClassName: nginx
 rules:
  - host: example.com
    http:
     paths:
      - path: /api
        pathType: Prefix
        backend:
         service:
           name: backend-service
           port:
            number: 80
      - path: /
        pathType: Prefix
        backend:
         service:
           name: frontend-service
           port:
            number: 80
```

Flow:
example.com/api/users
↓
Ingress
↓
backend-service
↓
backend Pods
And:
example.com/
↓
Ingress
↓
frontend-service
↓
frontend Pods
Simple ga:
Path-based Ingress tho **same domain nunchi multiple Services ki route cheyyachu.**

---

17. Real-time scenario: Ingress create chesav, but browser lo application open avvatledu. Ela troubleshoot chestav?
Ans:
Nenu bottom-up ga troubleshoot chestha.
First Ingress:
kubectl get ingress
kubectl describe ingress <ingress-name>
IngressClass:
kubectl get ingressclass
Controller running aa:
kubectl get pods -A
Then Service:
kubectl get svc
kubectl describe svc <service-name>
Service ki endpoints unnaya:
kubectl get endpoints
Modern clusters lo EndpointSlices kuda check cheyyachu:
kubectl get endpointslices
Then Pods:
kubectl get pods -o wide
Pod readiness:
Ready = True?
Then DNS:
example.com
↓
Correct IP?
TLS issue unte:
Certificate
Secret
Hostname
check chestha.
Flow:
Browser
↓
DNS
↓
Load Balancer
↓
Ingress Controller
↓
Ingress Rule
↓
Service
↓
Endpoints
↓
Pods
Simple ga:
Ingress issue ni **DNS → Load Balancer → Controller → Ingress → Service → Endpoints → Pods** order lo isolate chestha.

---

18. Real-time scenario: Ingress 404 **return chestundi. Main reasons enti?**
Ans:
Ingress 404 ante request **expected backend/rule ki reach avvalekapovachu**.
Possible reasons:
1. Wrong hostname
2. Path mismatch
3. Wrong pathType
4. Ingress rule not loaded by controller
5. Wrong IngressClass
6. Wrong namespace/service reference
7. Controller-specific routing behavior
8. Backend application itself returns 404
First:
kubectl describe ingress <name>
Check:
Host
Path
Backend Service
Port
IngressClass
Then Service:
kubectl get svc

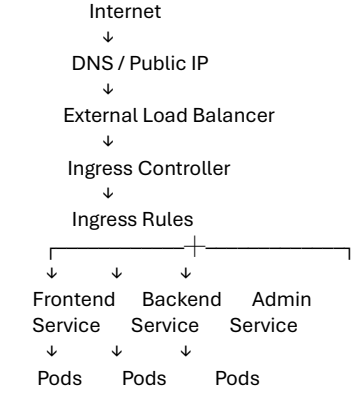
kubectl get endpointslices
Important distinction:
Ingress Controller 404
vs
Backend application 404
Both look similar to user, but root cause different.
Simple ga:
Ingress 404 = **Host/path/controller/backend mapping first verify cheyyali.**

---

19. Real-time scenario: HTTPS lo 502 Bad Gateway **vastundi. Ela troubleshoot chestav?**
Ans:
502 generally proxy/Ingress Controller backend ni properly reach cheyyalekapovadam lanti issue indicate cheyyachu.
First Ingress:
kubectl describe ingress <ingress-name>
Backend Service:
kubectl get svc
kubectl describe svc <service-name>
Endpoints:
kubectl get endpointslices
If no endpoints:
Service selector
↓
No matching Ready Pods
↓
No backend
↓
Possible 502
Check Pods:
kubectl get pods -o wide
kubectl describe pod <pod-name>
Also verify Service port mapping:
Ingress backend port
↓
Service port
↓
targetPort
↓
Container listening port
TLS itself separate ga verify cheyyali.
Simple ga:
502 troubleshooting = **Ingress → Service → Endpoints → targetPort → Pod/application listening port** check cheyyali.

---

20. Real-time interview scenario: Production lo multiple applications ni single public endpoint nunchi expose cheyyali. Ingress architecture explain cheyyi.
Ans:
Suppose production lo:
Frontend
Backend API
Admin Portal
Payment API
Each application ki separate Service untundi:
frontend-service
backend-service
admin-service
payment-service
External architecture:



Host-based example:
app.example.com
→ frontend-service

api.example.com
→ backend-service

admin.example.com
→ admin-service
Path-based example:
example.com/
→ frontend-service

example.com/api
→ backend-service

example.com/admin
→ admin-service
Production lo additionally:
1. HTTPS/TLS
2. Certificate management
3. DNS
4. IngressClass
5. Controller health
6. Backend Service health
7. Readiness probes
8. Rate limiting/WAF where appropriate
9. Access logs and monitoring
10. High availability
11. Security policies
12. Controller-specific configuration
Important interview point:
Ingress Resource
↓
Routing rules

Ingress Controller
↓
Actually implements routing

Service
↓
Stable backend endpoint

Pods
↓
Application
Simple ga:
Production Ingress architecture = **Internet → DNS/LB → Ingress Controller → Ingress rules → Services → Pods**, with TLS, monitoring and security around it.

---

17. RBAC & ServiceAccount — 20 Questions

1. Kubernetes lo RBAC ante enti?
Ans:
RBAC ante **Role-Based Access Control**. Kubernetes cluster lo **evaru em actions cheyyagalru, ye resources meeda cheyyagalru** ani control cheyyadaniki RBAC use chestam.
Example:
Developer
↓
Can view Pods
↓
Cannot delete Pods
Inko user:
Admin
↓
Can create
Can update
Can delete
RBAC permissions ni roles dwara define chestam.

Common resources:
Role
ClusterRole
RoleBinding
ClusterRoleBinding
Simple ga:
RBAC = **Kubernetes lo Who can do What on Which Resource ani control cheyyadam.**

---

2. Kubernetes RBAC lo main components enti?
Ans:
RBAC lo commonly four main objects untayi:
1. Role
2. ClusterRole
3. RoleBinding
4. ClusterRoleBinding
Their purpose:
Role
→ Namespace-scoped permissions

ClusterRole
→ Cluster-level permissions / reusable permissions

RoleBinding
→ Role/ClusterRole ni namespace lo subject ki bind chestundi

ClusterRoleBinding
→ ClusterRole ni cluster-wide subject ki bind chestundi
Flow:
Subject
↓
Binding
↓
Role / ClusterRole
↓
Permissions
Simple ga:
Role = **Permissions**
Binding = **Aa permissions evariki ivvali**

---

3. Role ante enti?
Ans:
Role anedi **specific Namespace lo permissions define cheyyadaniki** use chestam.
Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

Meaning:
Namespace = dev

Pods:
get ✅
list ✅
watch ✅

delete ❌
create ❌
Important:
Role itself user ki permission grant cheyyadu.
Binding required:
Role
↓
RoleBinding
↓
User/Group/ServiceAccount

Simple ga:
Role = **Oka namespace lo specific resource permissions define chese object.**

---

4. ClusterRole ante enti?
Ans:
ClusterRole anedi **cluster-level permissions define cheyyadaniki** use chestam.
Example:

```
kind: ClusterRole
rules:
 - apiGroups: [""]
   resources: ["nodes"]
   verbs: ["get", "list"]
```

Nodes cluster-scoped resource kabatti cluster-level access kavali.
ClusterRole ni cluster-wide bind cheyyachu:

```
ClusterRole
  ↓
ClusterRoleBinding
  ↓
ServiceAccount
```

ClusterRole ni RoleBinding dwara **specific namespace context lo** kuda bind cheyyachu.
So ClusterRole = always cluster-wide ani strict ga assume cheyyakudadhu; **how it is bound matters**.
Simple ga:
ClusterRole = **Reusable/cluster-level permission definition.**

---

5. Role and ClusterRole madhya difference enti?
Ans:
Role namespace-scoped.

```
dev namespace
  ↓
Role
  ↓
Pods
```

ClusterRole cluster-scoped permissions define cheyyagaladu.

```
ClusterRole
  ↓
Nodes
Namespaces
Cluster-wide resources
```

Example:
Role:
→ dev namespace Pods ni read cheyyachu

ClusterRole:
→ Nodes ni read cheyyachu
But ClusterRole ni RoleBinding tho specific namespace lo namespaced resources ki access ivvadaniki kuda use cheyyachu.
Simple ga:
Role = **Namespace-scoped definition**
ClusterRole = **Cluster-scoped/reusable definition**

---

6. RoleBinding ante enti?
Ans:
RoleBinding anedi **Role or ClusterRole permissions ni oka subject ki assign cheyyadaniki** use chestam.
Example:

```
Role
 ↓
pod-reader
User:
developer
Binding:
RoleBinding
 ↓
developer
 ↓
pod-reader
Result:
developer
  ↓
Can read Pods
```

RoleBinding itself permissions define cheyyadu.
It says:
Ee subject ki ee role permissions apply cheyyali.

Simple ga:
RoleBinding = **Role permissions ni User/Group/ServiceAccount ki attach cheyyadam.**

---

7. ClusterRoleBinding ante enti?
Ans:
ClusterRoleBinding anedi **ClusterRole ni cluster-wide subject ki bind cheyyadaniki** use chestam.
Example:
ClusterRole
↓
node-reader
Binding:
ClusterRoleBinding
↓
ServiceAccount
Result:
ServiceAccount
↓
Can read Nodes cluster-wide
Because Node cluster-scoped resource.
Difference:
RoleBinding
→ Namespace context

ClusterRoleBinding
→ Cluster-wide grant
Simple ga:
ClusterRoleBinding = **ClusterRole permissions ni cluster-wide subject ki grant cheyyadam.**

---

8. ServiceAccount ante enti?
Ans:
ServiceAccount anedi Kubernetes lo **Pod/application ki identity provide cheyyadaniki** use chese object.
Human user:
Developer
Application:
Pod
↓
ServiceAccount
Suppose application Kubernetes API ni access cheyyali.
Pod
↓
ServiceAccount
↓
RBAC permissions
↓
Kubernetes API
Example:
apiVersion: v1
kind: ServiceAccount
metadata:
name: app-sa
Pod:
serviceAccountName: app-sa
Simple ga:
ServiceAccount = **Application/Pod ki Kubernetes identity.**

---

9. User and ServiceAccount madhya difference enti?
Ans:
Kubernetes lo **human users** and **ServiceAccounts** different purposes kosam use chestam.
Human:
Developer
Admin
Operator
Application:
Pod
↓
ServiceAccount
Example:
Developer
→ kubectl use chestadu

Application Pod

→ ServiceAccount identity use chestundi
RBAC permissions rendu cases lo apply cheyyachu.
Example:
Developer → read Pods
App SA → read ConfigMaps
Simple ga:
User = **Human/client identity**
ServiceAccount = **Workload/application identity**

---

10. Pod ki ServiceAccount ela assign chestam?
Ans:
Pod specification lo:

```
spec:
 serviceAccountName: app-sa
```

Example:

```
apiVersion: v1
kind: Pod
metadata:
 name: my-app
spec:
 serviceAccountName: app-sa
 containers:
  - name: app
    image: example/app:1.0
```

Flow:
Pod
↓
serviceAccountName: app-sa
↓
ServiceAccount
↓
RBAC
↓
Allowed API operations
Deployment use chesthe Pod template lo set chestam:

```
spec:
 template:
  spec:
   serviceAccountName: app-sa
```

Simple ga:
Pod spec lo serviceAccountName set chesi **specific ServiceAccount identity assign chestam.**

---

11. ServiceAccount ki permissions automatically untaya?
Ans:
ServiceAccount create cheyyadam valla **arbitrary Kubernetes API permissions automatically raavu**.
Example:
ServiceAccount
↓
No custom RBAC binding
↓
Only baseline/default access behavior
If application ki Pods read permission kavali:
ServiceAccount
↓
RoleBinding
↓
Role
↓
get/list/watch Pods
So ServiceAccount identity and RBAC authorization separate concepts.
ServiceAccount
= Who am I?

RBAC
= What can I do?
Simple ga:
ServiceAccount = **Identity**
RBAC = **Permissions**

---

12. verbs **in RBAC ante enti?**

Ans:
RBAC verbs anedi **resource meeda em action perform cheyyavachho** define chestundi.
Common verbs:
get
list
watch
create
update
patch
delete
Example:
verbs:
 - get
 - list
 - watch
Meaning:
Read individual resource → get
List resources → list
Watch changes → watch
If:
verbs:
 - delete
only delete permission grant chestundi for matching resource/rule.
Simple ga:
RBAC verb = **Resource meeda ye action allowed ani define cheyyadam.**

---

13. RBAC lo Least Privilege principle ante enti?
Ans:
Least Privilege ante **required permissions matrame ivvali; unnecessary permissions ivvakudadhu**.
Bad design:
Application
 ↓
cluster-admin
Application ki actually Pods read cheyyadam matrame kavali anuko.
Better:
ServiceAccount
 ↓
Role
 ↓
get/list/watch Pods
Not:
create/delete/update everything
Why?
Compromised Pod
 ↓
Limited RBAC
 ↓
Limited blast radius
Simple ga:
Least Privilege = **Workload ki entha permission avasaramo anthe ivvadam.**

---

14. cluster-admin **role ni application ki ivvadam enduku dangerous?**
Ans:
cluster-admin extremely powerful permission set.
Application Pod compromise ayithe:
Compromised Pod
 ↓
cluster-admin credentials
 ↓
Potentially broad cluster control
Attacker potentially:
Secrets read
Workloads modify
RBAC modify
Resources delete
Other namespaces impact
cheyyagalige risk untundi, depending on cluster configuration.
Instead:
Application
↓
Dedicated ServiceAccount

↓
Minimal Role
↓
Only required permissions
Simple ga:
Production application ki cluster-admin ivvadam **least-privilege principle ki opposite and high-risk.**

---

15. Real-time scenario: Application Pod Kubernetes API nunchi Pods list cheyyali. Ela configure chestav?
Ans:
Suppose application ki:
GET Pods
LIST Pods
WATCH Pods
matrame kavali.
Step 1: ServiceAccount:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pod-reader
  namespace: app
```

Step 2: Role:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: app
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
```

Step 3: RoleBinding:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: app
subjects:
  - kind: ServiceAccount
    name: pod-reader
    namespace: app
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Step 4: Pod:

```
spec:
  serviceAccountName: pod-reader
```

Flow:
Application Pod
↓
ServiceAccount
↓
RoleBinding
↓
Role
↓
get/list/watch Pods
↓
Kubernetes API
Simple ga:
Pod → ServiceAccount → RoleBinding → Role → required API permission.

---

16. Real-time scenario: ServiceAccount ki permission undi ani check cheyyali. Ela verify chestav?
Ans:
Kubernetes lo kubectl auth can-i very useful.
Example:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:app:pod-reader \
  -n app
```

If allowed:
yes

Not allowed:
no
Specific action:
kubectl auth can-i delete pods \
  --as=system:serviceaccount:app:pod-reader \
  -n app
Can test:
get
list
watch
create
update
delete
Different resource kosam kuda.
Simple ga:
kubectl auth can-i = **RBAC permission actually allowed aa leda verify cheyyadaniki best troubleshooting command.**

---

17. Real-time scenario: Pod Kubernetes API call chestundi, but Forbidden **error vastundi. Ela troubleshoot chestav?**
Ans:
First error ni carefully read chestha:
User/ServiceAccount
Verb
Resource
Namespace
Then Pod ye ServiceAccount use chestundo:
kubectl get pod <pod-name> \
  -o jsonpath='{.spec.serviceAccountName}'
ServiceAccount:
kubectl get sa <sa-name> -n <namespace>
Bindings:
kubectl get rolebinding -n <namespace>
kubectl get clusterrolebinding
Role:
kubectl describe role <role-name> -n <namespace>
Permission test:
kubectl auth can-i get pods \
  --as=system:serviceaccount:<namespace>:<sa-name> \
  -n <namespace>
Possible reasons:
Wrong ServiceAccount
No RoleBinding
Wrong namespace
Wrong resource
Wrong API group
Missing verb
RoleBinding references wrong Role
Simple ga:
Forbidden = **Identity → Binding → Role → Resource → Verb → Namespace** chain verify cheyyali.

---

18. Real-time scenario: Developer ki dev **namespace lo Pods manage cheyyadaniki access kavali, but production namespace access undakudadhu. Ela design chestav?**
Ans:
Developer ki dev namespace lo required permissions matrame ivvali.
Example:
Developer
  ↓
RoleBinding
  ↓
Role
  ↓
dev namespace
Role:
pods:
get
list
watch
create
update
delete
RoleBinding:
Namespace = dev

Important:
dev access
≠
prod access
Production namespace lo binding create cheyyakapothe developer ki aa RoleBinding dwara production permissions raavu.
If developer ki only dev access kavali, avoid unnecessary ClusterRoleBinding.
Simple ga:
Namespace-specific developer access = **Role + RoleBinding in** dev**; unnecessary cluster-wide binding avoid cheyyali.**

---

19. Real-time scenario: Oka ServiceAccount ki get/list Pods **permission ivvali, but** delete Pods **permission ivvakudadhu. Ela explain chestav?**
Ans:
Role lo required verbs matrame define chestha:

```
rules:
 - apiGroups: [""]
   resources: ["pods"]
   verbs:
     - get
     - list
     - watch
```

Ikkada:
get ✅
list ✅
watch ✅
delete ❌
create ❌
update ❌
ServiceAccount:
app-sa
RoleBinding:
app-sa
↓
pod-reader Role
Permission test:

```
kubectl auth can-i delete pods \
  --as=system:serviceaccount:app:app-sa \
  -n app
```

Expected:
no
This is a good example of **least privilege**.
Simple ga:
Required verbs only grant cheyyali; unwanted verbs intentionally omit cheyyali.

---

20. Real-time interview scenario: Production Pod compromise ayindi. Attacker ServiceAccount token use chesi cluster lo resources access cheyyadaniki try chestunnadu. RBAC ela help chestundi?
Ans:
Production workload ki dedicated ServiceAccount use chesi, **minimum permissions only** assign chesthe compromise impact reduce cheyyachu.
Secure design:
Application Pod
↓
Dedicated ServiceAccount
↓
Minimal Role
↓
Only required resources
↓
Only required verbs
Example application ki:
ConfigMap read
Secret read
matrame kavali anuko.
Then:
get ConfigMaps ✅
get Secrets ✅
delete Pods ❌
create Deployments ❌
modify RBAC ❌
read Nodes ❌
Attacker token use chesina:

Compromised Pod
↓
ServiceAccount identity
↓
RBAC authorization
↓
Only allowed operations
Additional production security:
1. Least-privilege RBAC
2. Dedicated ServiceAccounts
3. Avoid cluster-admin
4. Avoid unnecessary ClusterRoleBindings
5. Restrict Secret access
6. Monitor API audit logs
7. Keep Kubernetes and workloads updated
8. Use network/security controls as defense in depth
Important interview point:
RBAC authentication kaadu. RBAC authorization.
Authentication
→ "Nuvvu evaru?"

Authorization / RBAC
→ "Nuvvu em cheyyagalavu?"
Simple ga:
ServiceAccount = **workload identity**
RBAC = **authorization**
Least privilege = **compromise ayina blast radius ni minimize cheyyadam.**

---

18. Taints & Tolerations — 20 Questions

1. Kubernetes lo Taint ante enti?
Ans:
Taint anedi **Node meeda rule/policy laga apply chesi, certain Pods ni aa Node meeda schedule avvakunda prevent cheyyadaniki** use chestam.
Suppose cluster lo special Node undi:
Node → GPU Node
Manaki GPU use cheyyani normal Pods aa Node meeda schedule avvakudadhu.
Appudu Node ki taint add chestam:
Node
↓
Taint
↓
Normal Pod → Not tolerated → Cannot schedule
Example concept:
gpu=true:NoSchedule
Meaning aa taint ni tolerate cheyyani Pods ni Node ki schedule cheyyakudadhu.
Simple ga:
Taint = **Node nunchi unwanted Pods ni discourage/block cheyyadaniki apply chese rule.**

---

2. Kubernetes lo Toleration ante enti?
Ans:
Toleration anedi **Pod ki particular Node taint ni tolerate cheyyagaladani declare cheyyadam**.
Node:
Taint:
gpu=true:NoSchedule
Pod:
Toleration:
gpu=true:NoSchedule
Appudu Pod aa taint ni tolerate chestundi and scheduling ki eligible avvachu.
Flow:
Node
↓
Taint
↓
Pod checks
↓
Toleration matches
↓
Pod can be scheduled

Important:
Toleration alone Pod ni aa Node ki force cheyyadu.
Other scheduling rules still apply.
Simple ga:
Toleration = **Pod ki "Ee Node taint ni nenu tolerate cheyyagalanu" ani declare cheyyadam.**

---

3. Taint and Toleration madhya difference enti?
Ans:
Taint **Node side** lo untundi.
Node
↓
Taint
Toleration **Pod side** lo untundi.
Pod
↓
Toleration
Example:
Node:
gpu=true:NoSchedule

Pod A:
No toleration
→ Cannot schedule

Pod B:
Matching toleration
→ Can be considered
So:
Taint = Node restriction
Toleration = Pod exception/permission to tolerate
Simple ga:
Taint = **Node says "avoid me unless you tolerate me."**
Toleration = **Pod says "I can tolerate this taint."**

---

4. Taint lo effect ante enti?
Ans:
Taint lo effect anedi **taint match/tolerate cheyyani Pods meeda em behavior apply cheyyalo** define chestundi.
Main effects:
NoSchedule
PreferNoSchedule
NoExecute
NoSchedule:
Non-tolerating new Pods
→ Won't be scheduled
PreferNoSchedule:
Avoid scheduling if possible
It is a preference rather than a hard guarantee.
NoExecute:
New non-tolerating Pods
→ won't schedule

Existing non-tolerating Pods
→ can be evicted
Simple ga:
effect = **Taint ni tolerate cheyyani Pod ki Kubernetes em cheyyalo.**

---

5. NoSchedule **ante enti?**
Ans:
NoSchedule effect unna taint ni Pod tolerate cheyyakapothe **Kubernetes aa Pod ni Node meeda newly schedule cheyyadu**.
Example:
Node taint:
gpu=true:NoSchedule
Pod A:
No toleration
Result:
Pod A
↓
Cannot schedule on that Node
Pod B:
Matching toleration
Result:

Pod B
↓
Eligible for that taint
Important:
NoSchedule primarily affects **scheduling of new Pods**. It does not by itself evict already-running Pods just because the taint was added.
Simple ga:
NoSchedule = **Non-tolerating new Pods aa Node ki schedule avvakudadhu.**

---

6. PreferNoSchedule **ante enti?**
Ans:
PreferNoSchedule anedi **soft restriction**.
Kubernetes possible ayithe aa taint unna Node ni avoid cheyyadaniki try chestundi, but absolute block kaadu.
Example:
Node taint:
special=true:PreferNoSchedule
Pod ki toleration lekapoyina:
Other suitable Nodes available
→ Prefer other Nodes
But:
No other suitable Node
unte Pod aa Node meeda schedule ayye possibility undi.
Comparison:
NoSchedule
→ Hard scheduling restriction

PreferNoSchedule
→ Soft preference
Simple ga:
PreferNoSchedule = **Possible ayithe avoid cheyyi, but absolute block kaadu.**

---

7. NoExecute **ante enti?**
Ans:
NoExecute effect strong behavior provide chestundi.
Tolerate cheyyani **new Pods schedule avvakunda** prevent cheyyadame kakunda, already running Pods ni kuda **evict cheyyagaladu**.
Example:
Node
↓
Taint:
maintenance=true:NoExecute
Existing Pod:
No matching toleration
Result:
Running Pod
↓
Eviction
New Pod:
No toleration
↓
Cannot schedule
So:
NoSchedule
→ New Pods blocked

NoExecute
→ New Pods blocked + existing non-tolerating Pods may be evicted
Simple ga:
NoExecute = **Non-tolerating existing Pods ni kuda Node nunchi evict cheyyagaladu.**

---

8. Node ki Taint ela add chestam?
Ans:
kubectl taint nodes command use cheyyachu.
Example:
kubectl taint nodes node1 gpu=true:NoSchedule
Meaning:
Node = node1
Key = gpu
Value = true
Effect = NoSchedule
Now:

Pod without matching toleration
→ Cannot schedule on node1
Check:
kubectl describe node node1
Taints section lo chudachu.
Simple ga:
kubectl taint nodes = **Node ki taint add/remove cheyyadaniki command.**

---

9. Node nunchi Taint ela remove chestam?
Ans:
Taint remove cheyyadaniki taint key/value/effect pattern tarvata - use cheyyachu.
Example:
kubectl taint nodes node1 gpu=true:NoSchedule-
Meaning:
gpu=true:NoSchedule
↓
Remove
Verify:
kubectl describe node node1
Taints section check cheyyali.
Important:
Taint remove
↓
New Pods can become eligible
But Pod immediately Node meeda schedule avuthundi ani guarantee kaadu. Other scheduling constraints still apply.
Simple ga:
Taint remove command = **Node scheduling restriction ni remove cheyyadam.**

---

10. Pod lo Toleration ela define chestam?
Ans:
Pod YAML lo spec.tolerations use chestam.
Example:

```
spec:
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
```

Node taint:
gpu=true:NoSchedule
Pod:
Matching toleration
So Pod aa taint ni tolerate chestundi.
Another form:

```
tolerations:
  - key: "gpu"
    operator: "Exists"
    effect: "NoSchedule"
```

Exists value specific ga check cheyyakunda key/effect matching based on tolerance allow chestundi.
Simple ga:
tolerations = **Pod ki tolerate cheyyalsina Node taints ni define cheyyadam.**

---

11. Toleration undi ante Pod automatically aa Node ki vellipothunda?
Ans:
No. Idi very important interview point.
Toleration only means:
"Ee taint valla nannu block cheyyakandi."
It does **not** mean:
"Nannu compulsory ga ee Node meeda schedule cheyyandi."
Example:
Node 1:
gpu=true:NoSchedule

Pod:
Matching toleration
Pod Node 1 ki schedule avvachu.
But cluster lo:
Node 2
Node 3
kuda unte scheduler other constraints and scoring batti different Node select cheyyachu.
If specifically GPU Node meedane run cheyyali ante:

Toleration
+
NodeSelector / NodeAffinity
use cheyyadam better.
Flow:
Toleration
→ Allows

Node Affinity
→ Attracts/Requires
Simple ga:
Toleration = **Permission to be scheduled there, not a placement instruction.**

---

12. Taints & NodeSelector/NodeAffinity kalipi enduku use chestam?
Ans:
Special Nodes ni **reserve cheyyadaniki** Taint use chestam, and specific workload ni **aa Nodes ki attract/require cheyyadaniki** NodeSelector/NodeAffinity use chestam.
Example GPU Node:
Node label:
gpu=true

Node taint:
gpu=true:NoSchedule
GPU Pod:
Toleration
+
NodeAffinity: gpu=true
Flow:
Normal Pod
↓
No toleration
↓
Blocked by taint

GPU Pod
↓
Toleration
↓
NodeAffinity gpu=true
↓
GPU Node
Why both?
Taint
→ Keeps unwanted workloads out

Affinity
→ Places desired workload there
Simple ga:
Taint = **Repel unwanted Pods**
Affinity = **Attract required Pods**

---

13. Taint and Toleration real-time lo eppudu use chestam?
Ans:
Common use cases:
1. GPU Nodes
2. Dedicated database Nodes
3. High-memory Nodes
4. Special hardware
5. Production-only Nodes
6. Maintenance scenarios
7. Compliance-sensitive workloads
8. System workloads
Example:
GPU Node
↓
gpu=true:NoSchedule
Only GPU workloads:
Toleration
+
GPU Node affinity
Database dedicated node:

Node:
database=true:NoSchedule
DB Pod:
Toleration
+
NodeAffinity
Normal application:
No toleration
→ Doesn't get scheduled there
Simple ga:
Special-purpose Nodes ni **dedicate/protect cheyyadaniki** Taints & Tolerations very useful.

---

14. Real-time scenario: GPU Nodes unnayi. Normal Pods GPU Nodes meeda schedule avvakudadhu, GPU Pods matrame akkada run avvali. Ela design chestav?
Ans:
First GPU Nodes ki label:
gpu=true
Taint:
gpu=true:NoSchedule
Now normal Pods:
No toleration
→ Blocked
GPU Pod:
Toleration:
gpu=true:NoSchedule
But toleration alone enough kaadu.
GPU Node ki specifically schedule cheyyadaniki NodeAffinity:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: gpu
              operator: In
              values:
                - "true"
```

So:

```
GPU Node
 ├── Label: gpu=true
 └── Taint: gpu=true:NoSchedule

GPU Pod
 ├── Toleration
 └── NodeAffinity gpu=true
```

Result:
Normal Pod → ❌
GPU Pod → ✅
Simple ga:
Dedicated GPU Node pattern = **Taint + Toleration + NodeAffinity.**

---

15. Real-time scenario: Node maintenance cheyyali. Existing Pods ni automatically evict cheyyadaniki em use chestav?
Ans:
NoExecute taint use cheyyachu.
Example:
kubectl taint nodes node1 maintenance=true:NoExecute
Effect:
Existing non-tolerating Pods
↓
Evicted

New non-tolerating Pods
↓
Cannot schedule
But maintenance purpose kosam production environments lo kubectl drain kuda important tool:
kubectl drain node1
drain Pod eviction and workload rescheduling ni safely coordinate cheyyadaniki use chestundi, subject to PodDisruptionBudgets and other constraints.
So interview lo:
Taint NoExecute
→ Taint-based eviction behavior

kubectl drain
→ Planned node maintenance workflow
Simple ga:
Maintenance kosam kubectl drain **is commonly used**, while NoExecute provides taint-based eviction behavior.

---

16. NoExecute **toleration lo** tolerationSeconds **ante enti?**
Ans:
NoExecute taint apply ayinappudu Pod immediately evict kakunda **entha time tolerate cheyyalo** tolerationSeconds specify cheyyachu.
Example:

```
tolerations:
 - key: "maintenance"
   operator: "Equal"
   value: "true"
   effect: "NoExecute"
   tolerationSeconds: 300
```

Meaning conceptually:
NoExecute taint added
↓
Pod tolerates it
↓
300 seconds
↓
After tolerance expires
↓
Pod can be evicted
Useful when temporary node condition ki Pod immediate ga react avvakudadhu anukunte.
Without tolerationSeconds, a matching NoExecute toleration can allow the Pod to remain despite that taint.
Simple ga:
tolerationSeconds = **NoExecute taint ni Pod entha time tolerate cheyyalo.**

---

17. Real-time scenario: Node NotReady **ayyaka Pods ki** NoExecute **taint kanipistundi. Why?**
Ans:
Kubernetes node health conditions ni handle cheyyadaniki **taints and tolerations mechanism** use chestundi.
For example Node problem situation lo:
Node condition
↓
Corresponding taint
↓
NoExecute behavior
↓
Non-tolerating Pods may be evicted
Conceptually:
Node unhealthy
↓
Taint
↓
Pod toleration check
↓
No toleration
↓
Eviction/rescheduling behavior
This helps Kubernetes react to node failures and health conditions.
However, actual node-condition tainting and eviction timing depend on Kubernetes node lifecycle/controller behavior and configured tolerations.
Simple ga:
Node unhealthy ayithe Kubernetes **taints + NoExecute + tolerations** use chesi affected Pods ni handle cheyyagaladu.

---

18. Real-time scenario: Pod Pending **lo undi. Event lo** had taint ... that the pod didn't tolerate **ani vastundi. Ela troubleshoot chestav?**
Ans:
First Pod events check chestha:
kubectl describe pod <pod-name>
Events lo:
had taint that the pod didn't tolerate
Then Nodes taints check:
kubectl get nodes
kubectl describe node <node-name>
Look for:
Taints:
Suppose:

dedicated=database:NoSchedule
Pod ki matching toleration unda check cheyyali:
spec:
 tolerations:
If Pod actually database workload ayithe:
Add matching toleration
But if it should **not** run on database Node:
Don't add toleration blindly.
Instead check whether other Nodes are available.
Also check:
NodeAffinity
NodeSelector
Resources
Node readiness
Other taints
Simple ga:
didn't tolerate event ante **Node taints and Pod tolerations mismatch** first check cheyyali.

---

19. Real-time scenario: Toleration add chesina taruvatha kuda Pod desired Node meeda schedule avvatledu. Why?
Ans:
Toleration add cheyyadam valla **taint barrier remove avuthundi**, but scheduler ki desired Node choose cheyyamani instruction ivvadu.
Example:
Node:
gpu=true
gpu=true:NoSchedule
Pod:
Matching toleration
Still Pod other Node ki vellachu.
Why?
Toleration
→ Taint ni tolerate chestundi

But:
NodeAffinity/Selector
→ Desired Node selection
Also check:
CPU/Memory available?
Node Ready?
Other taints?
NodeAffinity conflicts?
Pod affinity rules?
Topology constraints?
Correct design:
Toleration
+
NodeAffinity/NodeSelector
Simple ga:
Toleration = **allowed**
NodeAffinity/Selector = **desired/required placement**

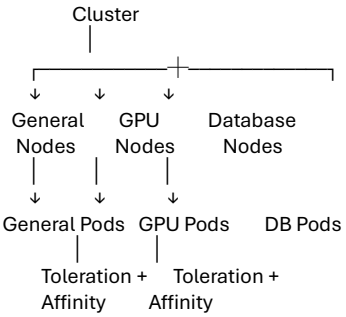
---

20. Real-time interview scenario: Production lo 3 types of Nodes unnayi — general, GPU, database. Workloads correct Nodes meeda matrame run avvali. Complete design explain cheyyi.
Ans:
Production cluster:
General Nodes
GPU Nodes
Database Nodes
General Nodes
General workloads kosam normal Nodes.
General Pods
 ↓
General Nodes
GPU Nodes
GPU Nodes ki:
Label:
gpu=true

Taint:
gpu=true:NoSchedule
GPU workload:

Toleration
+
NodeAffinity gpu=true
Normal workloads:
No toleration
→ Cannot enter GPU Node
Database Nodes
Database Nodes ki:
Label:
workload=database

Taint:
workload=database:NoSchedule
Database workload:
Toleration
+
NodeAffinity workload=database
Normal application:
No database toleration
→ Cannot enter DB Nodes
Overall architecture:



Why both Taint and Affinity?
Taint
→ Wrong workloads ni keep out

Toleration
→ Correct workload taint ni tolerate chestundi

NodeAffinity
→ Correct workload desired Node ni select chestundi
Production best practices:
1. Dedicated ServiceAccounts/RBAC separately manage cheyyali
2. Node labels consistent ga maintain cheyyali
3. Taints ni blindly add cheyyakudadhu
4. Toleration alone meeda placement depend avvakudadhu
5. NodeAffinity with required/preferred behavior carefully choose cheyyali
6. Resource requests correct ga define cheyyali
7. Node failures and rescheduling test cheyyali
Simple ga:
Dedicated Node architecture lo **Taint unwanted Pods ni repel chestundi, Toleration correct Pod ni allow chestundi, NodeAffinity correct Node ni select chestundi.**

---

19. Node Affinity / Pod Affinity — 20 Questions

1. Kubernetes lo Node Affinity ante enti?
Ans:
Node Affinity anedi **Pod ni particular labels unna Nodes meeda schedule cheyyadaniki** use chese scheduling rule.
Suppose nee cluster lo:
Node 1 → disk=ssd
Node 2 → disk=hdd
Node 3 → disk=ssd
Nee application ki SSD Node kavali anuko.
Appudu Pod lo Node Affinity define chestam:
Pod
↓
Node Affinity
↓

disk=ssd
↓
Node 1 / Node 3
Scheduler aa label match ayye Nodes ni consider chestundi.
Simple ga:
Node Affinity = **Pod ni specific Node labels based on Nodes ki schedule cheyyadaniki use chese rule.**

---

2. Node Affinity and NodeSelector madhya difference enti?
Ans:
Rendu kuda **Node labels based on Pod placement** kosame use chestam.
nodeSelector simple key-value matching:

```
nodeSelector:
  disk: ssd
```

Meaning:
disk=ssd unna Node meedane Pod schedule avvali.
Node Affinity more flexible:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      ...
```

Node Affinity lo:
In
NotIn
Exists
DoesNotExist
Gt
Lt
lanti operators use cheyyachu.
Also:
required
preferred
lanti scheduling behavior define cheyyachu.
Simple ga:
nodeSelector
→ Simple

Node Affinity
→ Flexible + advanced scheduling rules

---

3. Node Affinity lo requiredDuringSchedulingIgnoredDuringExecution **ante enti?**
Ans:
Idi **hard requirement**.
Meaning:
Pod schedule cheyyetappudu matching Node compulsory.
Example:
disk=ssd
Pod:
requiredDuringSchedulingIgnoredDuringExecution
Matching Node lekapothe:
Pod
↓
Pending
Matching Node dorikithe:
Pod
↓
SSD Node
IgnoredDuringExecution meaning: Pod already schedule ayyaka Node label later change ayina, aa label change valla existing Pod automatically evict avvadu.
Simple ga:
requiredDuringSchedulingIgnoredDuringExecution = **Schedule time lo condition compulsory; later label change valla existing Pod automatically remove cheyyadu.**

---

4. preferredDuringSchedulingIgnoredDuringExecution **ante enti?**
Ans:
Idi **soft preference**.
Meaning:
Scheduler possible ayithe matching Node meeda Pod ni place cheyyadaniki prefer chestundi, but mandatory kaadu.
Example:
Preferred:
disk=ssd
Nodes:

Node 1 → SSD
Node 2 → HDD
Scheduler possible ayithe:
Pod → Node 1
But Node 1 suitable kaakapothe:
Pod → Node 2
avvachu.
Comparison:
required
→ Must match

preferred
→ Try to match
Simple ga:
Preferred = **Node preference, not strict requirement.**

---

5. IgnoredDuringExecution **meaning enti?**
Ans:
Idi Node Affinity rule lo important concept.
Suppose Pod schedule ayye time:
Node:
disk=ssd
Pod requirement:
disk=ssd
Pod successfully schedule ayindi.
Later Node label change:
disk=ssd
↓
disk=hdd
IgnoredDuringExecution unte existing Pod ni **just because label changed ani automatically evict cheyyadu**.
So:
Scheduling time
→ Rule checked

After execution
→ Label mismatch alone doesn't evict Pod
Simple ga:
IgnoredDuringExecution = **Existing Pod ni later Node-label change valla automatically evict cheyyakunda undadam.**

---

6. Node Affinity lo operators enti?
Ans:
Node Affinity lo matchExpressions use chesi labels ni different operators tho match cheyyachu.
Common operators:
In
NotIn
Exists
DoesNotExist
Gt
Lt
Example:

```
matchExpressions:
 - key: environment
   operator: In
   values:
    - production
```

Meaning:
environment=production
NotIn:
environment NotIn
development
Exists:
gpu Exists
Meaning label key gpu exist avvali.
Simple ga:
Operators = **Node labels ni flexible ga match cheyyadaniki conditions.**

---

7. Node Affinity lo In **and** NotIn **difference enti?**
Ans:
In ante label value **given values lo one match avvali**.
Example:
key: environment

operator: In
values:
 - production
 - staging
Meaning:
environment=production ✅
environment=staging ✅
environment=dev ❌
NotIn ante given values lo undakudadhu.
key: environment
operator: NotIn
values:
 - dev
Meaning:
environment=dev ❌
environment=prod ✅
Simple ga:
In
→ Ee values lo undali

NotIn
→ Ee values lo undakudadhu

---

8. Node Affinity lo Exists **ante enti?**
Ans:
Exists operator **specific label key Node meeda exist avvali** ani chepthundi.
Example:
matchExpressions:
 - key: gpu
   operator: Exists
Suppose:
Node 1 → gpu=true
Node 2 → gpu=false
Node 3 → no gpu label
Node 1 and Node 2 lo gpu key exist kabatti:
Node 1 ✅
Node 2 ✅
Node 3 ❌
Value exact ga important kaadu; key presence important.
Simple ga:
Exists = **Node meeda aa label key undali.**

---

9. Pod Affinity ante enti?
Ans:
Pod Affinity anedi **already running/placed unna particular Pods ki close ga another Pod ni schedule cheyyadaniki** use chestam.
"Close" usually same topology domain, such as:
same Node
same zone
same region
depending on topologyKey.
Example:
Backend Pod
 ↓
Database Pod
Database and backend same zone lo run avvali anuko.
Pod Affinity use cheyyachu.
Backend Pod
↓
Pod Affinity
↓
Database Pod label match
↓
Same zone
Simple ga:
Pod Affinity = **One Pod ni another matching Pod unna topology area daggara schedule cheyyadam.**

---

10. Pod Anti-Affinity ante enti?
Ans:
Pod Anti-Affinity anedi **same type/matching Pods ni same topology area lo concentrate kakunda spread cheyyadaniki** use chestam.

Suppose 3 replicas:
Pod 1
Pod 2
Pod 3
Same Node lo anni run ayithe Node failure:
Node failure
↓
All 3 Pods affected
Anti-Affinity use chesthe:
Node 1 → Pod 1
Node 2 → Pod 2
Node 3 → Pod 3
Node failure impact reduce avvachu.
Simple ga:
Pod Anti-Affinity = **Matching Pods ni separate Nodes/zones lo spread cheyyadaniki use chese rule.**

---

11. Node Affinity and Pod Affinity madhya difference enti?
Ans:
Node Affinity:
Pod
↓
Node labels
↓
Matching Node
Pod Affinity:
Pod
↓
Other Pod labels
↓
Matching topology location
Example Node Affinity:
disk=ssd
Example Pod Affinity:
app=database
Meaning database Pod unna same zone/Node/domain lo application Pod ni place cheyyi.
So:
Node Affinity
→ Node properties based

Pod Affinity
→ Other Pods based
Simple ga:
Node Affinity = **Node ni select cheyyadam**
Pod Affinity = **Other Pod location ni consider cheyyadam**

---

12. Pod Affinity lo topologyKey **ante enti?**
Ans:
topologyKey anedi **"matching Pod tho entha level lo close ga undali?"** ani define chestundi.
Example:
topology.kubernetes.io/zone
Meaning:
Same Zone
Suppose:
Zone A
├── Node 1
└── Node 2

Zone B
├── Node 3
└── Node 4
Pod Affinity zone based unte:
Database Pod → Zone A
Backend Pod → Zone A
same zone lo place cheyyadaniki scheduler try/require chestundi based on rule.
If:
topologyKey = kubernetes.io/hostname
then topology level is typically **individual Node hostname**.
Simple ga:
topologyKey = **Pods madhya "same place" ni ye level lo define cheyyalo.**

---

13. Pod Affinity real-time lo eppudu use chestam?

Ans:
Pod Affinity useful when two workloads **close ga unte performance/architecture benefit** untundi.
Examples:
1. Application + Cache
2. Application + Side workload
3. Frontend + Local processing component
4. Latency-sensitive workloads
5. Related services in same zone
Example:
Application Pod
↓
Needs low-latency access
↓
Cache Pod
Pod Affinity:
app=cache
and topology:
same Node / same Zone
Then scheduler matching Cache Pod location ni consider chestundi.
Simple ga:
Pod Affinity = **Related workloads ni close ga place cheyyadaniki.**

---

14. Pod Anti-Affinity real-time lo enduku use chestam?
Ans:
High availability kosam Pod replicas ni **different Nodes/zones lo spread cheyyadaniki** Pod Anti-Affinity useful.
Suppose:
Deployment replicas = 3
Without spreading:
Node 1
├── Pod 1
├── Pod 2
└── Pod 3
Node 1 fail ayithe:
All replicas affected
Anti-Affinity/spreading use chesthe:
Node 1 → Pod 1
Node 2 → Pod 2
Node 3 → Pod 3
Then one Node fail ayina remaining replicas other Nodes lo undochu.
Simple ga:
Pod Anti-Affinity = **Replica failure impact reduce cheyyadaniki workloads ni separate cheyyadam.**

---

15. Real-time scenario: 3 replicas unnayi. Avi different Nodes lo run avvali. Ela design chestav?
Ans:
Pod Anti-Affinity use cheyyachu.
Suppose Pods ki label:
app=web
Pod spec lo:

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: app
              operator: In
              values:
                - web
        topologyKey: kubernetes.io/hostname
```

Meaning:
Matching web Pods
↓
Same hostname topology
↓
Avoid/forbid co-location
Result ideally:
Node 1 → Web Pod 1
Node 2 → Web Pod 2
Node 3 → Web Pod 3
But cluster lo enough eligible Nodes undali.
If only 2 Nodes unte 3 replicas ki strict anti-affinity:
Replica 1 → Node 1

Replica 2 → Node 2
Replica 3 → Pending
Simple ga:
Different Nodes lo replicas compulsory ante **required Pod Anti-Affinity with** kubernetes.io/hostname use cheyyachu.

---

16. required **and** preferred **Pod Affinity/Anti-Affinity madhya difference enti?**
Ans:
Concept Node Affinity lagaane.
required:
Condition compulsory
Matching topology available lekapothe Pod:
Pending
preferred:
Condition preference
Possible ayithe rule follow chestundi, but impossible ayithe Pod other suitable location lo schedule avvachu.
Example:
Preferred Anti-Affinity
→ Try to spread replicas

Required Anti-Affinity
→ Must spread according to rule
Production lo hard rules carefully design cheyyali because insufficient Nodes/Capacity unte Pods Pending avvachu.
Simple ga:
required
→ Must satisfy

preferred
→ Try to satisfy

---

17. Real-time scenario: Taints + Node Affinity rendu use chestunnav. Toleration kuda enduku kavali?
Ans:
Suppose GPU Node:
Label:
gpu=true

Taint:
gpu=true:NoSchedule
GPU Pod ki:
NodeAffinity:
gpu=true
matrame unte:
Affinity
→ GPU Node ni select cheyyali ani chepthundi
But:
Taint
→ Pod ni block chestundi
So Pod ki **matching Toleration** kuda kavali.
Correct:
GPU Pod
├— Toleration
└— NodeAffinity
Meaning:
Toleration
→ Taint ni tolerate cheyyi

NodeAffinity
→ GPU Node ni select cheyyi
Simple ga:
Taint + Affinity use chesthe usually correct workload ki **Toleration + NodeAffinity** pair avuthayi.

---

18. Real-time scenario: Pod Pending **lo undi. Node Affinity valla schedule avvatledu. Ela troubleshoot chestav?**
Ans:
First:
kubectl describe pod <pod-name>
Events check chestha.
Then Pod affinity rules:
kubectl get pod <pod-name> -o yaml
Check:
nodeSelector
nodeAffinity
podAffinity

podAntiAffinity
Then Nodes labels:
kubectl get nodes --show-labels
Suppose Pod requires:
disk=ssd
But Nodes:
Node 1 → disk=hdd
Node 2 → disk=hdd
Node 3 → disk=hdd
Then:
No matching Node
↓
Pod Pending
Also check:
Node Ready?
Taints?
Resources?
Affinity conflicts?
Topology constraints?
Simple ga:
Affinity Pending troubleshooting = **Pod rules → Node labels → Taints → Resources → Events** check cheyyali.

---

19. Real-time scenario: Pod Affinity use chesi Backend and Cache same Zone lo run cheyyali. Ela explain chestav?
Ans:
Suppose Cache Pods:
app=cache
Backend Pod ki Pod Affinity:

```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: cache
        topologyKey: topology.kubernetes.io/zone
```

Meaning:
Backend Pod
↓
Find Pod with app=cache
↓
Same zone
Architecture:

```
Zone A
 ├── Cache Pod
 └── Backend Pod

Zone B
 └── Other workloads
```

Backend scheduling time lo matching Cache Pod unna Zone ni requirement ga use chestundi.
But production lo idi carefully design cheyyali because matching Cache Pod unavailable ayithe Backend Pod Pending avvachu.
Simple ga:
Pod Affinity = **Backend Pod ni matching Cache Pod unna same zone lo place cheyyadam.**

---

20. Real-time interview scenario: Production application ki 6 replicas unnayi. High availability kavali, replicas different Nodes/zones lo spread avvali. Node Affinity, Pod Anti-Affinity, Taints anni ela combine chestav?
Ans:
Suppose production architecture:

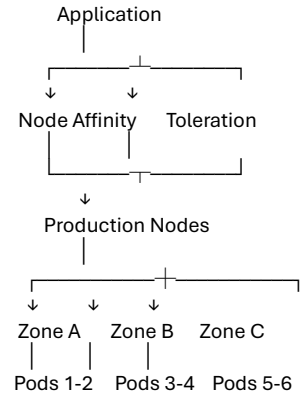
```
Zone A
 ├── Node 1
 └── Node 2

Zone B
 ├── Node 3
 └── Node 4

Zone C
 ├── Node 5
 └── Node 6
```

Production Nodes ki label:
environment=production
If dedicated production Nodes kavali ante taint:
environment=production:NoSchedule

Application Pod ki:
Toleration
+
NodeAffinity
NodeAffinity:
environment=production
So application only production Nodes ni consider chestundi.
Then Pod Anti-Affinity:
app=web
with topology:
kubernetes.io/hostname
or stronger zone-level spreading requirement depending on availability design.
Architecture:



Pod Anti-Affinity helps avoid putting matching replicas on the same topology domain.
For modern production designs, **topology spread constraints** are also worth considering for explicit, balanced distribution across zones/nodes.
Important distinction:
Taint
→ Unwanted workloads ni keep out

Toleration
→ Allowed workload ni permit

Node Affinity
→ Correct Node category select

Pod Anti-Affinity
→ Replicas ni separate/spread
Final flow:
Pod
↓
Toleration
↓
Can enter production Nodes
↓
Node Affinity
↓
Select production Nodes
↓
Pod Anti-Affinity / topology rules
↓
Spread replicas
↓
High availability
Simple ga:
Production HA scheduling lo **Taint + Toleration + Node Affinity + Pod Anti-Affinity (and often topology spread constraints)** combine chesi workload placement ni control chestam.

20. Resource Scheduling — 20 Questions

1. Kubernetes lo Resource Scheduling ante enti?
Ans:
Resource Scheduling ante **Pod ki kavalsina CPU, Memory lanti resources available unna suitable Node ni Kubernetes Scheduler select cheyyadam**.
Suppose cluster lo:
Node 1 → 8 CPU, 16 GB RAM
Node 2 → 4 CPU, 8 GB RAM
Node 3 → 2 CPU, 4 GB RAM
Pod ki:
CPU → 2 CPU
Memory → 4 GB
request undi anuko.
Scheduler Nodes ni evaluate chesi, Pod requirements satisfy ayye suitable Node ni select chestundi.
Flow:
Pod
↓
Scheduler
↓
Node resources check
↓
Other scheduling rules check
↓
Suitable Node select
↓
Pod scheduled
Simple ga:
Resource Scheduling = **Pod requirements ki suitable resources unna Node ni select cheyyadam.**

---

2. Kubernetes Scheduler ante enti?
Ans:
Kubernetes Scheduler anedi **unscheduled Pods ki suitable Node select chese control-plane component**.
Pod create ayindi:
Pod
↓
No Node assigned
↓
Scheduler
Scheduler check chestundi:
CPU/Memory
Node readiness
Taints
Node Affinity
Pod Affinity
Topology
Other scheduling constraints
Suitable Node select ayyaka:
Pod
↓
Node assigned
↓
Kubelet
↓
Container starts
Simple ga:
Scheduler = **Pod ekkada run avvalo decide chese component.**

---

3. Scheduler Pod ki Node ni select cheyyetappudu em factors consider chestundi?
Ans:
Scheduler only CPU/Memory matrame check cheyyadu. Multiple scheduling constraints consider chestundi.
Common factors:
1. CPU requests
2. Memory requests
3. Node readiness
4. Taints & Tolerations
5. NodeSelector

6. Node Affinity
7. Pod Affinity
8. Pod Anti-Affinity
9. Topology constraints
10. Volume-related constraints
11. Scheduling policies/plugins
Conceptually:
Pod
↓
Is Node feasible?
 ├— Resources available?
 ├— Taints tolerated?
 ├— Affinity satisfied?
 ├— Other constraints satisfied?
↓
Yes
↓
Candidate Node
↓
Scoring/preference
↓
Selected Node
Simple ga:
Scheduler = **Resources + constraints + preferences combine chesi Node select chestundi.**

---

4. Resource Requests ante enti?
Ans:
Resource Request ante **Pod/container ki minimum scheduling requirement laga declare chese CPU/Memory amount**.
Example:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

Meaning:
CPU request = 500 millicores
Memory request = 512 MiB
Scheduler Node ni select chestappudu ee requests ni consider chestundi.
Suppose Node lo allocatable memory lo 300Mi matrame available unte:
Pod request = 512Mi
Available = 300Mi
Aa Node Pod ki suitable kaadu.
Simple ga:
Request = **Pod ki scheduling kosam kavalsina resource reservation requirement.**

---

5. Resource Limits ante enti?
Ans:
Limit ante **Container resource usage ki maximum boundary** define cheyyadaniki use chestam.
Example:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
```

Meaning:
Request:
CPU = 500m
Memory = 512Mi

Limit:
CPU = 1 CPU
Memory = 1Gi
Important difference:
Request
→ Scheduling kosam important

Limit
→ Runtime resource ceiling/constraint
Memory limit exceed ayithe container **OOMKilled** avvachu.
CPU limit behavior CPU throttling ki lead avvachu depending on configuration/runtime.

Simple ga:
Limit = **Container runtime resource usage ki upper boundary.**

---

6. CPU 100m **ante enti?**
Ans:
Kubernetes lo CPU ni cores/fractions ga specify cheyyachu.
1 CPU = 1000m
So:
100m = 0.1 CPU
250m = 0.25 CPU
500m = 0.5 CPU
1000m = 1 CPU
2000m = 2 CPU
Example:
resources:
requests:
cpu: "500m"
Meaning Pod/container ki scheduling kosam **0.5 CPU** request.
Suppose Node allocatable CPU:
4 CPU
and existing workload requests:
3.7 CPU
remaining:
0.3 CPU
Pod request 500m ayithe aa Node fit avvakapovachu.
Simple ga:
1000m = 1 CPU; 500m = 0.5 CPU.

---

7. Resource requests scheduler ki enduku important?
Ans:
Scheduler Pod ki Node select cheyyetappudu **container resource requests ni consider chestundi**.
Suppose:
Node 1 available requested CPU = 2 CPU
Node 2 available requested CPU = 500m
New Pod:
CPU request = 1 CPU
Then:
Node 1 → Can fit
Node 2 → Cannot fit
So Node 1 feasible candidate.
Important point:
Scheduler generally **requests ni base cheskoni scheduling capacity evaluate chestundi**, actual current CPU usage ni simple ga use chesi placement decide cheyyadu.
Simple ga:
Requests = **Scheduler ki Pod capacity requirement teliyadaniki main input.**

---

8. Pod Pending **lo undi, reason** Insufficient cpu**. Meaning enti?**
Ans:
Meaning scheduler ki **Pod CPU request satisfy cheyyagalige suitable Node dorakaledu**.
Example:
Pod CPU request = 2 CPU
Nodes:
Node 1 → available requested capacity 1 CPU
Node 2 → available requested capacity 500m
Node 3 → available requested capacity 1 CPU
None can satisfy:
2 CPU
Result:
Pod
↓
Pending
Check:
kubectl describe pod <pod-name>
Events lo:
Insufficient cpu
Possible solutions:
1. Add/scale Nodes
2. Reduce unrealistic CPU requests
3. Check other workloads
4. Check scheduling constraints

Simple ga:
Insufficient cpu = **Pod request satisfy cheyyagalige CPU capacity unna eligible Node ledu.**

---

9. Pod Pending **lo undi, reason** Insufficient memory**. Ela troubleshoot chestav?**
Ans:
First:
kubectl describe pod <pod-name>
Events check chestha.
Then Nodes:
kubectl describe nodes
Check:
Capacity
Allocatable
Allocated resources
Pod memory request:

```
resources:
  requests:
    memory: "2Gi"
```

Suppose eligible Node lo remaining allocatable requested memory:
1Gi
Pod requires:
2Gi
So:
Cannot fit
↓
Pending
Also check whether:
Node Affinity
Taints
Pod Anti-Affinity
Topology
valla already available Nodes exclude ayyaya ani.
Simple ga:
Memory issue = **Pod request vs Node allocatable/available requested capacity + scheduling constraints** check cheyyali.

---

10. Node Capacity and Node Allocatable madhya difference enti?
Ans:
Capacity ante Node total resources.
Example:
Capacity:
CPU = 8
Memory = 32Gi
But Node lo Kubernetes system components and OS ki kuda resources kavali.
So Pod workloads kosam available portion:
Allocatable:
CPU = 7.5
Memory = 30Gi
Conceptually:

```
Node Capacity
 ├── Kubernetes/system overhead
 └── Pod workload capacity
        ↓
    Allocatable
```

Scheduler primarily **allocatable resources** ni consider chestundi.
Check:
kubectl describe node <node-name>
Simple ga:
Capacity = **Total Node resources**
Allocatable = **Pods scheduling kosam available ga expose chese resources after system reservations/overhead.**

---

11. requests **ekkuva pedithe em problem?**
Ans:
Requests realistic usage kanna unnecessarily high ga set chesthe **cluster capacity waste** avvachu.
Example:
Actual application need:
CPU ≈ 200m
But request:
CPU = 2
Suppose Node:
8 CPU
Scheduler 2 CPU request ni reserve/calculate chestundi even if application actual ga 200m matrame use chestunna.

Result:
Less Pods fit
→ Lower cluster utilization
→ More Nodes may be required
This is called **over-requesting**.
On the other hand, requests too low ga set chesthe scheduling overcommit/underestimation issues and contention risks increase avvachu.
Simple ga:
Requests = **Realistic ga set cheyyali; too high → waste, too low → contention risk.**

---

12. requests **and actual CPU/Memory usage same ga untaya?**
Ans:
No.
Request:
CPU request = 500m
Actual usage:
CPU usage = 200m
or:
CPU usage = 800m
avvachu, subject to limits.
Request mainly scheduling/capacity planning kosam.
Actual usage monitoring metrics nunchi telustundi.
Example:
Request → 500m
Actual → 200m
Scheduler 500m request ni capacity planning lo consider chestundi, not simply 200m actual usage.
Simple ga:
Request = **Declared requirement**
Usage = **Actually consumed resource**

---

13. Resource overcommitment ante enti?
Ans:
Overcommitment ante cluster lo **resource requests/limits and actual usage relationship valla available physical capacity kanna workloads demand boundaries ekkuva ga configure cheyyadam**.
Example:
Node:
CPU = 4
Multiple containers:
CPU limits total = 8
Physical CPU:
4 CPU
Limits total 8 CPU ayina actual ga all workloads simultaneously full CPU use cheyyakapovachu.
So CPU limits can be overcommitted.
Memory behavior more sensitive because memory cannot be safely throttled like CPU; excessive memory usage can cause OOM kills/evictions.
Simple ga:
Overcommitment = **Configured workload resource boundaries physical capacity kanna ekkuva undadam.**

---

14. CPU and Memory overcommitment same way behave chestaya?
Ans:
No. CPU and Memory behavior different.
CPU:
Can be throttled
Can be shared
Memory:
Cannot be throttled in the same way
Excessive usage → OOM / eviction pressure
Example:
CPU:
4 CPU Node
Limits total = 8 CPU
Possible because all containers may not use full CPU simultaneously.
Memory:
8Gi Node
Memory usage approaches/exceeds available memory
Then:
Memory pressure
↓
Eviction/OOM behavior
depending on conditions and QoS.

Simple ga:
CPU = **share/throttle cheyyagalige resource**
Memory = **exhaust ayithe OOM/eviction risk ekkuva.**

---

15. Kubernetes lo QoS Classes enti?
Ans:
Kubernetes Pods ki resource requests/limits configuration base cheskoni QoS class assign chestundi.
Main classes:
1. Guaranteed
2. Burstable
3. BestEffort
Guaranteed:
Containers ki CPU/Memory requests and limits properly equal and set according to Guaranteed criteria.
Burstable:
At least one container has requests/limits configured, but Pod does not meet Guaranteed criteria.
BestEffort:
Containers ki CPU/Memory requests and limits none.
Conceptually:
Guaranteed
→ Strongest resource guarantees

Burstable
→ Some requests/limits

BestEffort
→ No requests/limits
QoS class resource pressure situations lo behavior/prioritization ki relevant.
Simple ga:
QoS = **Resource configuration base cheskoni Pod priority/eviction behavior ni influence chese classification.**

---

16. Real-time scenario: Node full ga undi. New Pod schedule avvatledu. Ela troubleshoot chestav?
Ans:
First Pod:
kubectl describe pod <pod-name>
Events:
FailedScheduling
Then Nodes:
kubectl get nodes
kubectl describe node <node-name>
Check:
Allocatable
Allocated resources
Taints
Conditions
Pod requests:
CPU request
Memory request
Then check other constraints:
NodeAffinity
NodeSelector
PodAntiAffinity
Topology
Sometimes Node has physical free CPU visible in monitoring, but Pod still can't schedule because **requested allocatable capacity** doesn't satisfy the Pod request or another scheduling constraint blocks it.
Possible fixes:
1. Add Nodes
2. Right-size requests
3. Remove unnecessary constraints
4. Scale workloads appropriately
5. Check autoscaler configuration
Simple ga:
Pending resource issue = **Pod events + Node allocatable + requests + scheduling constraints** check cheyyali.

---

17. Real-time scenario: Pod ki CPU request 4 undi, but cluster lo Nodes anni 2 CPU or 3 CPU capacity matrame unnayi. Em jaruguthundi?
Ans:
Pod ki:
CPU request = 4 CPU
But Nodes:
Node 1 → 2 CPU
Node 2 → 3 CPU

Node 3 → 2 CPU
No Node can satisfy 4 CPU request.
So:
Pod
↓
Scheduler
↓
No feasible Node
↓
Pending
Even if total cluster CPU:
2 + 3 + 2 = 7 CPU
still Pod cannot be split across multiple Nodes.
A single Pod's containers must be placed together on one Node.
Simple ga:
Total cluster resources enough unna kuda **single Node lo Pod request fit avvali**.

---

18. Real-time scenario: Pod request 500m CPU, limit 2 CPU. Idi valid aa? Explain cheyyi.
Ans:
Yes, valid configuration.
CPU request = 500m
CPU limit = 2
Meaning:
Scheduling requirement ≈ 500m

Runtime upper limit = 2 CPU
Suppose application normal usage:
300m
Scheduler 500m request ni consider chestundi.
Traffic spike:
Usage → 1.5 CPU
Limit 2 CPU varaku allow cheyyachu subject to node capacity/runtime behavior.
If usage tries to exceed limit:
CPU
↓
Can be throttled
Memory example different:
memory request = 512Mi
memory limit = 1Gi
Limit exceed ayithe OOMKill risk.
Simple ga:
Request = **Scheduling baseline**
Limit = **Runtime maximum boundary**

---

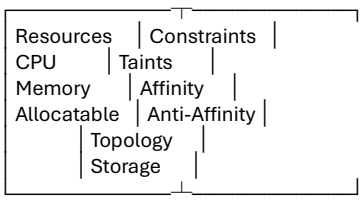
19. Real-time interview scenario: Scheduler Node select chestundi ani explain cheyyali. Complete flow enti?
Ans:
Suppose new Pod create ayindi.
Pod
↓
spec
Pod lo:
CPU request = 1 CPU
Memory request = 1Gi
NodeAffinity = environment=prod
Toleration = dedicated=prod:NoSchedule
Scheduler candidate Nodes ni evaluate chestundi.
Step 1 — Unschedulable Pod identify
Pod has no assigned Node
Step 2 — Nodes evaluate
Node 1
Node 2
Node 3
...
Step 3 — Resource feasibility
CPU available?
Memory available?
Step 4 — Taints
Pod tolerates Node taints?
Step 5 — Affinity/constraints
NodeAffinity?
NodeSelector?

PodAffinity?
PodAntiAffinity?
Topology?
Step 6 — Feasible Nodes
Node 2
Node 5
Step 7 — Scoring/preferences
Scheduler plugins/preferences based on configuration decide the best candidate.
Step 8 — Binding
Pod
↓
Selected Node
Then:
Kubelet
↓
Creates containers
Simple ga:
Pod
↓
Filter feasible Nodes
↓
Score/select best Node
↓
Bind Pod
↓
Kubelet runs it

---

20. Real-time interview scenario: Production cluster lo Pods Pending **avuthunnayi. CPU/Memory available unna kuda schedule avvatledu. Complete troubleshooting approach cheppu.**
Ans:
Idi real production lo very common scenario. CPU/Memory available undi ani chusi direct ga "scheduler problem" ani assume cheyyakudadhu.
First Pod:
kubectl describe pod <pod-name>
Events lo exact reason chudali.
Then:
1. Node status
kubectl get nodes
Check:
Ready?
SchedulingDisabled?
2. Node resources
kubectl describe node <node-name>
Check:
Capacity
Allocatable
Allocated resources
3. Pod requests
CPU request
Memory request
Node lo raw CPU free unna kuda **allocatable/requested capacity** fit avvakapovachu.
4. Taints
Node taints
Pod toleration unda?
spec.tolerations
5. Node Affinity / NodeSelector
Required label Node meeda unda?
Check:
kubectl get nodes --show-labels
6. Pod Anti-Affinity
Suppose:
3 replicas
and strict anti-affinity same hostname required.
Only 2 Nodes available ayithe:
Replica 1 → Node 1
Replica 2 → Node 2
Replica 3 → Pending
7. Topology constraints
Zones/Nodes distribution rules Pod ni block chestunnaya check cheyyali.
8. PVC / Volume constraints
Storage-related scheduling constraints kuda Pod placement ni affect cheyyachu.

9. Scheduler events/logs
If needed, cluster control-plane observability and scheduler logs/configuration investigate cheyyali.
10. Autoscaling
If no Node can fit the Pod request:
Pod Pending
↓
Cluster Autoscaler
↓
Can it add suitable Node?
Autoscaler configured and supported node group available ayithe scale-out happen avvachu.
Overall troubleshooting:
Pod Pending
↓
kubectl describe pod
↓
Find FailedScheduling reason
↓



↓
Fix root cause
↓
Scheduler retries
↓
Pod scheduled
Simple ga:
Production lo **"Node lo CPU free undi"** anedi alone sufficient kaadu. **Requests + Allocatable + Taints + Affinity + Anti-Affinity + Topology + Storage + Autoscaling** anni consider chesi troubleshoot cheyyali.

---

21. Helm — 20 Questions

1. Kubernetes lo Helm ante enti?
Ans:
Helm anedi Kubernetes kosam use chese **package manager**. Kubernetes applications ni install, configure, upgrade, rollback cheyyadaniki Helm use chestam.
Suppose nee application ki:
Deployment
Service
ConfigMap
Secret
Ingress
anni YAML files unnayi anuko.
Manual ga:
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f configmap.yaml
kubectl apply -f ingress.yaml
ila maintain cheyyadam difficult avvachu.
Helm use chesthe ee Kubernetes resources ni oka package laga manage cheyyachu.
Helm Chart
↓
Deployment
Service
ConfigMap
Secret
Ingress
Simple ga:
Helm = **Kubernetes applications ni package, install, upgrade, rollback, manage cheyyadaniki tool.**

---

2. Helm Chart ante enti?
Ans:
Helm Chart anedi **Kubernetes application ki required files and templates ni organize chese package**.
Example:

myapp/
├── Chart.yaml
├── values.yaml
└── templates/
  ├── deployment.yaml
  ├── service.yaml
  ├── configmap.yaml
  └── ingress.yaml
Chart lo:
Chart.yaml
→ Chart metadata

values.yaml
→ Default configuration values

templates/
→ Kubernetes resource templates
Example:
myapp Chart
  ↓
Deployment
Service
ConfigMap
Ingress
Simple ga:
Helm Chart = **Kubernetes application deploy cheyyadaniki required package of templates + configuration.**

---

3. Helm Chart lo Chart.yaml **file purpose enti?**
Ans:
Chart.yaml anedi Helm Chart ki **metadata file**.
Example:
apiVersion: v2
name: myapp
description: My application Helm chart
type: application
version: 1.0.0
appVersion: "2.5.0"
Important fields:
name
→ Chart name

version
→ Chart version

appVersion
→ Application version
Example:
Chart version = 1.0.0
Application version = 2.5.0
Rendu same undalsina requirement ledu.
Simple ga:
Chart.yaml = **Helm Chart metadata and version information.**

---

4. Helm lo values.yaml **ante enti?**
Ans:
values.yaml anedi **application configuration values ni store cheyyadaniki use chese file**.
Example:
replicaCount: 3

image:
 repository: nginx
 tag: "1.27"

service:
 type: ClusterIP
 port: 80
Template lo:
replicas: {{ .Values.replicaCount }}
Helm install time lo:
values.yaml
  ↓

Template
↓
Kubernetes YAML
↓
Deployment
Environment-specific configuration easy ga change cheyyachu.
Example:
dev → replicas=1
staging → replicas=2
prod → replicas=5
Simple ga:
values.yaml = **Templates ki configuration values provide chese default configuration file.**

---

5. Helm templates **directory enduku use chestam?**
Ans:
templates/ directory lo **Kubernetes resource templates** untayi.
Example:
templates/
├— deployment.yaml
├— service.yaml
└— ingress.yaml
Template:
replicas: {{ .Values.replicaCount }}
Values:
replicaCount: 3
Helm render chesinappudu:
{{ .Values.replicaCount }}
↓
3
So final Kubernetes manifest:
replicas: 3
Simple ga:
templates/ = **Dynamic Kubernetes YAML templates store chese place.**

---

6. Helm Chart install cheyyadaniki command enti?
Ans:
Common command:
helm install myapp ./mychart
Meaning:
myapp
→ Release name

./mychart
→ Chart location
Flow:
Helm Chart
↓
helm install
↓
Render templates
↓
Kubernetes resources
↓
Cluster
Example:
helm install frontend ./frontend-chart
Helm release name frontend.
Simple ga:
helm install = **Chart ni Kubernetes cluster lo deploy cheyyadaniki.**

---

7. Helm lo Release ante enti?
Ans:
Helm Chart install chesinappudu create ayye **deployed instance ni Release** antam.
Suppose same Chart:
myapp-chart
multiple times install chesthe:
Release 1 → dev-app
Release 2 → staging-app
Release 3 → prod-app
Same Chart, different releases.
Command:

helm install dev-app ./myapp
Here:
Chart → ./myapp
Release → dev-app
Helm release history maintain chestundi, so upgrades and rollbacks easier.
Simple ga:
Chart = **Package/template**
Release = **That Chart yokka deployed instance**

---

8. helm install **and** helm upgrade **madhya difference enti?**
Ans:
helm install:
New Release create
Example:
helm install myapp ./mychart
Already existing release ki changes apply cheyyali ante:
helm upgrade myapp ./mychart
Example:
Initial:
replicas = 2

Upgrade:
replicas = 5
Command:
helm upgrade myapp ./mychart --set replicaCount=5
Flow:
Install
→ New deployment

Upgrade
→ Existing release update
Simple ga:
install = **First deployment**
upgrade = **Existing Helm release update**

---

9. Helm lo Rollback ante enti?
Ans:
Upgrade tarvata application issue vachinappudu **previous working release revision ki return avvadam** rollback.
Suppose:
Revision 1 → Working
Revision 2 → New version
Revision 3 → Broken deployment
Rollback:
helm rollback myapp 1
Then:
Revision 3
↓
Rollback
↓
Revision 1
Helm release history maintain chestundi kabatti previous revisions ki rollback cheyyachu.
Check history:
helm history myapp
Simple ga:
Rollback = **Problematic Helm deployment ni previous known-good revision ki revert cheyyadam.**

---

10. helm list **command enduku use chestam?**
Ans:
helm list command cluster lo unna **Helm releases ni list cheyyadaniki** use chestam.
Example:
helm list
Output conceptually:
NAME NAMESPACE STATUS
frontend default deployed
backend default deployed
payment prod deployed
Specific namespace:
helm list -n prod
All namespaces:
helm list -A

Simple ga:
helm list = **Cluster lo Helm-managed releases ni chudataniki.**

---

11. helm status **command enduku use chestam?**
Ans:
helm status specific Helm release current status and useful deployment information chudataniki use chestam.
Example:
helm status myapp
It can show:
Release name
Namespace
Status
Revision
Resources/notes depending on chart
Suppose deployment issue vachindi anuko:
helm status myapp -n production
Then release state first-level ga inspect cheyyachu.
Simple ga:
helm status = **Specific Helm release current state/details check cheyyadaniki.**

---

12. Helm lo --set **enduku use chestam?**
Ans:
--set use chesi **command line nunchi values override** cheyyachu.
values.yaml:
replicaCount: 2
Install time:
helm install myapp ./mychart --set replicaCount=5
Then:
Default value → 2
Override value → 5
Multiple values:
helm install myapp ./mychart \
 --set replicaCount=5 \
 --set image.tag=2.0
Useful for quick changes.
But many production configurations ki separate values files often cleaner.
Simple ga:
--set = **Command line nunchi Helm values override cheyyadam.**

---

13. Multiple environments kosam Helm ni ela use chestav?
Ans:
Usually common Chart maintain chesi environment-specific values files use chestam.
Structure:
myapp/
├── Chart.yaml
├── values.yaml
└── templates/
Environment files:
values-dev.yaml
values-staging.yaml
values-prod.yaml
Example:
values-dev.yaml
replicaCount: 1
values-prod.yaml
replicaCount: 5
Deploy:
helm upgrade --install myapp ./myapp \
 -f values-prod.yaml
Flow:
Common Chart
 +
Environment values
 ↓
Rendered manifests
 ↓
Environment cluster/namespace
Simple ga:
One Chart + **different values files** = multi-environment deployment easy.

---

14. helm template **command ante enti?**

Ans:
helm template use chesi **Helm templates ni render chesi final Kubernetes YAML output ni local ga chudachu**, cluster ki apply cheyyakunda.
Example:
helm template myapp ./mychart
Output:
apiVersion: apps/v1
kind: Deployment
...
Useful for:
Template debugging
Manifest review
CI validation
Understanding rendered YAML
Suppose:
replicas: {{ .Values.replicaCount }}
and:
replicaCount: 3
helm template final output lo:
replicas: 3
Simple ga:
helm template = **Chart ni render chesi final Kubernetes manifests ni preview cheyyadam.**

---

15. Helm upgrade mundu rendered YAML validate cheyyali ante em chestav?
Ans:
First template render cheyyachu:
helm template myapp ./mychart
Then Helm-side validation kosam:
helm lint ./mychart
Also dry-run:
helm upgrade --install myapp ./mychart --dry-run
Useful commands:
helm lint
→ Chart structure/template issues detect cheyyadaniki

helm template
→ Rendered YAML inspect cheyyadaniki

--dry-run
→ Actual change apply cheyyakunda operation simulate/check cheyyadaniki
CI/CD pipeline lo kuda ee validations use cheyyachu.
Simple ga:
Deploy cheyyemundu **lint → render → dry-run/validation → deploy** workflow useful.

---

16. Helm lo Dependency ante enti?
Ans:
Oka application Chart ki vere Helm Charts kavali ante avi **Chart dependencies**.
Example:
myapp
├— backend
├— Redis
└— PostgreSQL
Suppose MyApp Chart ki Redis dependency undi.
Then Chart dependency declare cheyyachu.
Conceptually:
Parent Chart
↓
Dependency Chart
↓
Redis
Dependencies ni package/manage cheyyadaniki Helm supports chart dependency mechanisms.
Modern Helm Charts lo dependencies Chart.yaml lo declare cheyyachu.
Simple ga:
Dependency = **Oka Helm Chart ki required ayye another Chart.**

---

17. Helm v2 and Helm v3 madhya major difference enti?
Ans:
Interview lo very common question.
Helm v2 architecture lo:
Helm Client
↓

Tiller
↓
Kubernetes
Tiller server-side component ga work chesedi.
Helm v3 lo Tiller remove chesaru:
Helm Client
↓
Kubernetes API
So Helm v3 architecture simpler and Kubernetes RBAC model tho better integrate avuthundi.
Important interview point:
Helm v2 → Tiller
Helm v3 → No Tiller
Current Kubernetes environments lo generally **Helm v3+** use chestaru.
Simple ga:
Major change = **Helm 3 lo Tiller remove ayindi.**

---

18. Real-time scenario: Helm deployment successful ani chupistundi, but application work avvatledu. Ela troubleshoot chestav?
Ans:
helm status lo deployed ani undadam application healthy ani guarantee kaadu.
First:
helm status myapp -n production
Then Kubernetes resources:
kubectl get pods -n production
kubectl get svc -n production
kubectl get deploy -n production
Pods check:
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production
Then:
ConfigMap
Secret
Service selectors
Ingress
Readiness probes
Environment variables
Image
Resources
check cheyyali.
Helm rendered configuration inspect cheyyadaniki:
helm get manifest myapp -n production
Values:
helm get values myapp -n production
Flow:
Helm says deployed
↓
Check Pods
↓
Check application logs
↓
Check Service
↓
Check Config/Secrets
↓
Check Ingress/probes
↓
Find actual application issue
Simple ga:
Helm release deployed = **Helm operation successful**; application itself healthy ani automatic guarantee kaadu.

---

19. Real-time scenario: Production lo helm upgrade **chesaka application down ayindi. Immediate ga em chestav?**
Ans:
First panic avvakunda current release status/history check chestha:
helm status myapp -n production
helm history myapp -n production
Suppose:
Revision 10 → Working
Revision 11 → Bad upgrade
Application impact high and previous revision known-good ayithe:
helm rollback myapp 10 -n production
Then verify:

kubectl get pods -n production
kubectl rollout status deployment/<deployment-name> -n production
Application logs and health kuda check cheyyali.
After stabilization:
1. Find root cause
2. Compare old/new values
3. Check rendered manifests
4. Check image/config changes
5. Fix Chart
6. Test in lower environment
7. Retry production deployment

Simple ga:

Bad Upgrade
↓
Detect
↓
helm history
↓
Identify previous good revision
↓
helm rollback
↓
Verify application
↓
Root-cause analysis

---

20. Real-time interview scenario: Same Helm Chart ni Dev, QA, Prod environments lo deploy cheyyali. Best approach enti?
Ans:

Best practice generally **same reusable Chart** maintain chesi, environment-specific configuration ni separate values files lo manage cheyyadam.

Example:

```
myapp-chart/
├── Chart.yaml
├── values.yaml
└── templates/
```

Environment configuration:

```
values-dev.yaml
values-qa.yaml
values-prod.yaml
```

Dev:

```
helm upgrade --install myapp ./myapp-chart \
 -f values-dev.yaml \
 -n dev
```

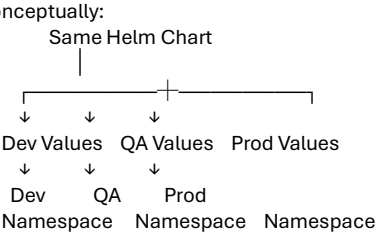
QA:

```
helm upgrade --install myapp ./myapp-chart \
 -f values-qa.yaml \
 -n qa
```

Prod:

```
helm upgrade --install myapp ./myapp-chart \
 -f values-prod.yaml \
 -n prod
```

Conceptually:



For production, CI/CD pipeline lo:

Git
↓
Helm lint
↓
helm template / validation
↓
Deploy to Dev
↓
Test
↓

QA
↓
Approval
↓
Production
Also secrets ni plain-text values.yaml lo blindly store cheyyakunda appropriate secret-management approach use cheyyali.
Interview lo final ga:
"I would keep one reusable Helm Chart and externalize environment-specific configuration into separate values files. I would validate the rendered manifests in CI/CD, deploy using helm upgrade --install, and use Helm history/rollback for release recovery."
Simple ga:
One Chart + environment-specific values + CI/CD validation + controlled upgrades/rollback = clean Helm deployment strategy.

---

22. Kubernetes Troubleshooting — 20 Questions

1. Kubernetes lo Troubleshooting ante enti?
Ans:
Kubernetes Troubleshooting ante **application or Kubernetes resource expected ga work avvakapothe root cause identify chesi fix cheyyadam**.
Example:
User
↓
Ingress
↓
Service
↓
Pod
↓
Container
Application open avvatledu anuko. Direct ga Pod ni blame cheyyakunda complete flow check cheyyali.
Common issues:
Pod Pending
Pod CrashLoopBackOff
Pod ImagePullBackOff
Service not reachable
Ingress not working
DNS failure
ConfigMap/Secret issue
Resource shortage
Node NotReady
Basic troubleshooting flow:
Problem
↓
Identify affected resource
↓
Check status
↓
Check Events
↓
Check Logs
↓
Find root cause
↓
Fix
↓
Verify
Simple ga:
Kubernetes Troubleshooting = **Problem → Evidence → Root Cause → Fix → Verify.**

---

2. Kubernetes lo first troubleshooting commands enti?
Ans:
Issue vachinappudu common ga first ee commands use chestam:
kubectl get pods
kubectl get svc
kubectl get deploy
kubectl get nodes
More information kosam:
kubectl describe pod <pod-name>
Logs:
kubectl logs <pod-name>

Recent events:
kubectl get events --sort-by=.lastTimestamp
Namespace-specific:
kubectl get pods -n production
Simple ga:
get
→ Status

describe
→ Detailed information + events

logs
→ Application/container behavior

events
→ Cluster scheduling/resource errors

---

3. Pod Pending **lo undi. Ela troubleshoot chestav?**
Ans:
Pending ante Pod create ayindi, kani **run avvadaniki Node placement or another prerequisite complete kaaledu**.
First:
kubectl get pod <pod-name>
Then:
kubectl describe pod <pod-name>
Bottom lo Events check cheyyali.
Common reasons:
Insufficient cpu
Insufficient memory
Node affinity mismatch
NodeSelector mismatch
Taint not tolerated
Pod Anti-Affinity
Volume/PVC issue
Topology constraints
Example:
Pod request = 4 CPU

Eligible Nodes:
Node 1 → 2 CPU
Node 2 → 3 CPU
No Node can fit the request.
Result:
Pod → Pending
Simple ga:
Pending troubleshooting lo kubectl describe pod **→ Events → resource/affinity/taint/storage reason** check cheyyali.

---

4. Pod CrashLoopBackOff **lo undi. Meaning enti?**
Ans:
CrashLoopBackOff ante **Container start avuthundi, crash avuthundi, Kubernetes malli restart chestundi; repeated failures valla restart madhya delay increase chestundi.**
Flow:
Container starts
↓
Application crashes
↓
Container exits
↓
Kubernetes restarts
↓
Again crashes
↓
CrashLoopBackOff
First logs:
kubectl logs <pod-name>
Previous crashed container logs:
kubectl logs <pod-name> --previous
Then:
kubectl describe pod <pod-name>
Common reasons:
Wrong application config
Missing Secret/ConfigMap

Wrong command/arguments
Application startup failure
Dependency unavailable
Permission issue
Probe configuration issue
Memory limit/OOM
Simple ga:
CrashLoopBackOff = **Container repeatedly crash/restart avuthondi.**

---

5. Pod ImagePullBackOff **lo undi. Reason enti?**
Ans:
Meaning Kubernetes container image ni pull cheyyadaniki try chesindi, but **image pull repeatedly fail avuthondi**.
Example:
image: myapp:v10
But registry lo v10 tag lekapothe:
Image pull
↓
Failed
↓
Retry
↓
Backoff
↓
ImagePullBackOff
Check:
kubectl describe pod <pod-name>
Events lo exact error chudali.
Common reasons:
Wrong image name
Wrong tag
Private registry authentication issue
Registry unavailable
ImagePullSecret issue
Network/DNS problem
Simple ga:
ImagePullBackOff = **Kubernetes container image ni registry nunchi successfully pull cheyyalekapothondi.**

---

6. ErrImagePull **and** ImagePullBackOff **madhya difference enti?**
Ans:
Rendu image pull problem related.
ErrImagePull:
Immediate image pull failure
ImagePullBackOff:
Repeated pull failures
+
Kubernetes retry delay increase chestundi
Flow:
Pull image
↓
Failure
↓
ErrImagePull
↓
Retry
↓
Repeated failure
↓
ImagePullBackOff
Example:
Wrong image tag
Fix:
Correct image/tag
Then Pod image successfully pull cheskuntundi.
Simple ga:
ErrImagePull = **Pull failed**
ImagePullBackOff = **Repeated pull failure + retry backoff**

---

7. Pod Running **lo undi, but application accessible kaadu. Ela troubleshoot chestav?**
Ans:
Running ante containers running state lo unnayi ani matrame. Application traffic accept chestundi ani guarantee kaadu.
Flow:

User
↓
Ingress?
↓
Service?
↓
Endpoints?
↓
Pod?
↓
Container port?
↓
Application
First:
kubectl get pods -o wide
Service:
kubectl get svc
Endpoints:
kubectl get endpoints
or EndpointSlices:
kubectl get endpointslice
Check labels/selectors:
Service selector
↓
Pod labels
Then Pod logs:
kubectl logs <pod-name>
Also readiness:
Is Pod Ready?
Simple ga:
Pod Running unna kuda **Service → Endpoint → Pod labels → Port → Readiness → Application** complete chain check cheyyali.

---

8. Service ki Endpoints levu. Common reason enti?
Ans:
Service selector ki match ayye **Ready Pods available lekapovadam** common reason.
Example:
Service:
selector:
app: backend
Pod:
labels:
app: frontend
Mismatch:
Service selector
app=backend

Pod label
app=frontend
So:
Service
↓
No matching Pod
↓
No endpoints
Another reason Pod matching label unna **Ready kakapovachu**, depending on how endpoints are being considered.
Check:
kubectl get svc <service-name>
kubectl get endpoints <service-name>
kubectl get pods --show-labels
Simple ga:
No Endpoints = **Service ki usable matching Pods dorakadam ledu; first selector + labels + readiness check cheyyali.**

---

9. Service selector and Pod labels mismatch ayithe em jaruguthundi?
Ans:
Service Pods ni selector dwara identify chestundi.
Service:
selector:
app: backend
Pod:
labels:
app: frontend

Mismatch kabatti:
Service
↓
Looking for app=backend
↓
Pod has app=frontend
↓
No match
↓
No Endpoint
So user request:
Client → Service
correct backend Pod ki reach avvadu.
Fix:
Service selector
=
Pod labels
Simple ga:
Service selector and Pod labels **match avvali**; otherwise Service traffic ki backend Pods identify cheyyaledu.

---

10. Kubernetes lo Events enduku important?
Ans:
Events Kubernetes lo **resource-related important actions and failures ki clues** istayi.
Example:
kubectl describe pod <pod-name>
Events:
FailedScheduling
Failed to pull image
Back-off restarting failed container
Unhealthy
FailedMount
Events often root cause identify cheyyadaniki first evidence.
Example:
Pod Pending
↓
describe
↓
FailedScheduling
↓
Insufficient memory
Then issue clear avvachu.
Events list:
kubectl get events --sort-by=.lastTimestamp
Simple ga:
Events = **Kubernetes resource ki recent ga em jarigindo troubleshooting clues.**

---

11. Pod logs check cheyyadaniki commands enti?
Ans:
Basic:
kubectl logs <pod-name>
Specific container:
kubectl logs <pod-name> -c <container-name>
Previous crashed container:
kubectl logs <pod-name> --previous
Follow live logs:
kubectl logs -f <pod-name>
Multiple replicas unna Deployment lo specific Pod first identify chesi logs check cheyyali.
Logs lo:
Application exceptions
Database connection errors
Config errors
Authentication errors
Startup failures
lanti information dorukutundi.
Simple ga:
kubectl logs = **Container/application behavior understand cheyyadaniki primary troubleshooting tool.**

---

12. Pod OOMKilled **ayindi. Meaning enti?**
Ans:
OOM = **Out Of Memory**.
Container memory usage available limit ni cross chesthe container OOMKilled avvachu.

Example:
resources:
 limits:
  memory: "512Mi"
Application usage:
Memory → 700Mi
Limit:
512Mi
Result:
Memory limit exceeded
↓
Container killed
↓
Restart
Check:
kubectl describe pod <pod-name>
and:
kubectl get pod <pod-name>
Possible fixes:
1. Investigate memory leak
2. Right-size memory limit
3. Review application behavior
4. Check memory requests
5. Check node memory pressure
Simple ga:
OOMKilled = **Container memory consumption limit/available memory constraints valla process killed ayindi.**

---

13. Liveness probe fail ayithe em jaruguthundi?
Ans:
Liveness Probe application **alive ga unda or stuck/dead state lo unda** ani Kubernetes ki indicate chestundi.
Repeated liveness failure:
Liveness fails
↓
Kubernetes considers container unhealthy
↓
Container restart
Example:
livenessProbe:
 httpGet:
  path: /health
  port: 8080
Application /health wrong response isthe liveness fail avvachu.
Troubleshoot:
kubectl describe pod <pod-name>
Check Events:
Liveness probe failed
Then application logs and probe configuration verify cheyyali.
Simple ga:
Liveness failure = **Kubernetes container unhealthy/stuck ani consider chesi restart cheyyachu.**

---

14. Readiness probe fail ayithe Pod restart avuthunda?
Ans:
Normally **No**.
Readiness probe purpose:
Pod traffic receive cheyyadaniki ready ga unda?
Readiness fail ayithe Pod:
Running
but
Not Ready
Service traffic nunchi Pod temporarily remove avvachu.
Flow:
Pod Running
↓
Readiness fails
↓
Pod NotReady
↓
Removed from Service endpoints
Liveness different:
Liveness fails
↓

Container restart
Simple ga:
Readiness
→ Traffic control

Liveness
→ Restart decision

---

15. Kubernetes Node NotReady **ayithe ela troubleshoot chestav?**
Ans:
First:
kubectl get nodes
Then:
kubectl describe node <node-name>
Check:
Conditions
Events
Taints
MemoryPressure
DiskPressure
PIDPressure
Kubelet-related status
Node-level checks access unte:
Kubelet
Container runtime
Disk
Memory
Network
verify cheyyali.
Typical reasons:
Kubelet stopped/unhealthy
Node network problem
Disk full
Memory pressure
Container runtime issue
Node unreachable
Example:
Node NotReady
↓
Kubelet unhealthy
↓
Node status not updated
↓
Pods impacted
Simple ga:
Node NotReady = **Node health + Kubelet + runtime + disk/memory/network + events** check cheyyali.

---

16. Kubernetes DNS issue vachindi. Ela troubleshoot chestav?
Ans:
Suppose Backend Pod:
backend → database
Database Service:
database
Backend:
database.default.svc.cluster.local
resolve avvatledu anuko.
First DNS resolution test kosam temporary/debug Pod use cheyyachu.
Check:
kubectl get pods -n kube-system
CoreDNS pods running unnaya chudali.
Then:
kubectl logs -n kube-system <coredns-pod>
Also check:
Service exists?
Service name correct?
Namespace correct?
Pod DNS configuration?
NetworkPolicy?
For example:
database
different namespace lo unte simply database resolve avvakapovachu depending on search domain; fully qualified name:

database.<namespace>.svc.cluster.local
use cheyyachu.
Simple ga:
DNS troubleshooting = **Service/name → namespace → CoreDNS → Pod DNS config → Network connectivity** check cheyyali.

---

17. Real-time scenario: Application works inside Pod, but outside nunchi access avvatledu. Ela troubleshoot chestav?
Ans:
Complete traffic path check chestha:
Internet/User
↓
LoadBalancer / Ingress
↓
Service
↓
Endpoints
↓
Pod
↓
Application
First:
kubectl get ingress
kubectl get svc
kubectl get endpoints
kubectl get pods
Then Service:
Service type correct?
Port correct?
TargetPort correct?
Selector correct?
Endpoints available?
Ingress:
Host correct?
Path correct?
IngressClass correct?
TLS correct?
Backend Service correct?
Pod:
Readiness?
ContainerPort?
Application listening address?
Also external LoadBalancer/cloud networking configuration check cheyyali.
Simple ga:
External access issue = **Ingress/LoadBalancer → Service → Endpoint → Pod → Application** path trace cheyyali.

---

18. Real-time scenario: Deployment rollout stuck ayindi. Ela troubleshoot chestav?
Ans:
First:
kubectl rollout status deployment/<deployment-name>
Then:
kubectl get pods
kubectl describe deployment <deployment-name>
Check:
New Pods create ayyaya?
Pods Ready avuthunnaya?
Old Pods terminate ayyaya?
Image pull problem unda?
Readiness probe fail avuthunda?
Insufficient resources?
ReplicaSets:
kubectl get rs
New ReplicaSet identify cheyyachu.
Suppose new Pods:
0/3 Ready
Then:
kubectl describe pod
kubectl logs
check chestha.
Possible reasons:
Bad image
Bad configuration
Readiness failure

Resource shortage
Scheduling issue
Application startup failure
Simple ga:
Rollout stuck = **Deployment → ReplicaSet → New Pods → Events → Logs → Readiness** chain troubleshoot cheyyali.

---

19. Real-time scenario: Service exists, Pods Running, but users ki 503 **vastondi. Root cause ela find chestav?**
Ans:
503 often means traffic path lo backend healthy/available ga lekapovachu, but exact cause architecture batti vary avuthundi.
First:
kubectl get ingress
kubectl get svc
kubectl get endpoints
kubectl get pods
Check:
Service has endpoints?
Pods Ready?
Ingress backend correct?
Important:
Running ≠ Ready
Suppose:
Pods:
Running
0/1 Ready
Service endpoints empty/absent avvachu, depending on readiness state.
Then:
kubectl describe pod <pod>
Check readiness probe.
If endpoints exist:
Check targetPort
Application listening port
Ingress backend
NetworkPolicy
Application logs
Flow:
503
↓
Ingress?
↓
Service?
↓
Endpoints?
↓
Ready Pods?
↓
Application?
Simple ga:
503 troubleshooting lo **"Pods Running unnayi"** ani stop avvakunda **Ready status + endpoints + Service/Ingress path** verify cheyyali.

---

20. Real-time interview scenario: Production application completely down. Kubernetes lo systematic troubleshooting approach enti?
Ans:
Production outage lo random commands run cheyyakunda **layer-by-layer** troubleshoot cheyyali.
Step 1 — Scope identify cheyyali
One Pod?
One Node?
One Service?
One Namespace?
Entire application?
Entire cluster?
Step 2 — High-level status
kubectl get nodes
kubectl get pods -A
kubectl get svc -A
Step 3 — Application Pods
kubectl get pods -n production -o wide
Look for:
Pending
CrashLoopBackOff
ImagePullBackOff

OOMKilled
NotReady
Step 4 — Events
kubectl get events -n production --sort-by=.lastTimestamp
Step 5 — Logs
kubectl logs <pod>
kubectl logs <pod> --previous
Step 6 — Service path
Ingress
↓
Service
↓
Endpoints
↓
Ready Pods
Verify:
kubectl get ingress -n production
kubectl get svc -n production
kubectl get endpoints -n production
Step 7 — Configuration
Check:
ConfigMap
Secret
Environment variables
Image version
Step 8 — Resources
Check:
CPU
Memory
Requests
Limits
Node pressure
Step 9 — Nodes
kubectl get nodes
kubectl describe node <node-name>
Check:
NotReady
MemoryPressure
DiskPressure
Kubelet
Runtime
Step 10 — Recent deployment/change
Interview lo very important question:
"What changed just before the outage?"
Check:
Deployment rollout
Helm upgrade
Image change
ConfigMap/Secret change
Ingress change
Node maintenance
NetworkPolicy change
If recent bad deployment clearly identified and rollback is safe/approved:
kubectl rollout undo deployment/<deployment-name>
or Helm-managed release ayithe:
helm history <release>
helm rollback <release> <revision>
Then verify:
Pods healthy?
Readiness passing?
Service endpoints available?
Ingress responding?
Application working?
Finally root cause document cheyyali.
Complete troubleshooting flow:
User reports outage
↓
Define scope
↓
Check Nodes
↓

Check Pods
↓
Check Events
↓
Check Logs
↓
Check Deployment/ReplicaSet
↓
Check Service/Endpoints
↓
Check Ingress/DNS
↓
Check Config/Secrets
↓
Check Resources
↓
Check recent changes
↓
Fix / Rollback
↓
Verify
↓
Root Cause Analysis

Interview lo strong answer:

"I don't start changing resources immediately. First I identify the scope and collect evidence using kubectl get, describe, logs, and Events. Then I trace the request path from Ingress or LoadBalancer to Service, Endpoints and Pods. In parallel I check scheduling, resources, configuration, probes, node health and recent changes. Once I identify the root cause, I apply the smallest safe fix or rollback, verify recovery, and then perform RCA."

Simple ga:

Kubernetes Troubleshooting golden rule:

Get → Describe → Events → Logs → Trace the traffic path → Find root cause → Fix → Verify

---

23. Real-time / Interview Scenarios — 20 Questions

1. Production lo oka Pod CrashLoopBackOff **lo undi. As a DevOps engineer first em chestav?**

Ans:

First direct ga Pod delete cheyyanu. Mundu **root cause identify** chestha.

First:

kubectl get pod <pod-name> -n production

Tarvata:

kubectl describe pod <pod-name> -n production

Events check chestha.

Application logs:

kubectl logs <pod-name> -n production

Previous crashed container logs:

kubectl logs <pod-name> -n production --previous

Check cheyyalsinavi:

Image correct aa?

ConfigMap/Secret correct aa?

Application startup fail avuthunda?

Liveness probe issue aa?

Memory OOMKilled aa?

Wrong command/arguments aa?

Dependency unavailable aa?

Flow:

CrashLoopBackOff
↓
Describe
↓
Events
↓
Logs
↓
Previous logs
↓
Root cause
↓
Fix
↓

Verify
Interview lo:
"I first collect evidence using describe, events and logs instead of immediately restarting or deleting the Pod."

---

2. Deployment lo 3 replicas configured unnayi, but only 1 Pod running undi. Ela investigate chestav?
Ans:
First:
kubectl get deployment
kubectl get rs
kubectl get pods
Suppose:
Deployment desired = 3
Available = 1
Then Deployment:
kubectl describe deployment <deployment-name>
ReplicaSet:
kubectl describe rs <replicaset-name>
Remaining Pods enduku create avvatledu ani Events check chestha.
Possible reasons:
Insufficient CPU/Memory
Taints
Node affinity
ImagePullBackOff
CrashLoopBackOff
Readiness failure
Resource quota
Example:
Deployment → 3 replicas
↓
ReplicaSet → tries to create 3 Pods
↓
2 Pods → Pending
↓
Insufficient memory
Then eligible Nodes/resource capacity check chestha.
Simple ga:
Deployment problem ani assume cheyyakunda **Deployment → ReplicaSet → Pods → Scheduling/Events** chain follow chestha.

---

3. Pod Pending **lo undi and Events lo** Insufficient memory **ani vastondi. What will you do?**
Ans:
Meaning scheduler ki Pod request satisfy cheyyagal Node available ga ledu.
Example:
Pod memory request = 2Gi

Node 1 available = 500Mi
Node 2 available = 1Gi
Pod schedule avvadu.
First:
kubectl describe pod <pod-name>
kubectl get nodes
Then Node resource availability check chestha.
Possible solutions:
Add/scale Nodes
Reduce incorrect resource requests
Move workloads
Check other workloads consuming resources
Use appropriate autoscaling
But blindly request reduce cheyyanu. Application actual memory requirement understand chesi change chestha.
Interview answer:
"I would verify whether the request is correctly sized, inspect node allocatable capacity and existing workloads, then either scale the cluster or right-size the workload."

---

4. Application Pods Running **lo unnayi, but users ki application open avvatledu. Ela troubleshoot chestav?**
Ans:
Running alone enough kaadu.
Traffic path:
User
↓
Ingress / LoadBalancer
↓
Service

↓
Endpoints
↓
Ready Pod
↓
Application
First:
kubectl get ingress
kubectl get svc
kubectl get endpoints
kubectl get pods
Check:
Pods Ready aa?
Service selector correct aa?
Endpoints unnaya?
Service port correct aa?
TargetPort correct aa?
Ingress backend correct aa?
Application correct port lo listen chestunda?
Logs:
kubectl logs <pod-name>
Simple ga:
User issue ni troubleshoot chestappudu **request path complete ga trace cheyyali.**

---

5. Service create chesavu, but Service ki endpoints levu. What is the likely issue?
Ans:
Most common reason **Service selector and Pod labels mismatch**.
Service:
selector:
app: backend
Pod:
labels:
app: frontend
So:
Service selector
↓
app=backend

Pod labels
↓
app=frontend

No match
↓
No endpoints
Check:
kubectl get svc <service-name> -o yaml
kubectl get pods --show-labels
kubectl get endpoints <service-name>
Also Pods Ready status check chestha.
Interview lo:
"I would verify the Service selector against Pod labels first, and then verify whether matching Pods are Ready."

---

6. Deployment upgrade chesaka new Pods start avvatledu. How do you troubleshoot?
Ans:
First rollout status:
kubectl rollout status deployment/<deployment-name>
Then:
kubectl get rs
kubectl get pods
New ReplicaSet identify chestha.
Then:
kubectl describe deployment <deployment-name>
kubectl describe pod <new-pod>
Events lo:
FailedScheduling
Failed to pull image
FailedMount
Unhealthy
lanti messages check chestha.
Logs kuda check chestha if container started.

Common causes:
Wrong image
Wrong tag
Bad ConfigMap/Secret
Resource shortage
Probe failure
Volume issue
Scheduling constraints
Simple ga:
Upgrade issue lo **new ReplicaSet and new Pods** meeda focus cheyyali.

---

7. Production deployment fail ayindi. Previous version healthy ga undi. Immediate action enti?
Ans:
First deployment status and history confirm chestha.
For Deployment:
kubectl rollout history deployment/<deployment-name>
Previous revision known-good and current deployment causing outage ayithe:
kubectl rollout undo deployment/<deployment-name>
Then:
kubectl rollout status deployment/<deployment-name>
Verify:
kubectl get pods
If Helm-managed:
helm history <release>
helm rollback <release> <revision>
Rollback tarvata root cause investigate cheyyali.
Important:
Rollback is **recovery action**, not root-cause fix.

---

8. Pod restart count continuously increase avuthondi. What will you check?
Ans:
First:
kubectl get pod <pod-name>
Restart count observe chestha.
Then:
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
Check:
Application crash
OOMKilled
Liveness probe
Exit code
Config issue
Dependency failure
Example:
Restart Count → 20
Reason → OOMKilled
Then memory usage/limits investigate chestha.
Another example:
Restart Count → 15
Events → Liveness probe failed
Then probe configuration/application health endpoint inspect chestha.
Simple ga:
High restart count = **why container is exiting** ane question answer cheyyali.

---

9. Pod OOMKilled **ayindi. Production lo em chestav?**
Ans:
First confirm:
kubectl describe pod <pod-name>
Memory limits:

```
resources:
  limits:
    memory: "512Mi"
```

Application actual behavior analyze chestha.
Possible reasons:
Memory leak
Unexpected traffic
Too-small memory limit
Large workload
Incorrect JVM/runtime settings

Immediate safe mitigation:
Increase capacity if justified
Scale replicas if workload allows
But permanent fix:
Application memory investigation
Right-sizing
Load testing
Memory leak fix
Interview lo:
"I wouldn't simply increase the limit. I would determine whether the application is genuinely under-provisioned or leaking memory."

---

10. Node NotReady **ayindi and several Pods are affected. What is your approach?**
Ans:
First:
kubectl get nodes
kubectl describe node <node-name>
Check:
MemoryPressure
DiskPressure
PIDPressure
Conditions
Events
Taints
If node access available:
Kubelet
Container runtime
Disk
Memory
Network
check chestha.
Affected Pods:
kubectl get pods -o wide
Node meeda unna workloads identify chestha.
If node failure is serious and cluster has enough capacity, workloads may be rescheduled depending on controller and node condition.
Simple ga:
Node NotReady
↓
Node health
↓
Kubelet/runtime
↓
Resources
↓
Affected Pods
↓
Cluster capacity

---

11. HPA configured undi, but replicas increase avvatledu. Ela troubleshoot chestav?
Ans:
First:
kubectl get hpa
kubectl describe hpa <hpa-name>
Check:
Current replicas
Desired replicas
CPU/Memory metrics
Min replicas
Max replicas
Example:
Current CPU = 90%
Max replicas = 5
Current replicas = 2
But still 2 replicas unte HPA events check cheyyali.
Also:
kubectl get deployment
Metrics available aa ani verify cheyyali.
Resource requests important, especially CPU utilization based HPA:

```
resources:
  requests:
    cpu: "200m"
```

Without appropriate requests, utilization-based behavior may not work as expected.
Simple ga:
HPA issue = **HPA status → metrics → resource requests → target/max replicas → Events**.

---

12. ConfigMap update chesavu, but existing application Pods old configuration use chestunnayi. Why?
Ans:
ConfigMap ni environment variable ga inject chesunte, **existing container environment automatically update avvadu**.
Example:

```
env:
 - name: APP_MODE
   valueFrom:
    configMapKeyRef:
     name: app-config
     key: mode
```

ConfigMap change ayina:
Existing Pod
↓
Old environment variable
new Pod create ayithe:
New Pod
↓
New environment value
Kabatti configuration change tarvata application restart/rollout required avvachu.
Deployment:
kubectl rollout restart deployment/<deployment-name>
Mounted ConfigMap files behavior different ga untundi; kubelet eventually updates mounted content, but application itself file changes ni automatically reload chestundi ani guarantee kaadu.
Simple ga:
ConfigMap update ≠ application automatically reload.

---

13. Secret accidentally wrong value tho deploy ayindi. Application fail avuthondi. What will you check?
Ans:
First application error identify chestha:
kubectl logs <pod-name>
Then Secret reference verify chestha:
kubectl describe pod <pod-name>
Check:
Correct Secret name?
Correct key?
Correct namespace?
Secret mounted correctly?
Environment variable correct aa?
Application expecting correct format aa?
Important:
Secret data ni unnecessarily terminal/logs lo print cheyyakudadhu.
Fix Secret through secure process, then restart/rollout application if required.
Example:
Secret
↓
Deployment
↓
Pod
↓
Environment / Volume
↓
Application
Ee chain lo ekkada mismatch undo identify cheyyali.
Simple ga:
Secret issue lo **reference + key + namespace + application expectation** verify cheyyali.

---

14. Ingress configured correctly anipistundi, but domain access avvatledu. What will you check?
Ans:
Complete flow:
DNS
↓
LoadBalancer
↓
Ingress Controller
↓
Ingress Rule
↓

Service
↓
Pod
First:
kubectl get ingress
kubectl describe ingress <ingress-name>
Check:
Host
Path
IngressClass
TLS
Backend Service
Backend Port
Then:
kubectl get svc
kubectl get endpoints
kubectl get pods
DNS externally resolve avuthunda kuda check cheyyali.
Possible issues:
DNS wrong
LoadBalancer issue
IngressClass mismatch
Host mismatch
TLS certificate issue
Service port wrong
No endpoints
Pod not Ready
Simple ga:
Ingress problem ni only Ingress YAML problem ga chudakunda **DNS → LB → Ingress → Service → Pod** trace cheyyali.

---

15. Database Pod ki application connect avvatledu. Ela troubleshoot chestav?
Ans:
First database Pod healthy aa:
kubectl get pods
Database Service:
kubectl get svc
kubectl get endpoints
Then application side nunchi connectivity test chestha.
Check:
Database Service name
Namespace
Port
TargetPort
Credentials
Secret
NetworkPolicy
DNS
Database listening port
Flow:
Application Pod
↓
DNS
↓
Database Service
↓
Database Endpoint
↓
Database Pod
Suppose Service:
db.default.svc.cluster.local:5432
but application configured:
db:3306
Then connection fail avvachu.
Simple ga:
Database connectivity = **DNS + Service + Port + Endpoint + Credentials + NetworkPolicy** check.

---

16. NetworkPolicy apply chesaka application-to-database communication stop ayindi. How do you troubleshoot?
Ans:
First NetworkPolicy list:
kubectl get networkpolicy
Then:

kubectl describe networkpolicy <policy-name>
Check:
PodSelector
NamespaceSelector
Ingress rules
Egress rules
Ports
Suppose database ki only:
app=backend
Pods nunchi traffic allowed.
But backend Pods label wrong ayithe:
Policy expects → app=backend
Pod has → app=api
Traffic deny avvachu.
Flow:
Backend Pod
↓
NetworkPolicy evaluation
↓
Allowed?
↓
Database Service
Simple ga:
NetworkPolicy issue lo **source Pod labels + destination selector + ports + ingress/egress rules** verify cheyyali.

---

17. Rolling update tarvata old and new Pods rendu run avuthunnayi. Is this a problem?
Ans:
Not necessarily. **RollingUpdate strategy lo temporary ga old and new Pods simultaneously run avvadam normal.**
Example:
Old version → v1
New version → v2
During rollout:
v1 Pods + v2 Pods
simultaneously run avvachu.
Kubernetes maxSurge and maxUnavailable settings rollout behavior control chestayi.
Example:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

Meaning rollout time lo extra Pod create cheyyachu, while maintaining availability.
Problem eppudu?
Old Pods never terminate
New Pods never become Ready
Replica count unexpected
Rollout stuck
Simple ga:
RollingUpdate lo old + new Pods temporary ga coexist avvadam **normal behavior**.

---

18. Production lo traffic suddenly increase ayindi. Kubernetes application ni automatically scale cheyyali. What will you use?
Ans:
Application stateless ayithe usually **HPA** use chestam.
Flow:
Traffic ↑
↓
CPU/Memory/Custom metric ↑
↓
HPA detects
↓
Desired replicas ↑
↓
Deployment creates Pods
Example:
Normal traffic:
3 Pods

High traffic:
8 Pods
HPA configuration lo:
minReplicas

maxReplicas
target metric
define chestam.
But HPA alone enough kaakapovachu.
If cluster itself capacity saripokapothe:
HPA
↓
More Pods requested
↓
No Node capacity
↓
Pods Pending
Appudu cluster autoscaling/node scaling also required.
Interview lo:
"For a stateless workload I'd use HPA, but I'd also ensure the cluster has enough node capacity for the additional replicas."

---

19. Company lo 100+ microservices unnayi. Each application ki manually YAML maintain cheyyadam difficult. What would you recommend?
Ans:
Common Kubernetes configurations reusable ga manage cheyyadaniki **Helm** or another templating/configuration approach use cheyyachu.
Helm example:
Common Chart
↓
Values
┌──┼──┐
↓ ↓ ↓
Dev QA Prod
Common templates:
Deployment
Service
ConfigMap
Ingress
Environment-specific values:
values-dev.yaml
values-qa.yaml
values-prod.yaml
Benefits:
Reusable templates
Versioning
Environment-specific configuration
Upgrade
Rollback
Release history
CI/CD integration tho:
Git
↓
Validation
↓
Helm package/render
↓
Deploy
↓
Monitor
Simple ga:
Large microservices environment lo **reusable, versioned deployment templates + environment-specific configuration** maintain cheyyadam important.

---

20. Interview scenario: "Your production Kubernetes application is down. Explain exactly how you would troubleshoot it."
Ans:
Idi very important interview question bro. Answer random commands laga kakunda **systematic approach** lo cheppali.
First, **impact and scope** identify chestha:
Entire application down aa?
One service aa?
One Pod aa?
One Node aa?
One namespace aa?
Then cluster health:
kubectl get nodes
kubectl get pods -A
Application namespace lo:

kubectl get pods -n production
kubectl get svc -n production
kubectl get ingress -n production
Pod states check chestha:
Pending
CrashLoopBackOff
ImagePullBackOff
OOMKilled
NotReady
Problematic Pod dorikithe:
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production
kubectl logs <pod-name> -n production --previous
Events:
kubectl get events -n production --sort-by=.lastTimestamp
Then traffic path trace chestha:
User
↓
DNS
↓
LoadBalancer
↓
Ingress
↓
Service
↓
Endpoints
↓
Ready Pod
↓
Container
↓
Application
Configuration check:
ConfigMap
Secret
Environment variables
Volumes
Resources check:
CPU
Memory
Requests
Limits
Node capacity
Recent changes check cheyyadam **very important**:
Deployment upgrade?
Helm upgrade?
New image?
ConfigMap change?
Secret change?
Ingress change?
NetworkPolicy change?
Node maintenance?
If recent deployment clearly caused outage and rollback is approved:
kubectl rollout undo deployment/<deployment-name>
or Helm:
helm history <release>
helm rollback <release> <revision>
Then recovery verify chestha:
kubectl rollout status deployment/<deployment-name>
kubectl get pods
kubectl get endpoints
Finally:
Root Cause Analysis
↓
Permanent Fix
↓
Testing
↓
Monitoring/Alert improvement
↓

Post-incident documentation

Interview lo complete answer ila cheppachu:

"First I would determine the scope and impact instead of making immediate changes. Then I would check node and workload health using kubectl get, inspect problematic resources with describe, review Events and application logs, and trace the request path from DNS/Ingress through Service and Endpoints to the Pods. I would also check configuration, resources, probes and recent changes such as deployments or Helm upgrades. If a recent change is confirmed as the cause and rollback is safe, I would roll it back, verify recovery, and then perform root-cause analysis and implement a permanent fix."

Golden interview formula:

Identify
↓
Collect Evidence
↓
Check Events
↓
Check Logs
↓
Trace Traffic
↓
Check Resources/Config
↓
Check Recent Changes
↓
Fix / Rollback
↓
Verify
↓
RCA

🔥 **Ippudu nee Kubernetes core topics complete:** Architecture → Pods → Deployment/ReplicaSet → Services/Networking → Labels/Selectors → Namespaces → ConfigMap/Secret → Volumes/Storage → Requests/Limits → Probes → Deployment Strategies → HPA → DaemonSet → StatefulSet → Job/CronJob → Ingress → RBAC/ServiceAccount → Taints/Tolerations → Affinity → Scheduling → Helm → Troubleshooting → Real-time Scenarios.
